

Mobile Applications Development



COMPUTER SYSTEMS ENGINEERING

LECTURER: AHMAD SHAQALIEH

SECOND SEMESTER 2025

LECTURE 1

A solid orange horizontal bar spanning the width of the slide at the bottom.

What Is Mobile Computing?

What is **computing**?

Operation of computers (according to oxfords advance learner's dictionary)

What is the **mobile**?

That someone /something can move or be moved easily and quickly from place to place

What is mobile computing?

Users with portable computers still have network connections while they move

Mobile Computing Definitions

A simple definition could be: Mobile Computing is using a computer (of one kind or another) while on the move

Another definition could be: Mobile Computing is when a work process is moved from a normal fixed position to a more dynamic position

A third definition could be: Mobile Computing is when a work process is carried out somewhere where it was not previously possible

Mobile Computing is an umbrella term used to describe technologies that enable people to access services **anytime**, and **anywhere**



Mobile Computing

Mobile computing is the act of interacting with a computer through the use of a mobile device.

Mobile computing has three main components which are **mobile hardware, mobile software, and mobile communication.**

A solid orange horizontal bar spanning the width of the slide, located at the bottom.

Why Go Mobile?

Enable anywhere/anytime connectivity

Bring computer communications to areas without pre-existing infrastructure

Enable mobility

Enable new applications

An exciting new research area

A solid orange horizontal bar spanning the width of the slide, located at the bottom.

Evolution (Seven waves of mobile computing)

The history of mobile computing can be divided into a number of eras, or waves, each characterized by a particular technological focus, interaction design trends, and by leading to fundamental changes in the design and use of mobile devices.

Mobile computing history has, so far, entailed seven particularly important waves.

Although not strictly sequential, they provide a good overview of the legacy on which current mobile computing research and design is built.

These waves are the basis for the technology that is used today in research and design of mobile computing

These seven categories are: Portability, Miniaturization, Connectivity, Convergence, Divergence, Apps, Digital Ecosystems

Reference: <https://www.interaction-design.org/literature/book/the-encyclopedia-of-human-computer-interaction-2nd-ed/mobile-computing>

Portability

Reducing the size of hardware to enable the creation of computers that could be physically moved around relatively easily.



Miniaturization

Creating new and significantly smaller mobile form factors that allowed the use of personal mobile devices while on the move



50mm x 50mm



35mm x 35mm



15mm x 15mm

Connectivity

Developing devices and applications that allowed users to be online and communicate via wireless data networks while on the move



Bluetooth®

Convergence

Integrating emerging types of digital mobile devices, such as Personal Digital Assistants (PDAs), mobile phones, music players, cameras, games, etc., into hybrid devices



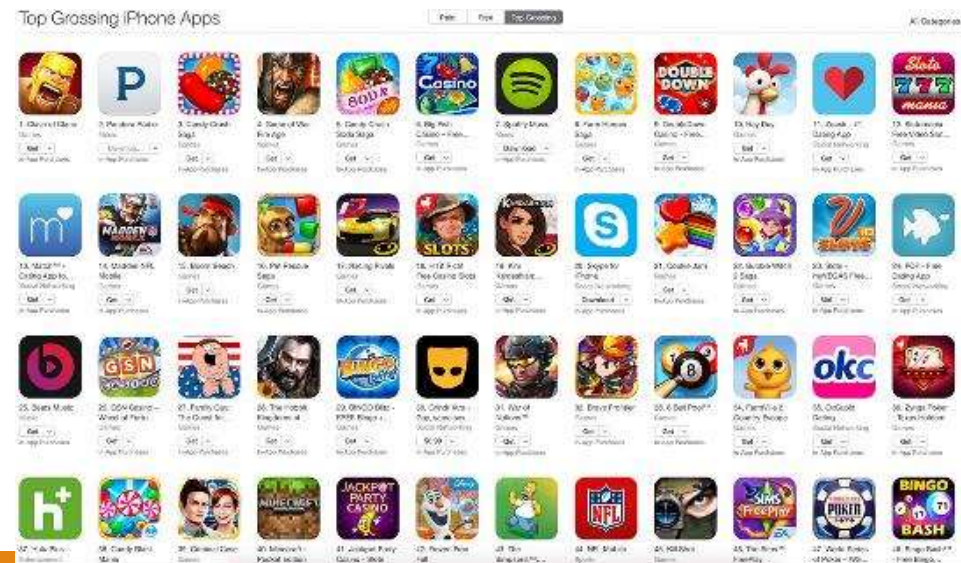
Divergence

Opposite approach to interaction design by promoting information appliances with specialized functionality rather than generalized ones



Applications (Apps)

The latest wave of applications (*apps*) is about developing matter and substance for use and consumption on mobile devices, and making access to this fun or functional interactive application content easy and enjoyable



Digital Ecosystems

The emerging wave of *digital ecosystems* is about the larger wholes of pervasive and interrelated technologies that interactive mobile systems are increasingly becoming a part of



Example: Smartphone

Portability: fit in your pocket

Miniaturization: make it possible to build device to fit in your pocket

Connectivity: Wi-Fi, LTE/4G, cellular, Bluetooth

Convergence: phone, camera, gaming device, movie streaming, music player, ...

Divergence: ?

Applications: “Rise of the Apps”

Digital Ecosystem: social networks, distributed gaming, video streaming, work apps, ...

A solid orange horizontal bar spanning the width of the slide at the bottom.

Flutter Framework



Flutter

What is Flutter?

In general, creating a mobile application is a very complex and challenging task.

There are many frameworks available, which provide excellent features to develop mobile applications.

For developing mobile apps, Android provides a native framework based on Java and Kotlin language, while iOS provides a framework based on Objective-C/Swift language.

Thus, we need two different languages and frameworks to develop applications for both OS.

Today, to overcome from this complexity, there are several frameworks have introduced that support both OS along with desktop apps.

These types of the framework are known as **cross-platform** development tools.



The cross-platform development framework has the ability to write one code and can deploy on the various platform (Android, iOS, and Desktop).

It saves a lot of time and development efforts of developers.

There are several tools available for cross-platform development, such as Ionic from Drifty Co. in 2013, Phonegap from Adobe, Xamarin from Microsoft, and React Native from Facebook.

In recent, a new framework has introduced in the cross-platform development family named **Flutter** developed from Google.



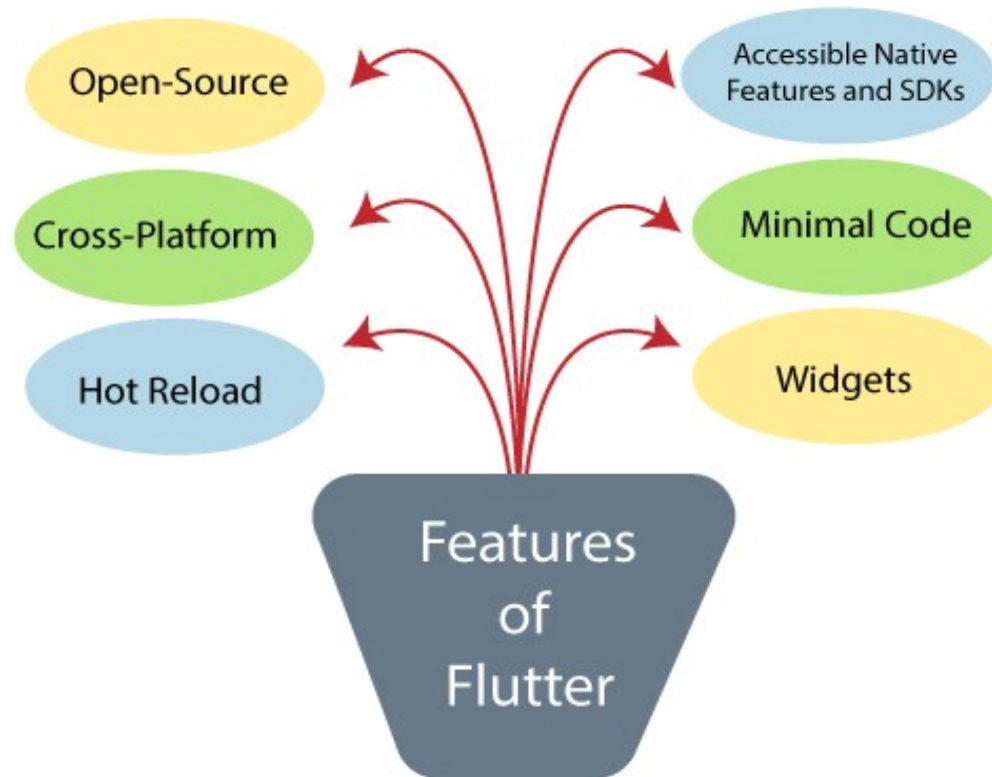
Flutter is a UI toolkit for creating fast, beautiful, natively compiled applications for mobile, web, and desktop with one programming language and single codebase. It is free and open-source. It was initially developed from **Google**.

Flutter apps use Dart programming language for creating an app.

The **dart programming** shares several same features as other programming languages, such as Kotlin and Swift, and can be trans-compiled into JavaScript code.

A solid orange horizontal bar spanning the width of the slide at the bottom.

Features of Flutter



Open-Source: Flutter is a free and open-source framework for developing mobile applications.

Cross-platform: This feature allows Flutter to write the code once, maintain, and can run on different platforms. It saves the time, effort, and money of the developers.

Hot Reload: Whenever the developer makes changes in the code, then these changes can be seen instantaneously with Hot Reload. It means the changes immediately visible in the app itself. It is a very handy feature, which allows the developer to fix the bugs instantly.

Accessible Native Features and SDKs: This feature allows the app development process easy and delightful through Flutter's native code, third-party integration, and platform APIs. Thus, we can easily access the SDKs on both platforms.



Minimal code: Flutter app is developed by Dart programming language, which uses JIT and AOT compilation to improve the overall start-up time, functioning and accelerates the performance. JIT enhances the development system and refreshes the UI without putting extra effort into building a new one.

Widgets: The Flutter framework offers widgets, which are capable of developing customizable specific designs. Most importantly, Flutter has two sets of widgets: Material Design and Cupertino widgets that help to provide a glitch-free experience on all platforms.



Advantage of Flutter

It makes the app development process extremely fast because of the hot-reload feature.

It provides the smoother and seamless scrolling experiences of using the app without much hangs or cuts, which makes running applications faster in comparison to other mobile app development frameworks.

Flutter reduces the time and efforts of testing. As we know, flutter apps are cross-platform so that testers do not always need to run the same set of tests on different platforms for the same app.

It has an excellent user interface because it uses a design-centric widget, high-development tools, advanced APIs, and many more features.



It is similar to a reactive framework where the developers do not need to update the UI content manually

It is suitable for MVP (Minimum Viable Product) apps because of its speedy development process and cross-platform nature.

Disadvantages of Flutter

The Flutter is a comparatively new language that needs continuous integration support through the maintenance of scripts.

It provides very limited access to SDK libraries. It means a developer does not have a lot of functionalities to create a mobile application. Such types of functionalities need to be developed by the Flutter developer themselves.

The Flutter apps do not support the browser. It only supports Android and iOS platforms.

It uses Dart programming for coding, so a developer needs to learn new technologies. However, it is easy to learn for developers.



Flutter Installation

System requirements for Windows

Operating System	Windows 7 or Later (I am Windows 10. You can also use Mac or Linux OS.).
Disk Space	400 MB (It does not include disk space for IDE/tools).
Tools	1. Windows PowerShell 2. Git for Windows 2.x (Here, Use Git from Windows Command Prompt option).
SDK	Flutter SDK for Windows
IDE	Android Studio (Official)

System Requirements for macOS

Operating System	macOS (64-bit)
Disk Space	2.8 GB (It does not include disk space for IDE/tools).
Tools	bash curl git 2.x mkdir rm unzip which
IDE	Xcode (Official)

Difference between Flutter and React native

Concept	Flutter	React Native
Develop By	It is first introduced by Google.	It is first introduced by Facebook.
Release	May 2017	June 2015
Programming Language	It uses Dart language to create a mobile app.	It uses JavaScript to create mobile apps.
Architecture	Flutter uses Business Logic Component (BLoC) architecture.	React Native uses Flux and Redux architecture. Flux created by Facebook, whereas Redux is the preferred choice among the community.

User Interface	It uses custom widgets to build the UI of the app.	It uses native UI controllers to create UI of the app.
Documentation	Flutter documentation is good, organize, and more informative. We can get everything that we want to be written in one place.	React native documentation is user-friendly but disorganized.
Performance	The performance of the Flutter application is fast. Flutter compiles the application by using the arm C/C++ library that makes it closer to machine code and gives the app a better native performance.	The performance of the React Native app is slow in comparison to the Flutter app. Here, sometimes developers face issues while running the hybrid application architecture.

Testing	Flutter provides a very rich set of testing features. This feature allows the developer to perform unit testing, integration testing, and widget testing.	React Native uses third-party tools that are available for testing the app.
Community Support	It has less community support as compared to React Native.	It has very strong community support where the questions and issues can be solved quickly.
Hot Reload	Supported	Supported

Popularity	81200 stars on GitHub (December 2019)	83200 stars on GitHub (December 2019)
Latest Version	Flutter-v1.12.13	React Native-v0.61.0
Industry Adoption	Google Ads Hamilton Reflectly Xianyu	Facebook Instagram LinkedIn Skype

Flutter Architecture

The Flutter architecture mainly comprises of four components.

- ❑ Flutter Engine
- ❑ Foundation Library
- ❑ Widgets
- ❑ Design Specific Widgets

Flutter Engine

It is a portable runtime for high-quality mobile apps and primarily based on the C++ language. It implements Flutter core libraries that include animation and graphics, file and network I/O, plugin architecture, accessibility support, and a dart runtime for developing, compiling, and running Flutter applications. It takes Google's open-source graphics library, **Skia**, to render low-level graphics.

Foundation Library

It contains all the required packages for the basic building blocks of writing a Flutter application. These libraries are written in Dart language.



Widgets

In Flutter, everything is a widget, which is the core concept of this framework.

Widget in the Flutter is basically a user interface component that affects and controls the view and interface of the app.

It represents an immutable description of part of the user interface and includes graphics, text, shapes, and animations that are created using widgets.

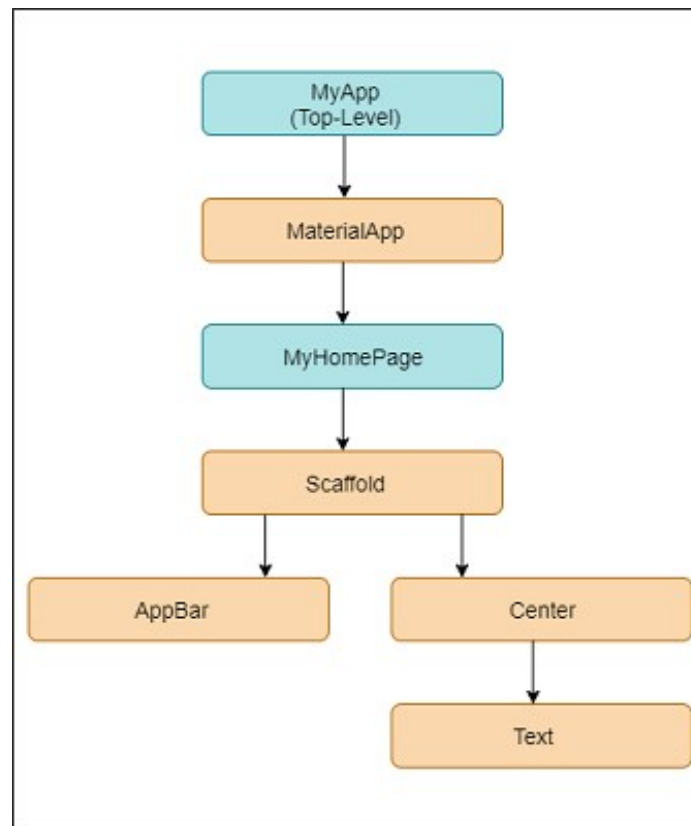
The widgets are similar to the React components.

In Flutter, the application is itself a widget that contains many sub widgets.

It means the app is the top-level widget, and its UI is build using one or more children widgets, which again includes sub child widgets.

This feature helps you to create a complex user interface very easily.





we can see that all the components are widgets that contain child widgets. Thus, the Flutter application is itself a widget.

Design Specific Widgets

The Flutter framework has two sets of widgets that conform to specific design languages. These are Material Design for Android application and Cupertino Style for IOS application.



Gestures

It is a widget that provides interaction (how to listen for and respond to) in Flutter using `GestureDetector`.

`GestureDetector` is an invisible widget, which includes tapping, dragging, and scaling interaction of its child widget.

We can also use other interactive features into the existing widgets by composing with the **`GestureDetector`** widget.

State Management

Flutter widget maintains its state by using a special widget, `StatefulWidget`.

It is always auto re-rendered whenever its internal state is changed.

The re-rendering is optimized by calculating the distance between old and new widget UI and render only necessary things that are changes.

Layers

Layers are an important concept of the Flutter framework, which are grouped into multiple categories in terms of complexity and arranged in the top-down approach.

The topmost layer is the UI of the application, which is specific to the Android and iOS platforms.

The second topmost layer contains all the Flutter native widgets. The next layer is the rendering layer, which renders everything in the Flutter app.

Then, the layers go down to Gestures, foundation library, engine, and finally, core platform-specific code.

The following diagram specifies the layers in Flutter app development.

Flutter System Overview

Framework
Dart

Material

Cupertino

Widgets

Rendering

Animation

Painting

Gestures

Foundation

Engine
C/C++

Service Protocol

Composition

Platform Channels

Dart Isolate Setup

Rendering

System Events

Dart VM Management

Frame Scheduling

Asset Resolution

Frame Pipelining

Text Layout

Embedder
Platform Specific

Render Surface Setup

Native Plugins

Packaging

Thread Setup

Event Loop Interop

Dart Programming

What is Dart Programming

Dart is an open-source, general-purpose, object-oriented programming language with C-style syntax developed by **Google in 2011**.

The purpose of Dart programming is to create a frontend user interfaces for the web and mobile apps.

It is under active development, compiled to native machine code for building mobile apps, inspired by other programming languages such as Java, JavaScript, C#, and is Strongly Typed.

Since Dart is a compiled language so you cannot execute your code directly; instead, the compiler parses it and transfer it into machine code.

It supports most of the common concepts of programming languages like classes, interfaces, functions, unlike other programming languages.

Dart language does not support arrays directly.



It supports collection, which is used to replicate the data structure such as arrays, generics, and optional typing.

The following example shows simple Dart programming.

```
void main() {  
  for (int i = 0; i < 5; i++) {  
    print('hello ${i + 1}');  
  }  
}
```

Data Type

Dart is a Strongly Typed programming language.

It means, each value you use in your programming language has a type either string or number and must be known when the code is compiled.

Here, we are going to discuss the most common basic data types used in the Dart programming language.

Data Type	Example	Descriptions
String	<code>String myName = 'javatpoint';</code>	It holds text. In this, you can use single or double quotation marks. Once you decide the quotation marks, you should have to be consistent with your choice.

num, int, double	int age = 25; double price = 125.50;	The num data type stands for a number. Dart has two types of numbers: ◦ Integer (It is a number without a decimal place.) ◦ Double (It is a number with a decimal place.)
Boolean	bool var_name = true; Or bool var_name = false;	It uses the bool keyword to represents the Boolean value true and false.
object	Person = Person()	Generally, everything in Dart is an object (e.g., Integer, String). But an object can also be more complex.

Variables and Functions

Variables are the namespace in memory that stores values.

The name of a variable is called as **identifiers**.

They are the data containers, which can store the value of any type. For example:

```
var myAge = 50;
```

Here, **myAge** is a variable that stores an integer value 50.

We can also give it int and double.

Dart has a feature Type Inference, which infers the types of values. So, if you create a variable with a **var** keyword, Dart can infer that variable as of type integer.

Besides variable, Functions are another core feature of any programming language.

Functions are a set of statements that performs a specific task.

They are organized into the logical blocks of code that are readable, maintainable, and reusable.

The function declaration contains the function name, return type, and parameters.

The following example explains the function used in Dart programming.

A solid orange horizontal bar spanning the width of the slide at the bottom.

```
//Function declaration
```

```
num addNumbers(num a, num b) {
```

```
    // Here, we use num as a type because it should work with int and double both.
```

```
    return a + b;
```

```
}
```

```
var price1 = 29.99;
```

```
var price2 = 20.81;
```

```
var total = addNumbers(price1, price2);
```

```
var num1 = 10;
```

```
var num2 = 45;
```

```
var total2 = addNumbers(num1, num2);
```

Operators

Dart language supports all operators, as you are familiar with other programming languages such as C, Kotlin, and Swift. The operator's name is listed below:

- ❑ Arithmetic
 - ❑ Equality
 - ❑ Increment and Decrement
 - ❑ Logical
 - ❑ Comparison
- 
- A solid orange horizontal bar spanning the width of the slide at the bottom.

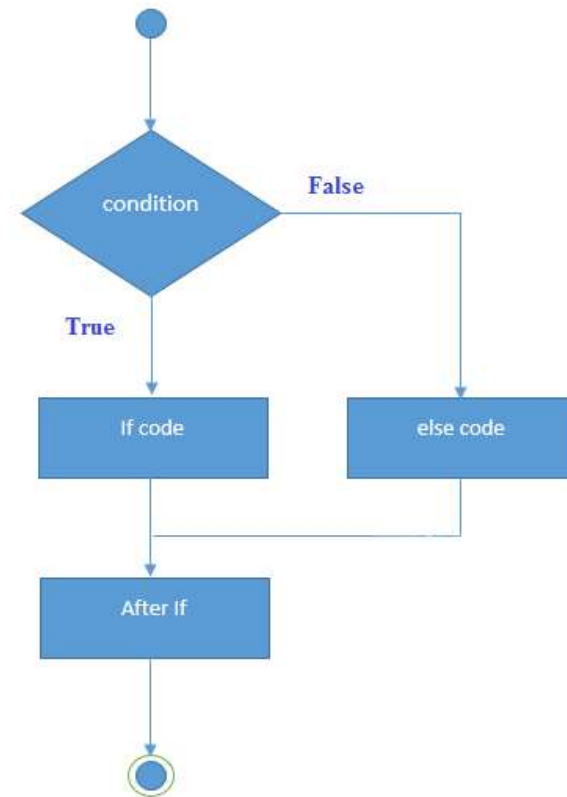
Decision Making and Loops

The decision-making is a feature that allows you to evaluate a condition before the instructions are executed.

The Dart language supports the following types of decision-making statements:

- ❑ If statement
- ❑ If-else statement
- ❑ Switch statement

The below diagram explains it more clearly.



Example

```
void main() {  
    var num = 12;  
    if (num % 2 == 0) {  
        print("Number is Even.");  
    } else {  
        print("Number is Odd.");  
    }  
}
```

Loops are used to execute a block of code repeatedly until a specified condition becomes true. Dart language supports the following types of loop statements:

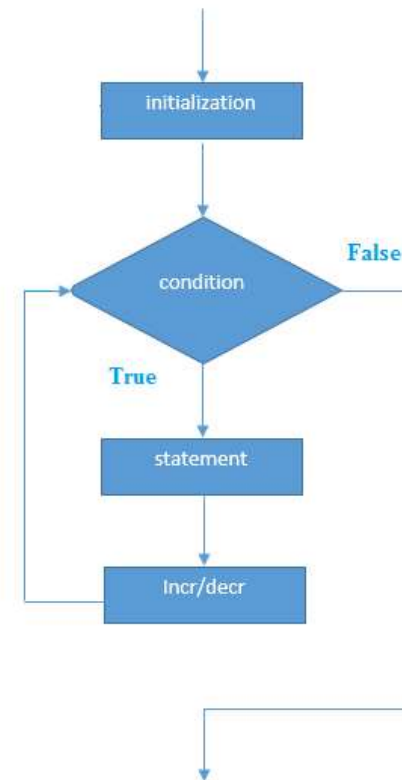
- ❑ for

- ❑ for..in

- ❑ while

- ❑ do..while

The below diagram explains it more clearly.



Example

```
void main() {  
    var name = ["Peter", "Rinky Ponting", "Abhishek"];  
  
    for (var prop in name) {  
        print(prop);  
    }  
}
```

Comments

Comments are the lines of **non-executable** code.

They are one of the main aspects of all programming languages.

The purpose of this is to provide information about the project, variable, or an operation.

There are three types of comments in Dart programming:

- ❑ **Make format comments:** It is a single line comment (//)
- ❑ **Block Comments:** It is a multi-line comment (/*...*/)
- ❑ **Doc Comments:** It is a document comment that used for member and types (///)

Continue and Break

Dart has also used the continue and break keyword in the loop, and elsewhere it required.

The continue statement allows you to skip the remaining code inside the loop and immediately jump to the next iteration of the loop.

Example

```
void main() {  
  for(int i=1;i<=10;i++){  
    if(i==5){  
      print("Hello");  
      continue; //it will skip the rest statement  
    }  
    print(i);  
  }  
}
```

The **break** statement allows you to terminate or stops the current flow of a program and continues execution after the body of the loop. The following example gives a detailed explanation.

Example

```
void main() {  
    for(int i=1;i<=10;i++){  
        if(i==5){  
            print("Hello");  
            break;//it will terminate the rest statement  
        }  
        print(i);  
    }  
}
```

Final and Const Keyword

We can use a final keyword to restrict the user. It can be applied in many contexts, such as variables, classes, and methods.

Const keyword is used to declare constant. We cannot change the value of the const keyword after assigning it.

Example

```
void main() {  
    final a = 100;  
    const pi = 3.14;  
    print(a);  
    print(pi);  
}
```

Object-Oriented Programming

Dart is an object-oriented programming language, which means every value in a Dart is an object.

A number is also an object in Dart language.

Dart programming supports the concept of OOPs features like objects, classes, interfaces, etc.

Object: An object is an entity, which has state and behavior. It can be physical or logical. In Dart, every value is an object, even primitive values like text and number. Dart can also allow you to build your custom object to express more complex relations between data.

Class: A class is a collection of objects. It means the objects are created with the help of classes because every object needs a blueprint based on which you can create an individual object.

A class definition includes the following things:

- ❑ Fields
- ❑ Methods
- ❑ Constructor
- ❑ Getters and setters

Example, to explain the OOPs concept:

```
class Mobile {  
    // Property Declaration  
    String color, brandName, modelName;  
  
    // Method Creation  
    String calling() {  
        return "Mobile can do call to everyone.";  
    }  
    String musicPlay() {  
        return "Mobile can play all types of Music.";  
    }  
}
```

```
String clickPicture() {  
    return "Mobile can take pictures.";  
}  
  
void main() {  
    // Object Creation  
    var myMob = new Mobile();  
  
    // Accessing Class's Property  
    myMob.color = "Black";  
    myMob.brandName = "Apple Inc.";  
    myMob.modelName = "iPhone 11 Pro";  
}
```

//Display Output

print(myMob.color);

print(myMob.modelName);

print(myMob.brandName);

print(myMob.calling());

print(myMob.musicPlay());

print(myMob.clickPicture());

}

A solid orange horizontal bar spanning the width of the slide at the bottom.

In the above example, we define a class **Mobile**, which has three variables of string type and three functions or methods.

Then, we create a **main** function which Dart will execute first when your app starts.

Inside the main, we create an **object** to access the class's properties.

Finally, we print the output.

A solid orange horizontal bar spanning the width of the slide at the bottom.

Difference Between Dart and Java

Feature	Dart	Java
Purpose	Primarily used for building mobile and web applications	Used for a wide range of applications, including Android mobile apps, web applications, and enterprise systems
Syntax and Structure	More concise and expressive, similar to other modern programming languages	More verbose syntax, familiar to programmers with a background in languages like C or C++
Typing	Dynamically-typed	Statically-typed
Performance	Fast execution times	Fast execution times, but maybe faster and more efficient due to static typing

Developer Community	Niche language with a smaller developer community	Extremely popular language with a large and active developer community
Tools and Frameworks	May not have the same level of support and resources as Java	A wide range of tools and frameworks are available

Example Code of Dart and Java

Java

```
/*package whatever //do not write package name here */  
  
import java.io.*;  
  
class GFG {  
    public static void main (String[] args) {  
        // System.out.println function is used  
        // to print 'Hello, GeekLand!' to the console  
        System.out.println("Hello, GeekLand!");  
    }  
}
```

Dart



```
void main() {  
    // print function is used to  
    // print 'Hello, GeekLand!' to the console  
    print('Hello, GeekLand!');  
}
```





Mobile Applications Developments



COMPUTER SYSTEMS ENGINEERING

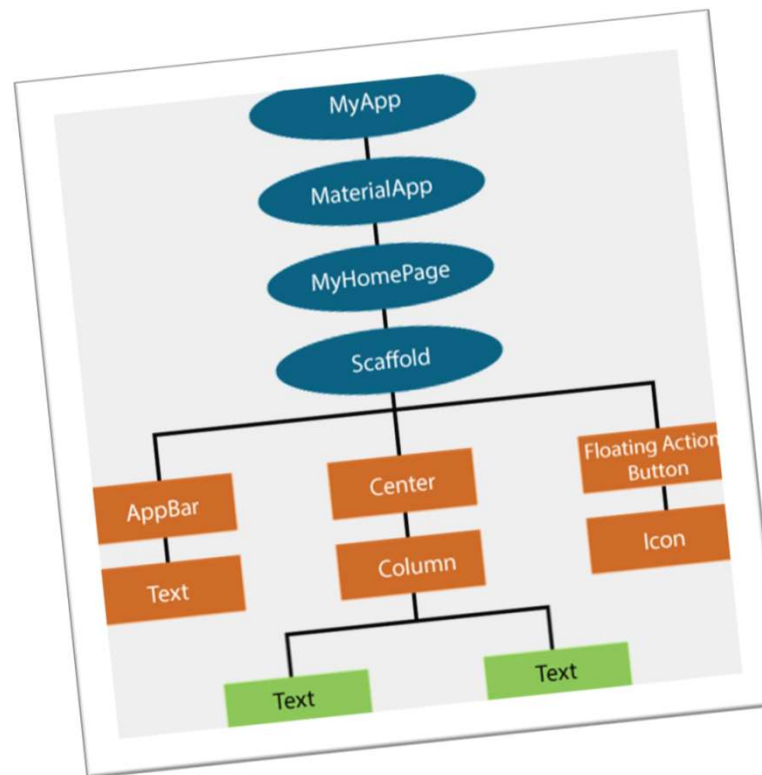
LECTURER: AHMAD SHAQALIEH

SECOND SEMESTER 2025

LECTURE 2

A solid orange horizontal bar spanning the width of the slide at the bottom.

Flutter Widgets



we are going to learn the concept of a widget, how to create it, and their different types available in the Flutter framework.

If you are familiar with React or Vue.js, then it is easy to understand the Flutter.

Whenever you are going to code for building anything in Flutter, it will be inside a widget.

The central purpose is to build the app out of widgets.

It describes how your app view should look like with their current configuration and state.

When you made any alteration in the code, the widget rebuilds its description by calculating the difference of previous and current widget to determine the minimal changes for rendering in UI of the app.



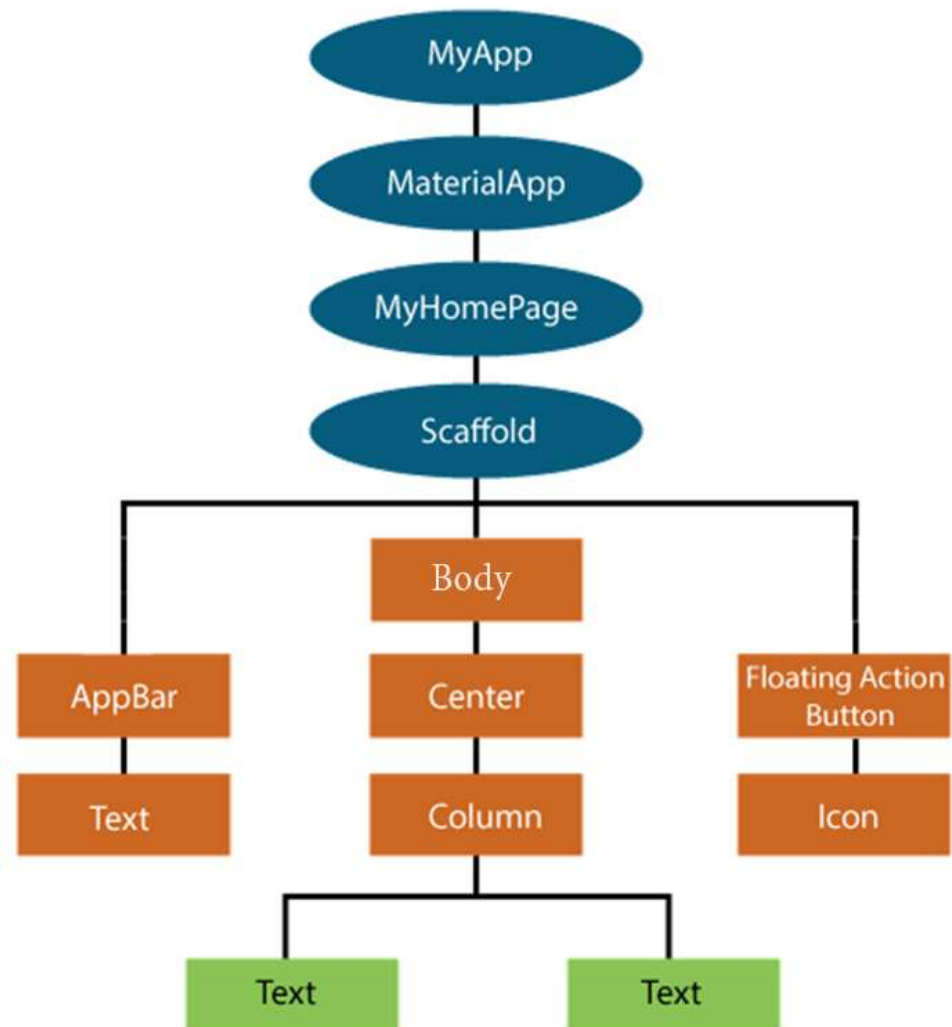
Widgets are nested with each other to build the app.

It means the root of your app is itself a widget, and all the way down is a widget also.

For example, a widget can display something, can define design, can handle interaction, etc.

The below image is a simple visual representation of the widget tree.





We can create the Flutter widget like this:

```
Class ImageWidget extends StatelessWidget {  
    // Class Stuff  
}
```

The below image is **Hello World Example**



```
import 'package:flutter/material.dart';

class MyHomePage extends StatelessWidget {
  MyHomePage({Key key, this.title}) : super(key: key);
  // This widget is the home page of your application.
  final String title;

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text(this.title),
      ),
      body: Center(
        child: Text('Hello World'),
      ),
    );
  }
}
```



```
import 'package:flutter/material.dart';

void main() => runApp(MyApp());

class MyApp extends StatelessWidget {
  // This widget is the root of your application.
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Hello World Demo Application',
      theme: ThemeData(
        primarySwatch: Colors.blue,
      ),
      home: MyHomePage(title: 'Home page'),
    );
  }
}
```

```
class MyHomePage extends StatelessWidget {
  MyHomePage({Key key, this.title}) : super(key: key);

  final String title;

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text(this.title),
      ),
      body: Center(
        child:
          Text(
            'Hello World',
          )
        ),
      );
  }
}
```

Let us understand the dart code line by line.

- ❑ **Line 1:** imports the flutter package, material. The material is a flutter package to create user interface according to the Material design guidelines specified by Android.
- ❑ **Line 3:** This is the entry point of the Flutter application. Calls runApp function and pass it an object of MyApp class. The purpose of the runApp function is to attach the given widget to the screen.
- ❑ **Line 5 - 17:** Widget is used to create UI in flutter framework.

StatelessWidget is a widget, which does not maintain any state of the widget.

MyApp extends StatelessWidget and overrides its build method.



The purpose of the build method is to create a part of the UI of the application. Here, build method uses MaterialApp, a widget to create the root level UI of the application.

It has three properties - title, theme and home.

- o title is the title of the application.

- o theme is the theme of the widget.

Here, we set blue as the overall color of the application using ThemeData class and its property, primarySwatch.

- o home is the inner UI of the application, which we set another widget, MyHomePage

- **Line 19 - 38:**

MyHomePage is same as MyApp except it returns ScaffoldWidget.

Scaffold is a top level widget next to MaterialApp widget used to create UI conforming material design.

It has two important properties, appBar to show the header of the application and body to show the actual content of the application.

AppBar is another widget to render the header of the application and we have used it in appBar property.

In body property, we have used Center widget, which centers it child widget.

Text is the final and inner most widget to show the text and it is displayed in the center of the screen.



Types of Widget

We can split the Flutter widget into two categories:

- I. Visible (Output and Input)
- II. Invisible (Layout and Control)

Visible widget

The visible widgets are related to the user input and output data.

Some of the important types of this widget are: **Text**

Text

A Text widget holds some text to display on the screen.

We can align the text widget by using **textAlign** property, and style property allow the customization of Text that includes font, font weight, font style, letter spacing, color, and many more.



We can use it as like below code snippets.

```
new Text(  
  'Hello, Javatpoint!',  
  textAlign: TextAlign.center,  
  style: new TextStyle(fontWeight: FontWeight.bold),  
)
```

Button

This widget allows you to perform some action on click.

Flutter does not allow you to use the Button widget directly; instead, it uses a type of buttons like a FlatButton and a RaisedButton.

```
//FlatButton Example
new FlatButton(
  child: Text("Click here"),
  onPressed: () {
    // Do something here
  },
),
```

```
//RaisedButton Example
new RaisedButton(
  child: Text("Click here"),
  elevation: 5.0,
  onPressed: () {
    // Do something here
  },
),
```

In the above example, the **onPressed** property allows us to perform an action when you click the button, and **elevation** property is used to change how much it stands out.

Image

This widget holds the image which can fetch it from multiple sources like from the asset folder or directly from the URL. It provides many constructors for loading image, which are given below:

- **Image:** It is a generic image loader, which is used by **ImageProvider**.
 - ❑ **asset:** It load image from your project asset folder.
 - ❑ **file:** It loads images from the system folder.
 - ❑ **memory:** It load image from memory.
 - ❑ **network:** It loads images from the network.

To add an image in the project, you need first to create an assets folder where you keep your images and then add the below line in **pubspec.yaml** file.

```
assets:  
  - assets/
```

Now, add the following line in the dart file.

```
Image.asset('assets/computer.png')
```

```
class MyHomePage extends StatelessWidget {  
  MyHomePage({Key key, this.title}) : super(key: key);  
  // This widget is the home page of your application.  
  final String title;  
  
  @override  
  Widget build(BuildContext context) {  
    return Scaffold(  
      appBar: AppBar(  
        title: Text(this.title),  
      ),  
      body: Center(  
        child: Image.asset('assets/computer.png'),  
      ),  
    );  
  }  
}
```

When you run the app, it will give the following output.



Icon

This widget acts as a container for storing the Icon in the Flutter. The following code explains it more clearly

```
new Icon(  
  Icons.add,  
  size: 34.0,  
)
```

Invisible widget

The invisible widgets are related to the layout and control of widgets. It provides controlling how the widgets actually behave and how they will look onto the screen. Some of the important types of these widgets are:

Column

A column widget is a type of widget that arranges all its children's widgets in a vertical alignment.

It provides spacing between the widgets by using the the **mainAxisAlignment** and **crossAxisAlignment** properties.

In these properties, the main axis is the vertical axis, and the cross axis is the horizontal axis.



Example: The below code snippets construct two widget elements vertically.

```
new Column(  
  mainAxisAlignment: MainAxisAlignment.center,  
  children: <Widget>[  
    new Text(  
      "VegElement",  
    ),  
    new Text(  
      "Non-vegElement"  
    ),  
  ],  
)
```

Row

The row widget is similar to the column widget, but it constructs a widget horizontally rather than vertically. Here, the main axis is the horizontal axis, and the cross axis is the vertical axis.

Example

The below code snippets construct two widget elements horizontally.




```
new Row(  
  mainAxisAlignment: MainAxisAlignment.spaceEvenly,  
  children: <Widget>[  
    new Text(  
      "VegElement",  
    ),  
    new Text(  
      "Non-vegElement"  
    ),  
  ],  
)
```

Center

This widget is used to center the child widget, which comes inside it. All the previous examples contain inside the center widget.

Example

```
Center(  
  child: new Column(  
    mainAxisAlignment: MainAxisAlignment.spaceEvenly,  
    children: <Widget>[  
      new Text(  
        "VegElement",  
      ),  
      new Text(  
        "Non-vegElement"  
      ),  
    ],  
  ),  
)
```

Padding

This widget wraps other widgets to give them padding in specified directions.

You can also provide padding in all directions.

We can understand it from the below example that gives the text widget padding of 6.0 in all directions.

Example

```
Padding(  
  padding: const EdgeInsets.all(6.0),  
  child: new Text(  
    "Element 1",  
  ),  
)
```

Scaffold

This widget provides a framework that allows you to add common material design elements like AppBar, Floating Action Buttons, Drawers, etc.

Stack

It is an essential widget, which is mainly used for **overlapping** a widget, such as a button on a background gradient.



State Management Widget

In Flutter, there are mainly two types of widget:

- ❑ StatelessWidget

- ❑ StatefulWidget

StatefulWidget

A StatefulWidget has state information. It contains mainly two classes: the **state object** and the **widget**.

It is dynamic because it can change the inner data during the widget lifetime.

This widget does not have a **build()** method.

It has **createState()** method, which returns a class that extends the Flutter's State Class.

The examples of the StatefulWidget are Checkbox, Radio, Slider, InkWell, Form, and TextField.

```
class Car extends StatefulWidget {  
  const Car({ Key key, this.title }) : super(key: key);  
  
  @override  
  _CarState createState() => _CarState();  
}  
  
class _CarState extends State<Car> {  
  @override  
  Widget build(BuildContext context) {  
    return Container(  
      color: const Color(0xFEEFE),  
      child: Container(  
        child: Container( //child: Container() )  
      )  
    );  
  }  
}
```

StatelessWidget

The StatelessWidget does not have any state information. It remains static throughout its lifecycle. The examples of the StatelessWidget are Text, Row, Column, Container, etc.

Example

```
class MyStatelessCarWidget extends StatelessWidget {  
  const MyStatelessCarWidget ({ Key key }) : super(key: key);  
  
  @override  
  Widget build(BuildContext context) {  
    return Container(color: const Color(0x0xFEEFE));  
  }  
}
```




Mobile Applications Developments



COMPUTER SYSTEMS ENGINEERING

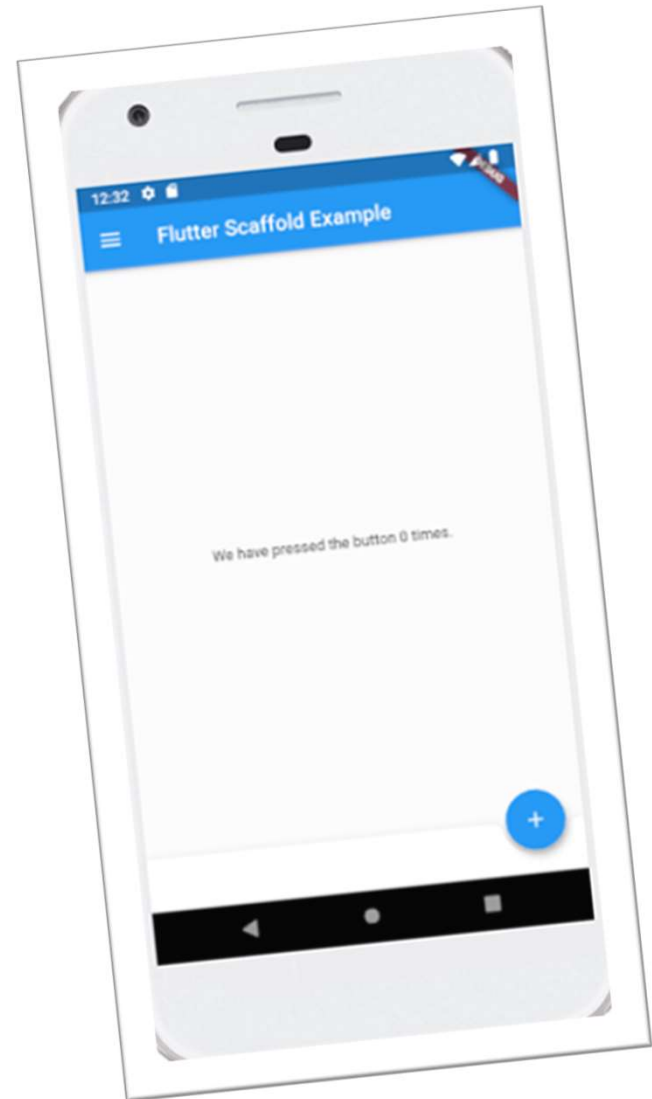
LECTURER: AHMAD SHAQALIEH

SECOND SEMESTER 2025

LECTURE 3

A solid orange horizontal bar spanning the width of the slide at the bottom.

Flutter Scaffold



The Scaffold is a widget in Flutter used to implements the basic material **design visual layout structure**.

It is quick enough to create a general-purpose mobile application and contains almost everything we need to create a functional and responsive Flutter apps.

This widget is able to occupy the whole device screen. In other words, we can say that it is mainly responsible for creating a base to the app screen on which the child widgets hold on and render on the screen.

It provides many widgets or APIs for showing Drawer, SnackBar, BottomNavigationBar, AppBar, FloatingActionButton, and many more.



The Scaffold class is a shortcut to set up the look and design of our app that allows us not to build the individual visual elements manually.

It saves our time to write more code for the look and feel of the app.

The following are the **constructor and properties** of the Scaffold widget class.

```
const Scaffold({  
  Key key,  
  this.appBar,  
  this.body,  
  this.floatingActionButton,  
  this.floatingActionButtonLocation,  
  this.persistentFooterButtons,  
  this.drawer,  
  this.endDrawer,  
  this.bottomNavigationBar,  
  this.bottomSheet,  
  this.floatingActionButtonAnimator,  
  this.backgroundColor,  
  this.resizeToAvoidBottomPadding = true,  
  this.primary = true,  
})
```

Let us understand all of the above properties in detail.

1. appBar:

It is a **horizontal bar** that is mainly displayed at the **top** of the Scaffold widget.

It is the main part of the Scaffold widget and displays at the top of the screen.

Without this property, the Scaffold widget is incomplete.

It uses the appBar widget that itself contains various properties like elevation, title, brightness, etc. See the below example:

```
Widget build(BuildContext context)
{
  return Scaffold(
    appBar: AppBar(
      title: Text('First Flutter Application'),
    ),
  );
}
```

In the above code, the title property uses a **Text widget** for displaying the text on the screen.



2. body:

It is the other primary and required property of this widget, which will **display the main content in the Scaffold**.

It signifies the place below the appBar and behind the floatingActionButton & drawer.

The widgets inside the body are positioned at the top-left of the available space by default.

See the below code:



```
Widget build(BuildContext context) {  
  return Scaffold(  
    appBar: AppBar(  
      title: Text('First Flutter Application'),  
    ),  
    body: Center(  
      child: Text("Welcome to Javatpoint",  
        style: TextStyle( color: Colors.black, fontSize: 30.0,  
        ),  
      ),  
    ),  
  )  
}
```

In the above code, we have displayed a text "**Welcome to !!**" in the body attribute.

This text is aligned in the **center** of the page by using the **Center widget**.

Here, we have also styled the text by using the **TextStyle** such as color, font size, etc.

3. drawer:

It is a **slider panel** that is displayed at the side of the body.

Usually, it is hidden on the mobile devices, but the user can swipe it left to right or right to left to access the drawer menu.

It uses the **Drawer widget properties** slides in a **horizontal direction** from the Scaffold edge to show navigation links in the application.

An appropriate **icon** for the drawer is set automatically in an appBar property.

The **gesture** is also set automatically to open the drawer. See the following code.



```
drawer: Drawer(  
  child: ListView(  
    children: const <Widget>[  
      DrawerHeader(  
        decoration: BoxDecoration(  
          color: Colors.red,  
        ),  
        child: Text(  
          'Welcome to Javatpoint',  
          style: TextStyle(  
            color: Colors.green,  
            fontSize: 30,  
          ),  
        ),  
      ),  
      ListTile(  
        title: Text('1'),  
      ),  
    ],  
  ),  
),
```

```
ListTile(  
  title: new Text("All Mail Inboxes"),  
  leading: new Icon(Icons.mail),  
),  
Divider(  
  height: 0.2,  
),  
ListTile(  
  title: new Text("Primary"),  
),  
ListTile(  
  title: new Text("Social"),  
),  
ListTile(  
  title: new Text("Promotions"),  
),  
],  
),
```

In the above code, we use the drawer property of Scaffold for creating a drawer.

We have also used some other widgets to make it attractive.

In the **ListView** widget, we have divided the panel into two parts, **Header** and **Menu**.

The DrawerHeader property modifies the panel header that also contains an icon or details according to the application.

Again, we have used **ListTile** to add the list items in the menu.



4. **floatingActionButton:**

It is a button displayed at the bottom right corner and floating above the body.

It is a circular icon button that floats over the content of a screen at a fixed place to promote a primary action in the application.

While scrolling the page, its position cannot be changed.

It uses the FloatingActionButton widget properties using **Scaffold.floatingActionButton**.

See the below code:



```
Widget build(BuildContext context) {  
  return Scaffold(  
    appBar: AppBar(title: Text('First Flutter Application')),  
    body: Center(  
      child: Text("Welcome to Javatpoint!!"),  
    ),  
    floatingActionButton: FloatingActionButton(  
      elevation: 8.0,  
      child: Icon(Icons.add),  
      onPressed: () {  
        print('I am Floating Action Button');  
      }  
    );  
  }  
}
```

In the above code, we have used the **elevation** property that gives a **shadow effect** to the button.

We have also used the Icon widget to give an icon to the button using preloaded Flutter SDK icons.

The **onPressed()** property will be called when the user taps the button, and the statements "**I am Floating Action Button**" will be printed on the console.

5. backgroundColor: This property is used to set the background color of the whole Scaffold widget.

```
backgroundColor: Colors.yellow,
```

6. primary:

It is used to tell whether the Scaffold will be displayed at the top of the screen or not.

Its default value is **true** that means the height of the appBar extended by the height of the screen's status bar.

primary: **true/false**,

7. **persistentFooterButton:**

It is a list of buttons that are displayed at the bottom of the Scaffold widget.

These property items are always visible; even we have scroll the body of the Scaffold.

It is always wrapped in a **ButtonBar widget**.

They are rendered below the body but above the bottomNavigationBar.



```
persistentFooterButtons: <Widget>[
  RaisedButton(
    onPressed: () {},
    color: Colors.blue,
    child: Icon(
      Icons.add,
      color: Colors.white,
    ),
  ),
  RaisedButton(
    onPressed: () {},
    color: Colors.green,
    child: Icon(
      Icons.clear,
      color: Colors.white,
    ),
  ),
],
```

In the above code, we have used the **RaisedButton** that displays at the bottom of the Scaffold. We can also use the **FlatButton** instead of the RaisedButton.

8. bottomNavigationBar:

This property is like a **menu that displays a navigation bar** at the bottom of the Scaffold.

It can be seen in most of the mobile applications.

This property allows the developer to add multiple icons or texts in the bar as items.

It should be rendered below the body and persistentFooterButtons.

See the below code:



```
bottomNavigationBar: BottomNavigationBar(  
  currentIndex: 0,  
  fixedColor: Colors.grey,  
  items: [  
    BottomNavigationBarItem(  
      title: Text("Home"),  
      icon: Icon(Icons.home),  
    ),  
    BottomNavigationBarItem(  
      title: Text("Search"),  
      icon: Icon(Icons.search),  
    ),  
    BottomNavigationBarItem(  
      title: Text("User Profile"),  
      icon: Icon(Icons.account_circle),  
    ),  
  ],  
  onTap: (int itemIndex){  
    setState() {  
      _currentIndex = itemIndex;  
    };  
  },  
),
```

In the previous code, we have used the `BottomNavigationBar` widget to display the menu bar.

The **`fixedColor`** property is used for the **`color`** of the active icon.

The **`BottomNavigationBarItems`** widget is used to add items in the bar containing text and icon as its child property.

We had also used **`onTap(int itemIndex)`** function to perform an action when we tapped on the items, which works according to their index position.



9. endDrawer:

It is similar to a drawer property, but they are displayed at the right side of the screen by default.

It can be swiped right to left or left to right.

10. resizeToAvoidBottomInset:

If it is **true**, the body and the Scaffold's floating widgets should adjust their size themselves to avoid the onscreen keyboard.

The bottom property defines the onscreen keyboard height.

11. floatingActionButtonLocation:

By default, it is positioned at the bottom right corner of the screen. It is used to determine the position of the floatingActionButton.

It contains many predefined constants, such as centerDocked, centerFloat, endDocked, endFloat, etc.



That's all about the Scaffold's various properties that give us an overview of the Scaffold widget.

Its main purpose is to get familiar with different properties and their usage in Flutter application.

Let us see the example where we have tried to use most of the Scaffold properties to understand this widget quickly and easily.

In this example, we are going to see a Scaffold widget with an AppBar, BottomAppBar, FloatingActionButton, floatingActionButtonLocation, and drawer properties.



```
import 'package:flutter/material.dart';
```

```
void main() => runApp(MyApp());
```

```
/// This Widget is the main application widget.
```

```
class MyApp extends StatelessWidget {
```

```
  @override
```

```
  Widget build(BuildContext context) {
```

```
    return MaterialApp(
```

```
      home: MyStatefulWidget(),
```

```
    );
```

```
  }
```

```
}
```

```
class MyStatefulWidget extends StatefulWidget {
```

```
  MyStatefulWidget({Key key}) : super(key: key);
```

```
@override
_MyStatefulWidgetState createState() => _MyStatefulWidgetState();
}

class _MyStatefulWidgetState extends State<MyStatefulWidget> {
  int _count = 0;

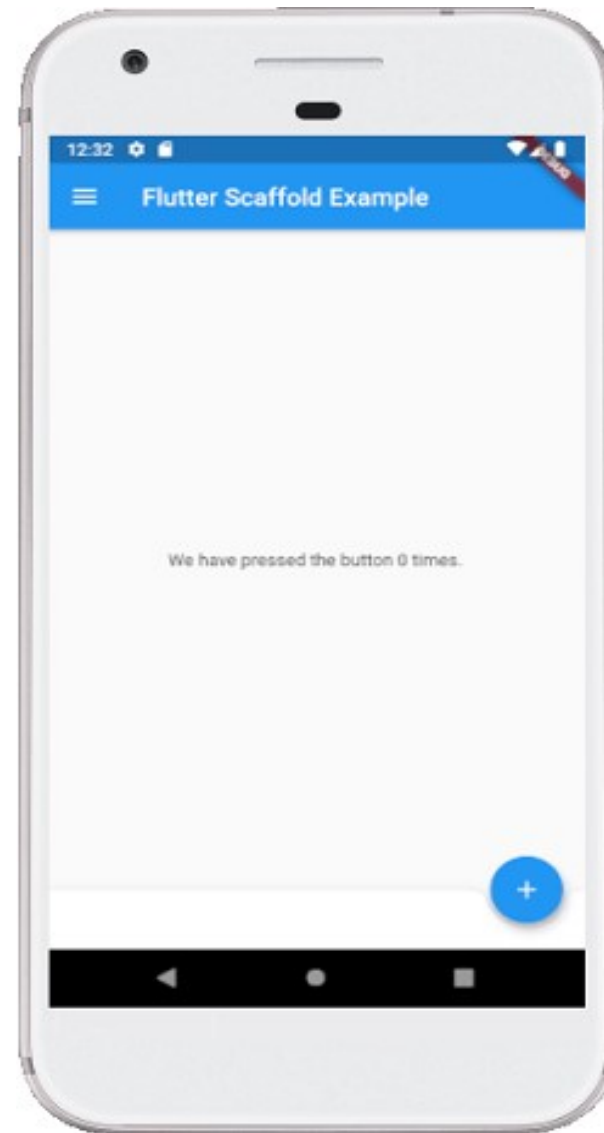
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text('Flutter Scaffold Example'),
      ),
      body: Center(
        child: Text('We have pressed the button $_count times.'),
      ),
      bottomNavigationBar: BottomAppBar(
        shape: const CircularNotchedRectangle(),
        child: Container(
          height: 50.0,
        ),
      ),
    );
  }
}
```

```
),  
floatingActionButton: FloatingActionButton(  
  onPressed: () => setState(() {  
    _count++;  
  }),  
  tooltip: 'Increment Counter',  
  child: Icon(Icons.add),  
)  
floatingActionButtonLocation: FloatingActionButtonLocation.endDocked,  
drawer: Drawer(  
  elevation: 20.0,  
  child: Column(  
    children: <Widget>[  
      UserAccountsDrawerHeader(  
        accountName: Text("javatpoint"),  
        accountEmail: Text("javatpoint@gmail.com"),  
        currentAccountPicture: CircleAvatar(  
          backgroundColor: Colors.yellow,  
          child: Text("abc"),  
        ),  
      ],  
    ),  
  ),  
)
```

```
),  
ListTile(  
  title: new Text("Inbox"),  
  leading: new Icon(Icons.mail),  
),  
Divider( height: 0.1,),  
ListTile(  
  title: new Text("Primary"),  
  leading: new Icon(Icons.inbox),  
),  
ListTile(  
  title: new Text("Social"),  
  leading: new Icon(Icons.people),  
),  
ListTile(  
  title: new Text("Promotions"),  
  leading: new Icon(Icons.local_offer),  
)  
],  
),  
),  
);  
}  
}
```

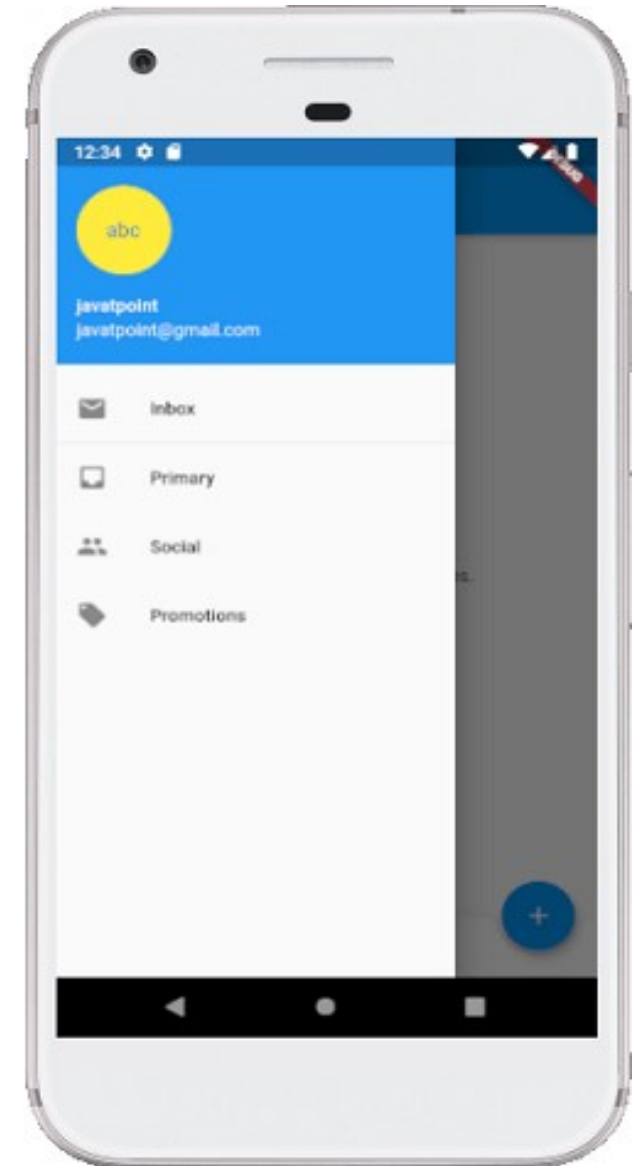
Output:

When we run this project in the IDE, we will see the UI as the following screenshot.



If we click on the three lines that can be seen in the top left corner of the screen, we will see the drawer.

The drawer can be swiped right to left or left to right. See the below image.





Mobile Applications Developments



COMPUTER SYSTEMS ENGINEERING

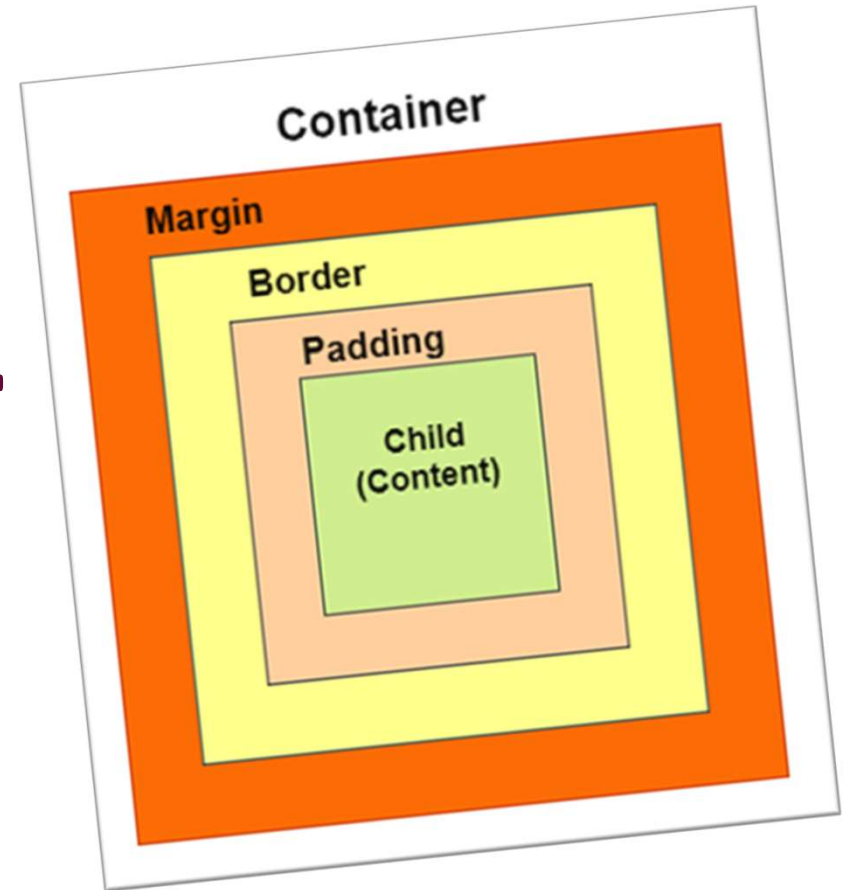
LECTURER: AHMAD SHAQALIEH

SECOND SEMESTER 2025

LECTURE 4

A solid orange horizontal bar spanning the width of the slide at the bottom.

Flutter Container



The container in Flutter is a **parent widget that can contain one child widget** and manage it efficiently through width, height, padding, background color, etc.

It is a widget that combines common painting, positioning, and sizing of the child widgets.

It is also a class to store one or more widgets and position them on the screen according to our needs.

Generally, it is similar to a box for storing contents.

It allows many attributes to the user for decorating its child widgets, such as using **margin**, which separates the container with other contents.

A container widget is same as **<div>** tag in html.

If this widget does not contain any child widget, it will fill the whole area on the screen automatically.



Otherwise, it will wrap the child widget according to the specified height & width.

It is to **note that** this widget cannot render directly without any parent widget.

We can use Scaffold widget, Center widget, Padding widget, Row widget, or Column widget as its parent widget.



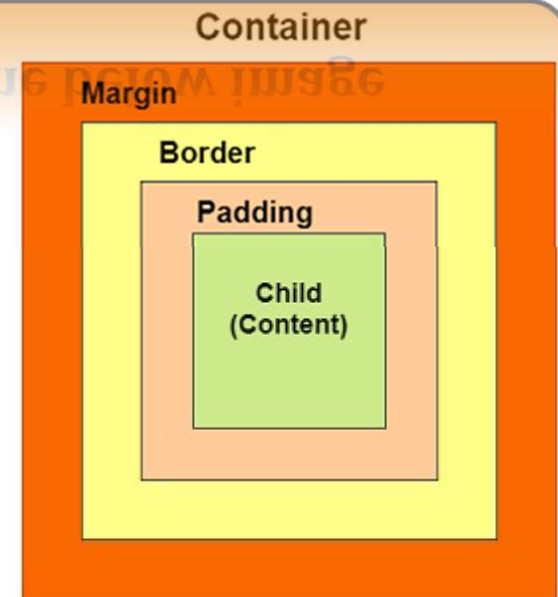
Why we need a container widget in Flutter?

If we have a widget that needs some background styling may be a color, shape, or size constraints, we may try to **wrap it in a container widget**.

This widget helps us to compose, decorate, and position its child widgets. If we wrap our widgets in a container, then without using any parameters, we would not notice any difference in its appearance.

But if we add any properties such as color, margin, padding, etc. in a container, we can style our widgets on the screen according to our needs.

A basic container has a margin, border, and padding properties surrounding its child widget, as shown in the below image



Constructors of the container class

The following are the syntax of container class constructor:

```
Container({Key key,  
    AlignmentGeometry alignment,  
    EdgeInsetsGeometry padding,  
    Color color,  
    double width,  
    double height,  
    Decoration decoration,  
    Decoration foregroundDecoration,  
    BoxConstraints constraints,  
    Widget child,  
});
```

Properties of Container widget

Let us learn some of the essential properties of the container widget in detail.

1. child: This property is used to store the child widget of the container. Suppose we have taken a Text widget as its child widget that can be shown in the below example:

```
Container(  
  child: Text("Hello! I am in the container widget", style: TextStyle(fontSize: 25)),  
)
```

2. color: This property is used to set the **background color of the text**. It also changes the background color of the entire container. See the below example:

```
Container(  
  color: Colors.green,  
  child: Text("Hello! I am in the container widget", style: TextStyle(fontSize: 25)),  
)
```

3. height and width: This property is used to set the container's height and width according to our needs. By default, the container always takes the space based on its child widget. See the below code:

```
Container(  
  width: 200.0,  
  height: 100.0,  
  color: Colors.green,  
  child: Text("Hello! I am in the container widget", style: TextStyle(fontSize: 25)),  
)
```

4. margin: This property is used to surround the **empty space around the container**. We can observe this by seeing white space around the container. Suppose we have used the **EdgeInsets.all(25)** that set the equal margin in all four directions, as shown in the below example:

```
Container(  
  width: 200.0,  
  height: 100.0,  
  color: Colors.green,  
  margin: EdgeInsets.all(20),  
  child: Text("Hello! I am in the container widget", style: TextStyle(fontSize: 25)),  
)
```

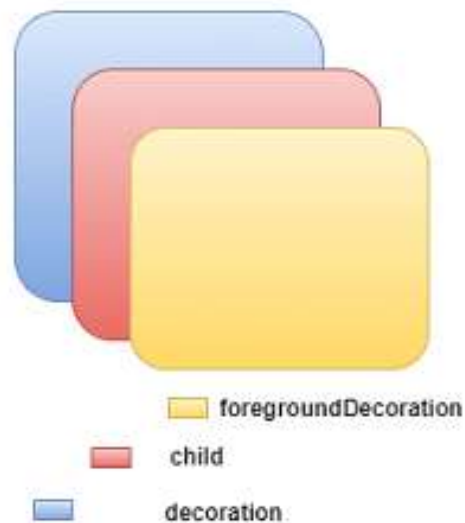
5. padding: This property is used to **set the distance** between the border of the container (all four directions) and its child widget. We can observe this by seeing the space between the container and the child widget. Here, we have used an `EdgeInsets.all(35)` that set the space between text and all four container directions:

```
Container(  
  width: 200.0,  
  height: 100.0,  
  color: Colors.green,  
  padding: EdgeInsets.all(35),  
  margin: EdgeInsets.all(20),  
  child: Text("Hello! I am in the container widget", style: TextStyle(fontSize: 25)),  
)
```

6. alignment: This property is used to **set the position** of the child within the container. Flutter allows the user to align its element in various ways such as center, bottom, bottom center, topLeft, centerRight, left, right, and many more. In the below example, we are going to align its child into the bottom right position.

```
Container(  
  width: 200.0,  
  height: 100.0,  
  color: Colors.green,  
  padding: EdgeInsets.all(35),  
  margin: EdgeInsets.all(20),  
  alignment: Alignment.bottomRight,  
  child: Text("Hello! I am in the container widget", style: TextStyle(fontSize: 25)),  
)
```

7. decoration: This property allows the developer to **add decoration on the widget**. It decorates or paint the widget behind the child. If we want to decorate or paint in front of a child, we need to use the **foregroundDecoration** parameter. The below image explains the difference between them where the foregroundDecoration covers the child and decoration paint behind the child.



The decoration property supported many parameters, such as color, gradient, background image, border, shadow, etc. It is to make sure that **we can either use the color property in a container or decoration, but not in both.**

See the below code where we have added a border and shadow property to decorate the box:

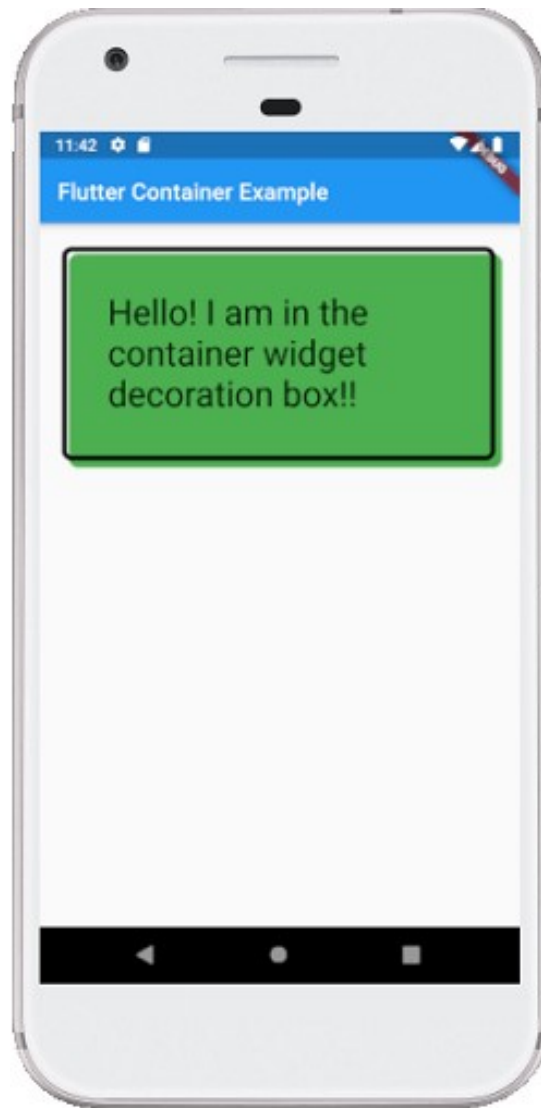
```
import 'package:flutter/material.dart';

void main() => runApp(MyApp());

/// This Widget is the main application widget.
class MyApp extends StatelessWidget {
```

```
@override
Widget build(BuildContext context) {
  return MaterialApp(
    home: Scaffold(
      appBar: AppBar(
        title: Text("Flutter Container Example"),
      ),
      body: Container(
        padding: EdgeInsets.all(35),
        margin: EdgeInsets.all(20),
        decoration: BoxDecoration(
          border: Border.all(color: Colors.black, width: 4),
          borderRadius: BorderRadius.circular(8),
          boxShadow: [
            new BoxShadow(color: Colors.green, offset: new Offset(6.0, 6.0)),
          ],
        ),
      ),
    ),
  );
}
```

```
    child: Text("Hello! I am in the container widget decoration box!!",  
      style: TextStyle(fontSize: 30)),  
  ),  
),  
);  
}  
}
```



8. transform: The transform property allows developers to **rotate the container**. It can rotate the container in any direction, i.e., change the container coordinate in the parent widget. In the below example, we will rotate the container in the **z-axis**.

```
Container(  
  width: 200.0,  
  height: 100.0,  
  color: Colors.green,  
  padding: EdgeInsets.all(35),  
  margin: EdgeInsets.all(20),  
  alignment: Alignment.bottomRight,  
  transform: Matrix4.rotationZ(0.1),  
  child: Text("Hello! I am in the container widget", style: TextStyle(fontSize: 25)),  
)
```

9. constraints: This property is used when we want to **add additional constraints to the child**. It contains various constructors, such as tight, loose, expand, etc. Let's see how to use these constructors in our app:

tight: If we use size property in this, it will give fixed value to the child.

```
Container(  
  color: Colors.green,  
  constraints: BoxConstraints.tight(Size size)  
    : minWidth = size.width, maxWidth = size.width,  
    minHeight = size.height, maxHeight = size.height;  
  child: Text("Hello! I am in the container widget", style: TextStyle(fontSize: 25)),  
)
```

expand: Here, we can choose the height, width, or both values to the child.

```
Container(  
  color: Colors.green,  
  constraints: BoxConstraints.expand(height: 60.0),  
  child: Text("Hello! I am in the container widget", style: TextStyle(fontSize: 25)),  
)
```

Let us understand it with an example where we will try to cover most of the container properties. Open the **main.dart** file and replace it with the below code:

```
import 'package:flutter/material.dart';

void main() => runApp(MyApp());

/// This Widget is the main application widget.
class MyApp extends StatelessWidget {
```

```
@override
Widget build(BuildContext context) {
  return MaterialApp(
    home: MyContainerWidget(),
  );
}

class MyContainerWidget extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        appBar: AppBar(
          title: Text("Flutter Container Example"),
        ),
      ),
    );
  }
}
```

```
body: Container(  
  width: double.infinity,  
  height: 150.0,  
  color: Colors.green,  
  margin: EdgeInsets.all(25),  
  padding: EdgeInsets.all(35),  
  alignment: Alignment.center,  
  transform: Matrix4.rotationZ(0.1),  
  child: Text("Hello! I am in the container widget!!",  
    style: TextStyle(fontSize: 25)),  
),  
,  
);  
}  
}
```


When we run this app, it will give the following screenshot:





Mobile Applications Developments



COMPUTER SYSTEMS ENGINEERING

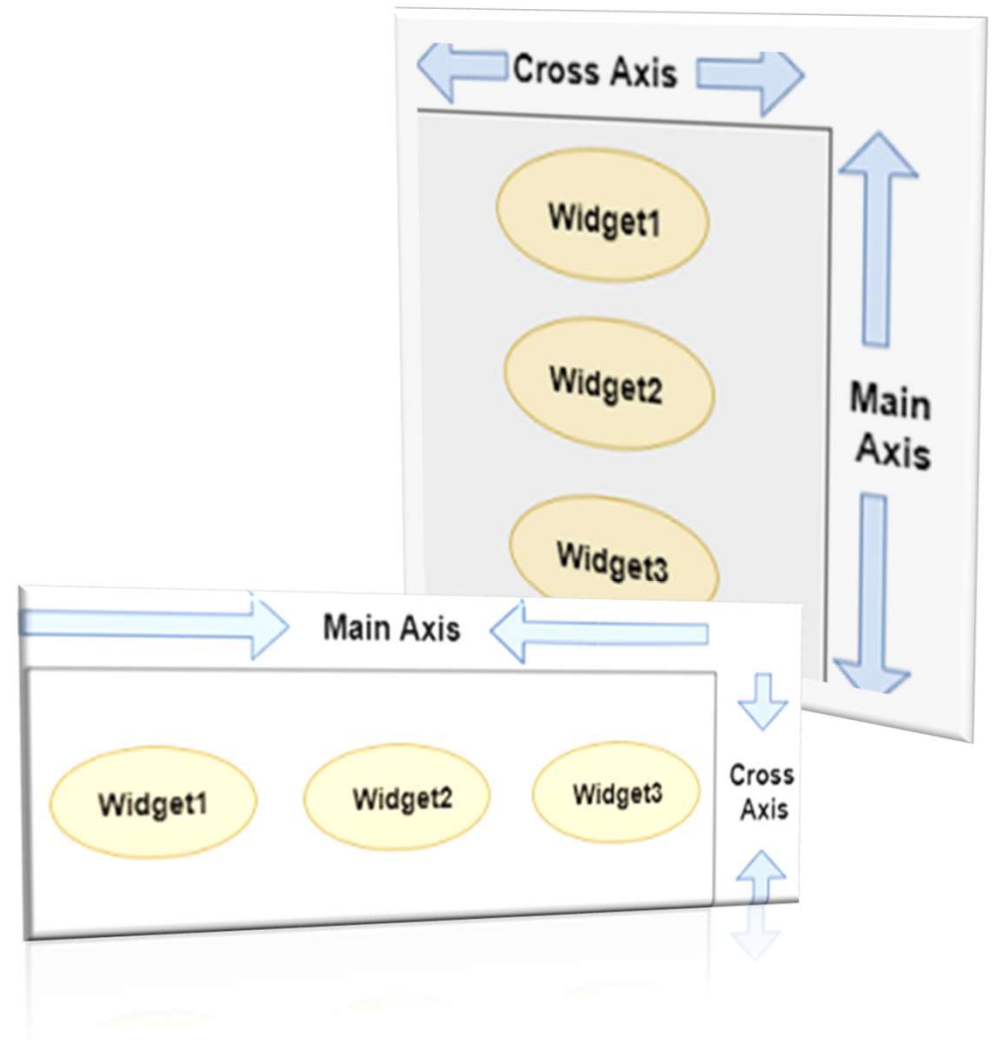
LECTURER: AHMAD SHAQALIEH

SECOND SEMESTER 2025

LECTURE 5

A solid orange horizontal bar spanning the width of the slide at the bottom.

Flutter Row and Column



The rows and columns are not a single widget; they are two different widgets, namely Row and Column.

Here, we will integrate these two widgets together because they have similar properties that help us understand them efficiently and quickly.

Row and column are the two essential widgets in Flutter that allows developers to **align children horizontally and vertically according to our needs**.

These widgets are very necessary when we design the application user interface in Flutter.



Key Points

- ❑ Row and Column widgets are the most commonly used layout patterns in the Flutter application.
- ❑ Both may take several child widgets.
- ❑ A child widget can also be a row or column widget.
- ❑ We can stretch or constrain a particular children's widget.
- ❑ Flutter also allows developers to specify how child widgets can use row and column widgets' available space.

Row Widget

This widget arranges its children in a horizontal direction on the screen.

In other words, it will expect child widgets in a horizontal array.

If the child widgets need to fill the available horizontal space, we must wrap the children widgets in an Expanded widget.

A row widget does **not appear scrollable** because it displays the widgets within the visible view.

So it is considered wrong if we have more children in a row which will not fit in the available space.

If we want to make a scrollable list of row widgets, we need to use the ListView widget.

We can control how a row widget aligns its children based on our choice using the property **crossAxisAlignment** and **mainAxisAlignment**.

The row's **cross-axis** will run **vertically**, and the **main axis** will run **horizontally**.

See the below visual representation to understand it more clearly.



Note: Flutter row widget has several other properties like `mainAxisSize`, `textDirection`, `verticalDirection`, etc. Here, we will discuss only `mainAxisAlignment` and `crossAxisAlignment` properties.

We can align the row's children widget with the help of the following properties:

- ❑ **start:** It will place the children from the starting of the main axis.
 - ❑ **end:** It will place the children at the end of the main axis.
 - ❑ **center:** It will place the children in the middle of the main axis.
 - ❑ **spaceBetween:** It will place the free space between the children evenly.
 - ❑ **spaceAround:** It will place the free space between the children evenly and half of that space before and after the first and last children widget.
 - ❑ **spaceEvenly:** It will place the free space between the children evenly and before and after the first and last children widget.
- 

```
import 'package:flutter/material.dart';

void main() { runApp(MyApp()); }

class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: MyHomePage(),
    );
  }
}

class MyHomePage extends StatefulWidget {
  @override
  _MyHomePageState createState() => _MyHomePageState();
}
```

```
class MyHomePage extends StatefulWidget {  
  @override  
  _MyHomePageState createState() => _MyHomePageState();  
}
```

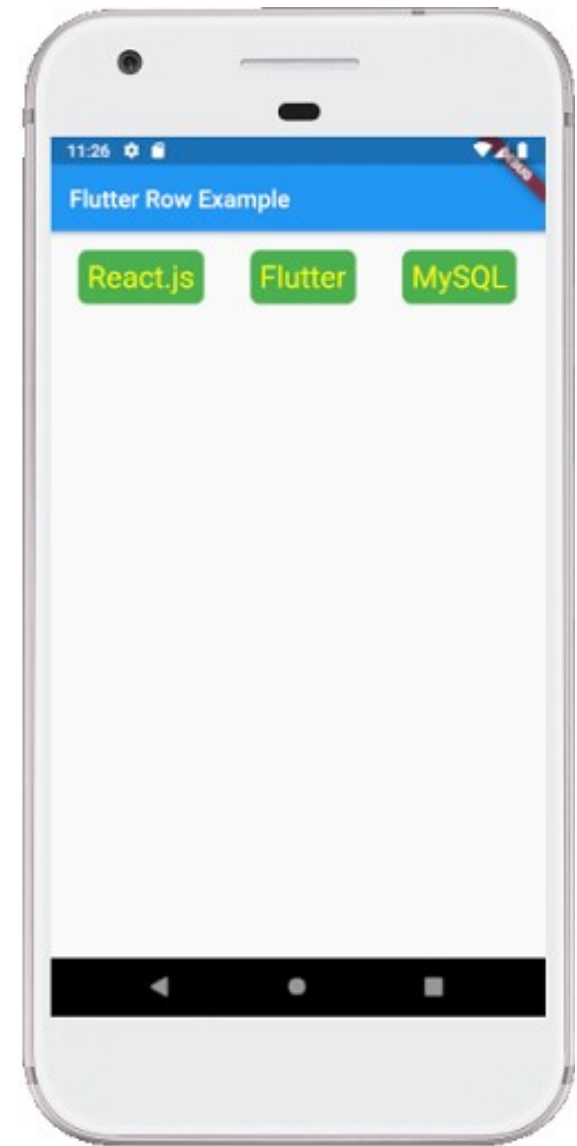
```
class _MyHomePageState extends State<MyHomePage> {  
  @override  
  Widget build(BuildContext context) {  
    return Scaffold(  
      appBar: AppBar(  
        title: Text("Flutter Row Example"),  
      ),  
      body: Row(  
        mainAxisAlignment: MainAxisAlignment.spaceEvenly,  
        children:<Widget>[  
          Container(  
            margin: EdgeInsets.all(12.0),  
            padding: EdgeInsets.all(8.0),  
            decoration: BoxDecoration(  
              color: Colors.white,  
              border: Border.all(  
                color: Colors.black,  
                width: 1.0,  
              ),  
            ),  
          ],  
        ),  
      ),  
    );  
  }  
}
```

```
        borderRadius: BorderRadius.circular(8),
        color: Colors.green
    ),
    child: Text("React.js", style: TextStyle(color: Colors.yellowAccent, fontSize: 25)),
),
Container(
    margin: EdgeInsets.all(15.0),
    padding: EdgeInsets.all(8.0),
    decoration: BoxDecoration(
        borderRadius: BorderRadius.circular(8),
        color: Colors.green
    ),
    child: Text("Flutter", style: TextStyle(color: Colors.yellowAccent, fontSize: 25)),
),
Container(
    margin: EdgeInsets.all(12.0),
    padding: EdgeInsets.all(8.0),
    decoration: BoxDecoration(
        borderRadius: BorderRadius.circular(8),
        color: Colors.green
```

```
),  
  child: Text("MySQL",style: TextStyle(color:Colors.yellowAccent,fontSize:25),),  
)  
]  
),  
);  
}  
}
```

Output:

When we run this app, we should get the UI as screenshot.



Column

This widget arranges its children in a vertical direction on the screen.

In other words, it will expect a vertical array of children widgets.

If the child widgets need to fill the available vertical space, we must wrap the children widgets in an Expanded widget.

A column widget does **not appear scrollable** because it displays the widgets within the visible view.

So it is considered wrong if we have more children in a column which will not fit in the available space.

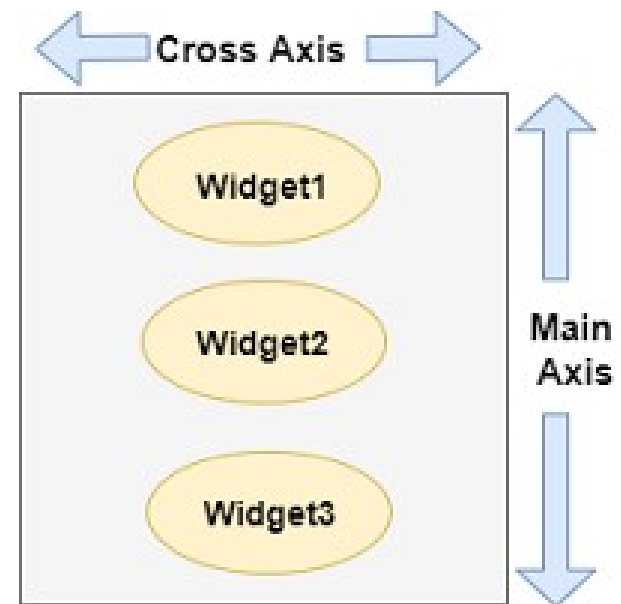
If we want to make a scrollable list of column widgets, we need to use the ListView Widget.

We can also control how a column widget aligns its children using the property `mainAxisAlignment` and `crossAxisAlignment`.

The column's **cross-axis** will run **horizontally**, and the **main axis** will run **vertically**.

The below visual representation explains it more clearly.

Note: Column widget also aligns its content by using the same properties as we have discussed in row widget such as `start`, `end`, `center`, `spaceAround`, `spaceBetween`, and `spaceEvenly`.



Let us understand it with the help of an example where we are going to align the content such that there is a free space between the children evenly in a column:

```
import 'package:flutter/material.dart';

void main() { runApp(MyApp()); }

class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: MyHomePage()
    );
  }
}

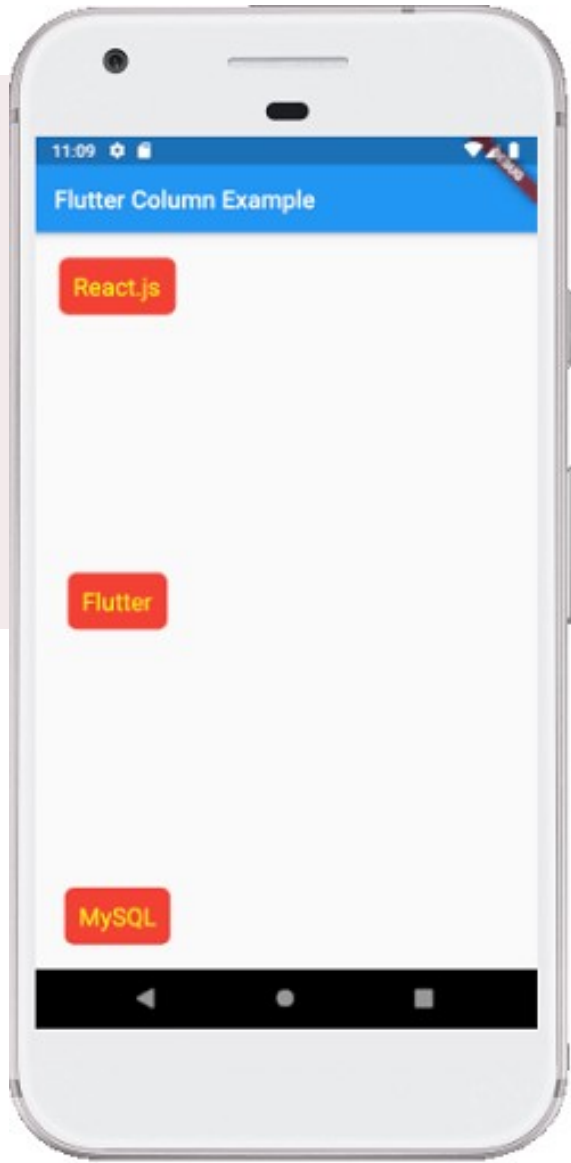
class MyHomePage extends StatefulWidget {
  @override
  _MyHomePageState createState() => _MyHomePageState();
}
```



```
class _MyHomePageState extends State<MyHomePage> {  
  @override  
  Widget build(BuildContext context) {  
    return Scaffold(  
      appBar: AppBar(  
        title: Text("Flutter Column Example"),  
      ),  
      body: Column(  
        mainAxisAlignment: MainAxisAlignment.spaceBetween,  
        children:<Widget>[  
          Container(  
            margin: EdgeInsets.all(20.0),  
            padding: EdgeInsets.all(12.0),  
            decoration:BoxDecoration(  
              borderRadius:BorderRadius.circular(8),  
              color:Colors.red  
            ),  
            child: Text("React.js",style: TextStyle(color:Colors.yellowAccent,fontSize:20),),  
          ),  
        ],  
      ),  
    );  
  }  
}
```

```
Container(  
  margin: EdgeInsets.all(20.0),  
  padding: EdgeInsets.all(12.0),  
  decoration: BoxDecoration(  
    borderRadius: BorderRadius.circular(8),  
    color: Colors.red  
  ),  
  child: Text("Flutter", style: TextStyle(color: Colors.yellowAccent, fontSize: 20)),  
,  
Container(  
  margin: EdgeInsets.all(20.0),  
  padding: EdgeInsets.all(12.0),  
  decoration: BoxDecoration(  
    borderRadius: BorderRadius.circular(8),  
    color: Colors.red  
  ),  
,
```

```
child: Text("MySQL",style: TextStyle(color:Colors.yellowAccent,fontSize:20)),  
    )  
  ]  
),  
);  
}  
}
```



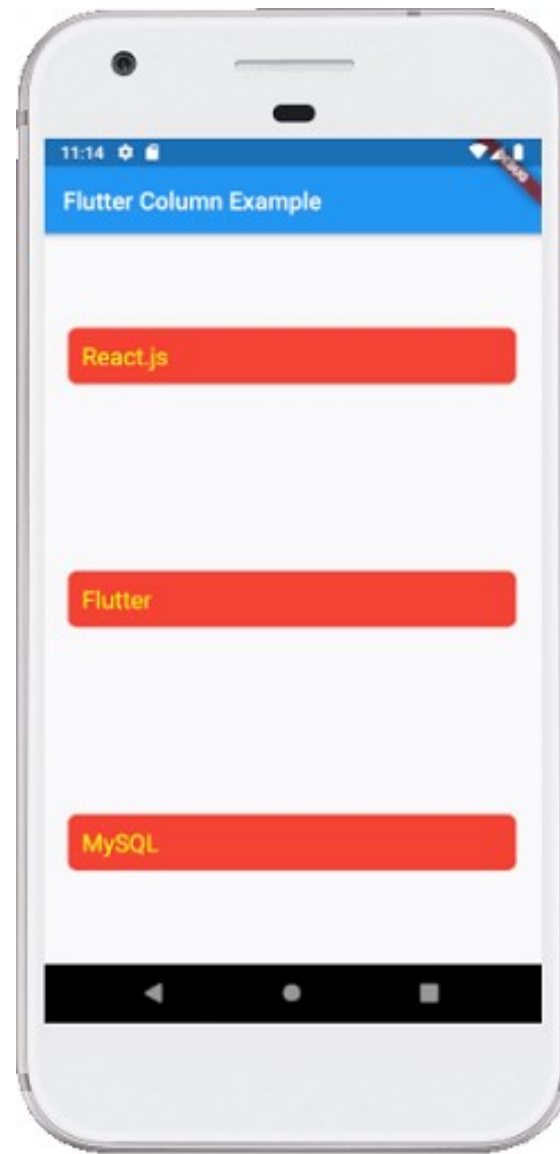
Flutter also allows developers to align the child widget with a combination of `crossAxisAlignment` and `mainAxisAlignment` for both row and column widget.

Let us take the above example of column widget where we will set `mainAxisAlignment` as `MainAxisAlignment.spaceAround` and `crossAxisAlignment` is `CrossAxisAlignment.stretch`.

It will make the height of the column is equal to the height of the body.

See the below screenshot.





Drawbacks OF Row and Column Widget:

- ❑ Row widget in Flutter does not have horizontal scrolling. So if we have inserted a large number of children in a single row that cannot be fit in that row, we will see the Overflow message.
- ❑ Column widget in Flutter does not have vertical scrolling. So if we have inserted a large number of children in a single column whose total children size is not equal to the height of the screen, we will see the Overflow message.



Mobile Applications Developments



COMPUTER SYSTEMS ENGINEERING

LECTURER: AHMAD SHAQALIEH

SECOND SEMESTER 2025

LECTURE 6

Flutter Text



A Text is a widget in Flutter that allows us to **display a string of text with a single line in our application.**

Depending on the layout constraints, we can break the string across multiple lines or might all be displayed on the same line.

If we do not specify any styling to the text widget, it will use the closest **DefaultTextStyle** class style. This class does not have any explicit style.

Here is a simple example to understand this widget. This example shows our **project's title** in the application bar and a **message** in the application's body.



```
import 'package:flutter/material.dart';
```

```
void main() { runApp(MyApp()); }
```

```
class MyApp extends StatelessWidget {
```

```
  @override
```

```
  Widget build(BuildContext context) {
```

```
    return MaterialApp(
```

```
      theme: ThemeData(
```

```
        primarySwatch: Colors.green,
```

```
      ),
```

```
      home: MyTextPage()
```

```
    );
```

```
  }
```

```
}
```

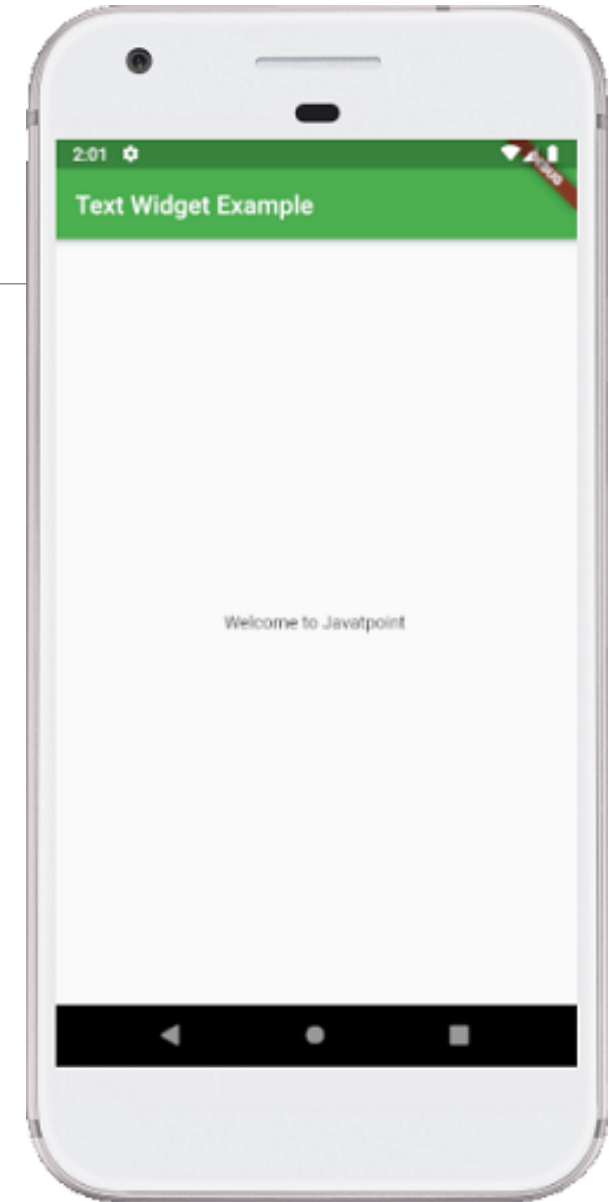
```
class MyTextPage extends StatelessWidget {  
  @override  
  Widget build(BuildContext context) {  
    return Scaffold(  
      appBar: AppBar(  
        title:Text("Text Widget Example")  
      ),  
      body: Center(  
        child:Text("Welcome to Javatpoint")  
      ),  
    );  
  }  
}
```

In the previous code, we have used a **MaterialApp** widget that calls the home screen using the **MyTextPage()** class.

This class contains the **scaffold** widget, which has **appBar** and **body** where we have used the **Text** widget to display the title and body, respectively.

It is a simple scenario of Text widget where we have to pass the string that we want to display on our page.

When we run this application in the emulator or device, we should get the UI similar to screenshot:



Text Widget Constructor

The text widget constructor used to make the custom look and feel of our text in Flutter:

```
const Text(String data,{  
  Key key,  
  TextStyle style,  
  StrutStyle strutStyle,  
  TextAlign textAlign,  
  TextDirection textDirection,  
  TextOverflow overflow,  
  bool softWrap,  
  double textScaleFactor,  
  int maxLines,  
  String semanticsLabel,  
  TextWidthBasis textWidthBasis,  
  TextHeightBehavior textHeightBehavior  
})
```

Text Widget Constructor

The following are the essential properties of the Text widget used in our application:

TextAlign: It is used to specify how our text is aligned horizontally. It also controls the text location.

```
Text(  
    'Hello World!',  
    textAlign: TextAlign.start,  
)
```

TextDirection: It is used to determine how text Align values control the layout of our text. Usually, we write text from left to right, but we can change it using this parameter.

```
Text(  
    "Welcome Mr's أحمد",  
    textDirection: TextDirection.ltr  
),
```

Welcome Mr's أحمد

```
Text(  
    "Welcome Mr's أحمد",  
    textDirection: TextDirection.rtl  
),
```

أحمد Welcome Mr's

Overflow: It is used to determine when the text will not fit in the available space. It means we have specified more text than the available space.(...,cut,fade,visible)

TextScaleFactor: It is used to determine the scaling to the text displayed by the Text widget. Suppose we have specified the text scale factor as 1.5, then our text will be 50 percent larger than the specified font size.(ex: use in zoom)

SoftWrap: It is used to determine whether or not to show all text widget content when there is not enough space available. If it is true, it will show all content. Otherwise, it will not show all content.

MaxLines: It is used to determine the maximum number of lines displayed in the text widget.

Ex: 3 lines,3 points (Overflow,SoftWrap,Maxlines).

TextHeightBehavior: It is used to control how the paragraph appears between the first line and descent of the last line.(f,l,b)

Style: It is the most common property of this widget that allows developers to styling their text. It can do styling by specifying the background color, font size, font weight, letter and word spacing, shadows, etc.

See the table to understand it more easily:

Attributes	Descriptions
background	It determines the paint as a background for the text.
fontWeight	It determines the thickness of the text.
fontSize	It determines the size of the text.
fontFamily	It is used to specify the typeface for the font. For this, we need to download a typeface file in our project, and then keep this file into the assets/font folder. Finally, config the pubspec.yaml file to use it in the project.
fontStyle	It is used to style the font either in bold or italic form.

Attributes	Descriptions
Color	It is used to determine the color of the text.
letterSpacing	It is used to determine the distance between the characters of the text.
wordSpacing	It is used to specify the distance between two words of the text.
shadows	It is used to paint underneath the text.

TextDecorationStyle() const

Values

solid → const TextDecorationStyle to Draw a solid line
double → const TextDecorationStyle to Draw two lines
dotted → const TextDecorationStyle to Draw a dotted line
dashed → const TextDecorationStyle to Draw a dashed line
wavy → const TextDecorationStyle to Draw a sinusoidal line

Styling text in Flutter
Styling text in Flutter
Styling text in Flutter
Styling text in Flutter
Styling text in Flutter
Styling text in Flutter
Styling text in Flutter
Styling text in Flutter
Styling text in Flutter

```
import 'package:flutter/material.dart';
```

```
void main() { runApp(MyApp()); }
```

```
class MyApp extends StatelessWidget {
```

```
  @override
```

```
  Widget build(BuildContext context) {
```

```
    return MaterialApp(
```

```
      theme: ThemeData(
```

```
        primarySwatch: Colors.green,
```

```
      ),
```

```
      home: MyTextPage()
```

```
    );
```

```
  }
```

```
}
```

```
class MyTextPage extends StatelessWidget {
```

```
  @override
```

```
  Widget build(BuildContext context) {
```

```
    return Scaffold(
```

```
      appBar: AppBar(
```

```
        title: Text("Text Widget Example")
```

```
      ),
```

```
      body: Center(
```

```
        child: Text(
```

```
          "Hello World! This is a Text Widget.",
```

```
          style: TextStyle(
```

```
            fontSize: 35,
```

```
            color: Colors.purple,
```

```
            fontWeight: FontWeight.w700,
```

```
            fontStyle: FontStyle.italic,
```

```
            letterSpacing: 8,
```

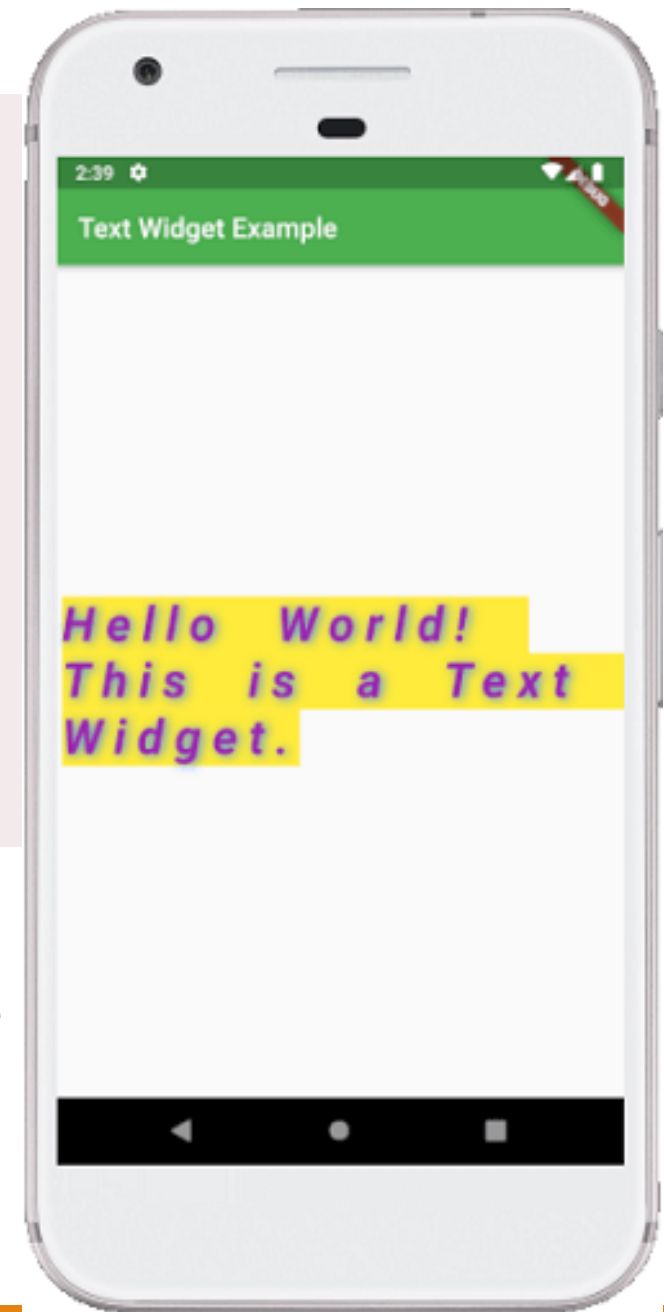
```
            wordSpacing: 20,
```

```
            backgroundColor: Colors.yellow,
```

```
    shadows: [  
      Shadow(color: Colors.blueAccent, offset: Offset(2,1), blurRadius:10)  
    ],  
  ),  
),  
);  
}  
}
```

Output:

When we run this application in the emulator or device, we should get the UI similar to screenshot:



Flutter Rich Text Widget

Rich text

Quill component for Flutter

This *library* supports **mobile** platform **only** and ~~web~~ is not supported.

You are welcome to use [Bullet Journal](#):

Sometimes we want to **show a line or a paragraph with multiple styles** such as bold, italicized, underlined, different color, different font or everything at once.

In that case, we should have to use the RichText widget that allows us to perform multiple text styles without switching many widgets.

RichText is a very useful widget in Flutter, which is used for displaying a paragraph of text on the UI with multiple styles.

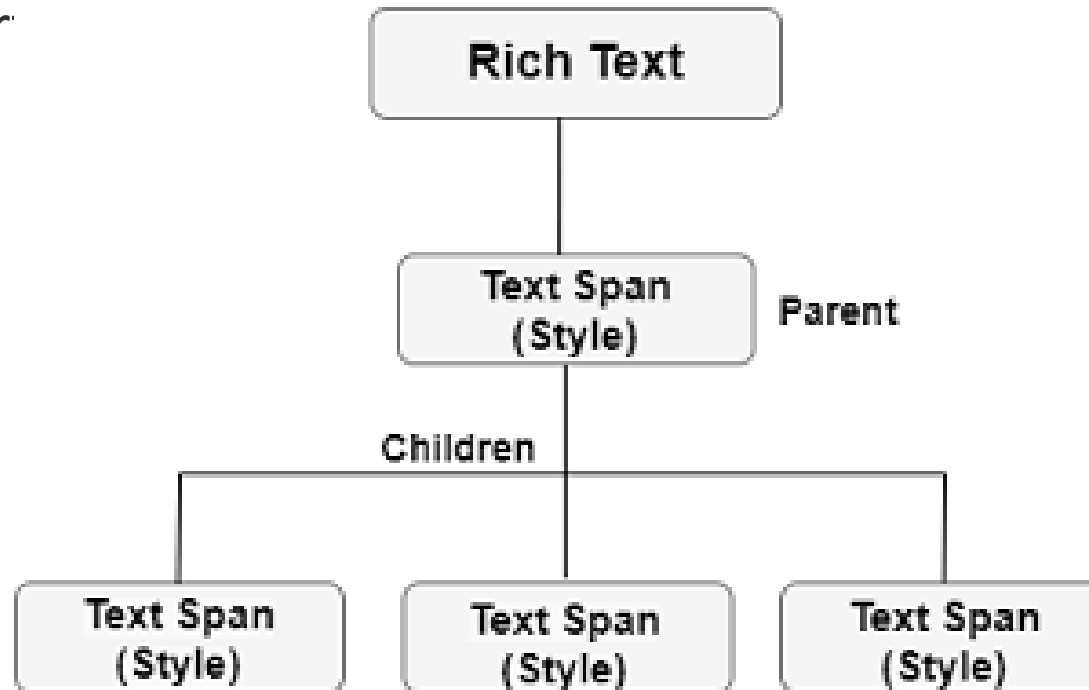
Inside the widget, we can have different styles by giving it a **tree of TextSpan** widgets.

Each TextSpan can set its own style for overriding the default style.

Rich Text Structure

The below image explains the structure of the Rich Text widget.

In this image, the parent Text Span has its own style property and a text parameter, and then it can contain several children Text Span who have their own style proper



Example:

Suppose our app has a login screen and an option to create a new user account.

The creation of new user account section has a combination of regular text and colored text, as shown in the side screen:

The image shows a login form within a light gray container. At the top, the label "Email" is in bold black text, followed by a white rectangular input field with a thin gray border. Below this, the label "Password" is in bold black text, followed by a similar white input field. To the right of the password field is a blue link "Forgot password?". Below the password field is a solid blue button with the text "Log in" in white. At the bottom of the container is a light gray footer area containing the text "Don't have an account? Sign up", where "Sign up" is a blue link.

Email

Password [Forgot password?](#)

[Log in](#)

Don't have an account? [Sign up](#)

In the previous image, we can see the text "**Don't have an account? Sign up**" is a single line text, where the last part is a clickable word that navigates the user on the sign-up page.

To make this type of text, we will **divide** it into two parts.

The first part uses the **parent** TextSpan and input the text "**Don't have an account?**" with the **black** color.

The second part uses the **children** TextSpan and input the text "**Sign up**" with a **blue** color.

See the below code:

```
RichText(  
  text: TextSpan(  
    text: 'Don\'t have an account?',  
    style: TextStyle(color: Colors.black, fontSize: 20),  
    children: <TextSpan>[  
      TextSpan(text: ' Sign up',  
        style: TextStyle(color: Colors.blueAccent, fontSize: 20)  
      )  
    ]  
  ),  
)
```

Since the "Sign up" is a clickable text, so we need to implement **onTap()** action on this part.

The TextSpan contains **TapGestureRecognizer()** that implement onTap() action.

In our example, we will use the recognizer property to make the text tappable.

Let us show the entire code snippet to understand it more clearly.

```
import 'package:flutter/material.dart';
```

```
import 'package:flutter/gestures.dart';
```

```
void main() { runApp(MyApp()); }
```

```
class MyApp extends StatelessWidget {
```

```
  @override
```

```
  Widget build(BuildContext context) {
```

```
    return MaterialApp(
```

```
      theme: ThemeData(
```

```
        primarySwatch: Colors.green,
```

```
      ),
```

```
      home: MyTextPage()
```

```
    );
```

```
  }
```

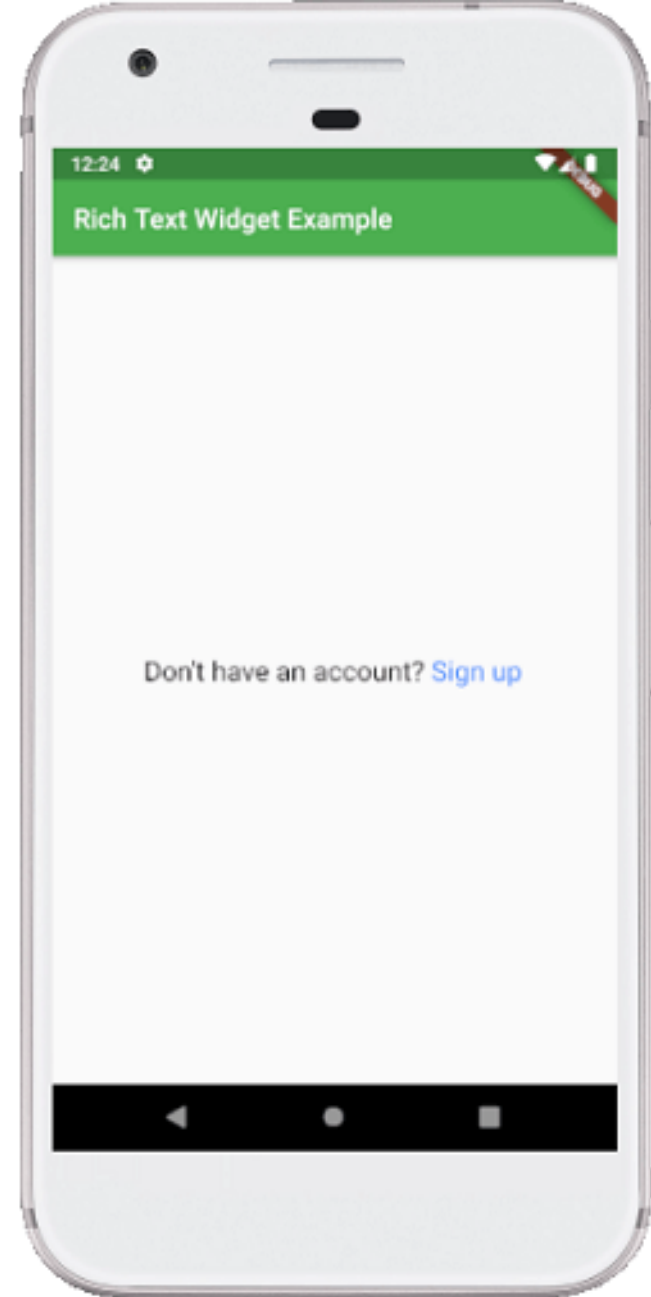
```
}
```

```
class MyTextPage extends StatelessWidget {  
  @override  
  Widget build(BuildContext context) {  
    return Scaffold(  
      appBar: AppBar(  
        title: Text("Rich Text Widget Example")  
      ),  
      body: Container(  
        child: Center(  
          child: RichText(  
            text: TextSpan(  
              text: 'Don\'t have an account?',  
              style: TextStyle(color: Colors.black, fontSize: 20),  
              children: <TextSpan>[  
                TextSpan(text: ' Sign up',  
                  style: TextStyle(color: Colors.blueAccent, fontSize: 20),  
                  recognizer: TapGestureRecognizer()  
                    ..onTap = () {}  
                ]  
            )  
          )  
        )  
      )  
    );  
  }  
}
```

```
    )  
  ]  
  ),  
),  
)  
)  
);  
}  
}
```

Output:

When we run this application in the emulator or device, we should get the UI similar to the screenshot:



How to show an icon in a text widget in Flutter?

Sometimes the developers need to show an **icon** with the text widget.

To do this, Flutter provides a **WidgetSpan()** inside the `RichText()` widget to add an icon with the text widget.

The below example explains it in a simple way:

```
import 'package:flutter/material.dart';
```

```
void main() { runApp(MyApp()); }
```

```
class MyApp extends StatelessWidget {
```

```
  @override
```

```
  Widget build(BuildContext context) {
```

```
    return MaterialApp(
```

```
      home: MyTextPage()
```

```
    );
```

```
  }
```

```
}
```

```
class MyTextPage extends StatelessWidget {
```

```
  @override
```

```
  Widget build(BuildContext context) {
```

```
    return Scaffold(
```

```
      appBar: AppBar(
```

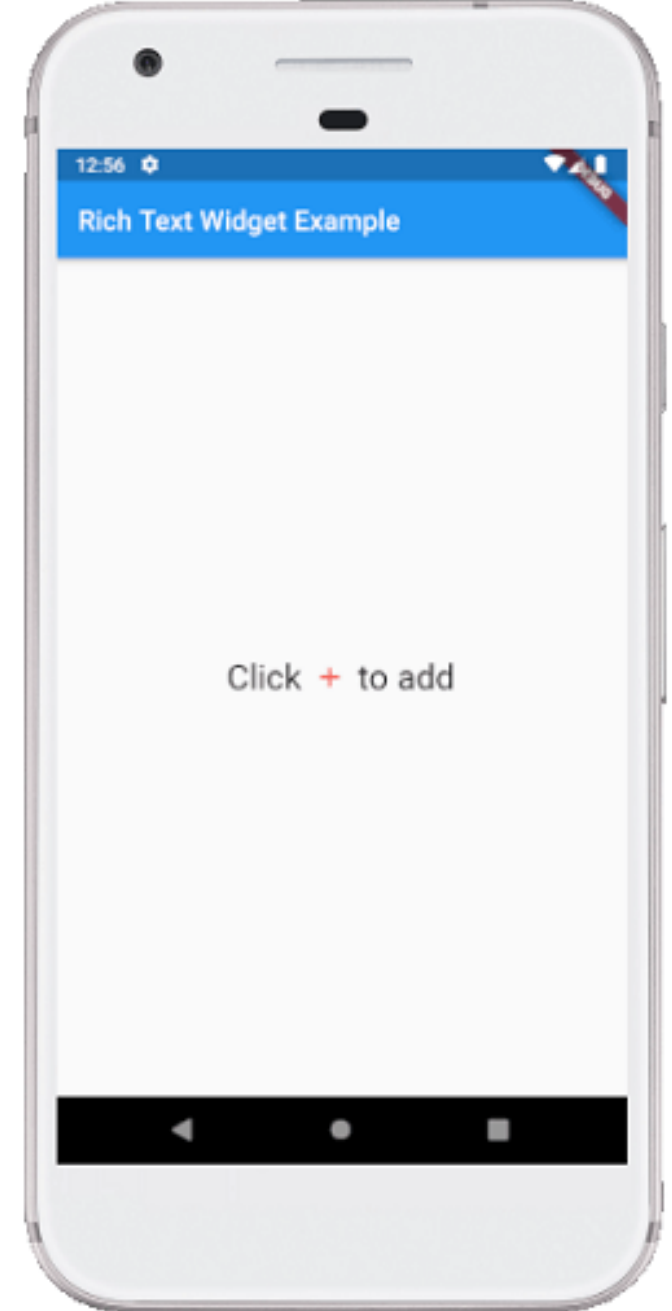
```
        title: Text("Rich Text Widget Example")
```

```
      ),
```

```
body: Container(  
  child: Center(  
    child: RichText(  
      text: TextSpan(  
        style: Theme.of(context).textTheme.body1,  
        children: [  
          TextSpan(text: 'Click ', style: TextStyle(fontSize: 25)),  
          WidgetSpan(  
            child: Padding(  
              padding: const EdgeInsets.symmetric(horizontal: 2.0),  
              child: Icon(Icons.add, color: Colors.red),  
            ),  
          ),  
          TextSpan(text: ' to add', style: TextStyle(fontSize: 25)),  
        ],  
      ),  
    ),  
  ),  
);  
}
```

Output:

When we run this application in the emulator or device, we should get the UI similar to screenshot:





Mobile Applications Developments



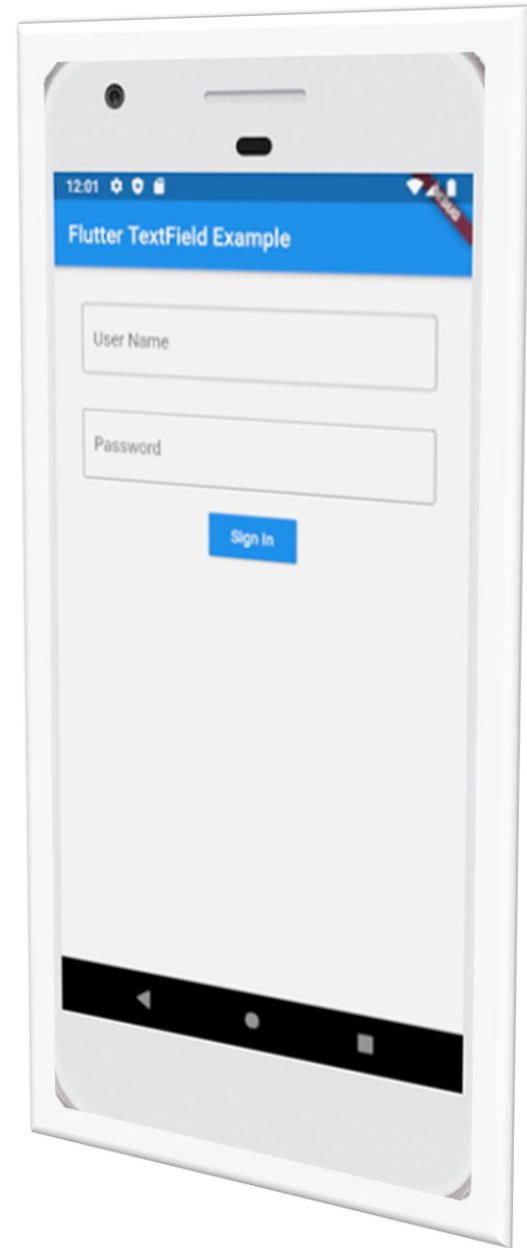
COMPUTER SYSTEMS ENGINEERING

LECTURER: AHMAD SHAQALIEH

SECOND SEMESTER 2025

LECTURE 7

Flutter Text Field



A TextField or TextBox is an **input element** which holds the alphanumeric data, such as name, password, address, etc.

It is a GUI control element that enables the user to **enter text information** using a programmable code.

It can be of a single-line text field (when only one line of information is required) or multiple-line text field (when more than one line of information is required).

TextField in Flutter is the most commonly used **text input widget** that allows users to collect inputs from the keyboard into an app.

We can use the **TextField** widget in building forms, sending messages, creating search experiences, and many more.

By default, Flutter decorated the Text Field with an underline. We can also add several attributes with Text Field, such as label, icon, inline hint text, and error text using an Input Decoration as the decoration.

If we want to remove the decoration properties entirely, it is required to set the decoration to **null**.

The following code explains

a **demo example of Text Filed widget** in Flutter:

```
TextField (  
  decoration: InputDecoration(  
    border: InputBorder.none,  
    labelText: 'Enter Name',  
    hintText: 'Enter Your Name'  
  ),  
);
```

Some of the most common attributes used with the TextField widget are as follows:

- ❑ **decoration:** It is used to show the decoration around TextField.
- ❖ **border:** It is used to create a default rounded rectangle border around TextField.
- ❖ **labelText:** It is used to show the label text on the selection of TextField.
- ❖ **hintText:** It is used to show the hint text inside TextField.
- ❖ **icon:** It is used to add icons directly to the TextField.

We are going to see how to use TextField widget in the Flutter app through the following steps:

Constructors

```
TextField({Key? key, TextEditingController? controller, FocusNode? focusNode, InputDecoration? decoration = const InputDecoration(), TextInputType? keyboardType, TextInputAction? textInputAction, TextCapitalization textCapitalization = TextCapitalization.none, TextStyle? style, StrutStyle? strutStyle, TextAlign textAlign = TextAlign.start, TextAlignVertical? textAlignVertical, TextDirection? textDirection, bool readOnly = false, @Deprecated('Use `contextMenuBuilder` instead. ' 'This feature was deprecated after v3.3.0-0.5.pre.') ToolbarOptions? toolbarOptions, bool? showCursor, bool autofocus = false, String obscuringCharacter = '•', bool obscureText = false, bool autocorrect = true, SmartDashesType? smartDashesType, SmartQuotesType? smartQuotesType, bool enableSuggestions = true, int? maxLines = 1, int? minLines, bool expands = false, int? maxLength, MaxLengthEnforcement? maxLengthEnforcement, ValueChanged<String>? onChanged, VoidCallback? onEditingComplete, ValueChanged<String>? onSubmitted, AppPrivateCommandCallback? onAppPrivateCommand, List<TextInputFormatter>? inputFormatters, bool? enabled, double cursorWidth = 2.0, double? cursorHeight, Radius? cursorRadius, Color? cursorColor, BoxHeightStyle selectionHeightStyle = ui.BoxHeightStyle.tight, BoxWidthStyle selectionWidthStyle = ui.BoxWidthStyle.tight, Brightness? keyboardAppearance, EdgeInsets scrollPadding = const EdgeInsets.all(20.0), DragStartBehavior dragStartBehavior = DragStartBehavior.start, bool? enableInteractiveSelection, TextSelectionControls? selectionControls, GestureTapCallback? onTap, TapRegionCallback? onTapOutside, MouseCursor? mouseCursor, InputCounterWidgetBuilder? buildCounter, ScrollController? scrollController, ScrollPhysics? scrollPhysics, Iterable<String>? autofillHints = const <String>[], Clip clipBehavior = Clip.hardEdge, String? restorationId, bool scribbleEnabled = true, bool enableIMEPersonalizedLearning = true, EditableTextContextMenuBuilder? contextMenuBuilder = _defaultContextMenuBuilder, SpellCheckConfiguration? spellCheckConfiguration, TextMagnifierConfiguration? magnifierConfiguration})
```

Creates a Material Design text field.

const

1

Label

Label
| Input text

2

Label

Label
| Input text

Dart

Install SDK

Format

Reset

▶ Run

⋮

DEBUG

```
41 @override
42 Widget build(BuildContext context) {
43   return Scaffold(
44     body: Center(
45       child: TextField(
46         decoration: InputDecoration(
47           border: OutlineInputBorder(),
48           icon: Icon(Icons.send),
49           prefixIcon: Icon(Icons.add),
50           suffixIcon: Icon(Icons.home),
51         ),
52         controller: _controller,
53         onSubmitted: (String value) async {
54           await showDialog<void>(
55             context: context,
56             builder: (BuildContext context) {
57               return AlertDialog(
58                 title: const Text('Thanks!'),
59                 content: Text(
60                   'You typed "$value", which has length ${value.characters.length}',
61                 ),
62                 actions: <Widget>[
```



1

2

3



+

2

3



Console



5 issues

show

Step 1: Create a Flutter project in the IDE you used. Here, I am going to use Android Studio.

Step 2: Open the project in Android Studio and navigate to the **lib** folder. In this folder, open the **main.dart** file and import the **material.dart** package as given below:

```
import 'package:flutter/material.dart';
```

Step 3: Next, call the main **MyApp** class using void main run app function and then create your main widget class named as **MyApp** extends with **StatefulWidget**:

```
void main() => runApp( MyApp() );
```

```
class MyApp extends StatefulWidget { }
```


Step 4: Next, we need to create the Scaffold widget -> Column widget in the class widget build area as given below:

```
class MyApp extends StatefulWidget {  
  @override  
  Widget build(BuildContext context) {  
    return Scaffold(  
      appBar: AppBar(  
        title: Text('Flutter TextField Example'),  
      ),  
      body: Padding(  
        padding: EdgeInsets.all(15),  
        child: Column(  
          children: <Widget> [  
  
          ]  
        )  
      )  
    );  
  }  
}
```

Step 5: Finally, create the TextField widget as the below code.

```
child: TextField(  
  obscureText: true,  
  decoration: InputDecoration(  
    border: OutlineInputBorder(),  
    labelText: 'Password',  
    hintText: 'Enter Password',  
  ),  
)
```

Let us see the complete source code that contains the TextField Widget.

This Flutter application takes **two TextFields and one RaisedButton**.

After filling the details, the user clicks on the button.

Since we have not specified any value in the **onPressed ()** property of the button, it cannot print them to console.

```
import 'package:flutter/material.dart';

void main() {
  runApp(MaterialApp( home: MyApp(),));
}

class MyApp extends StatefulWidget {
  @override
  _State createState() => _State();
}

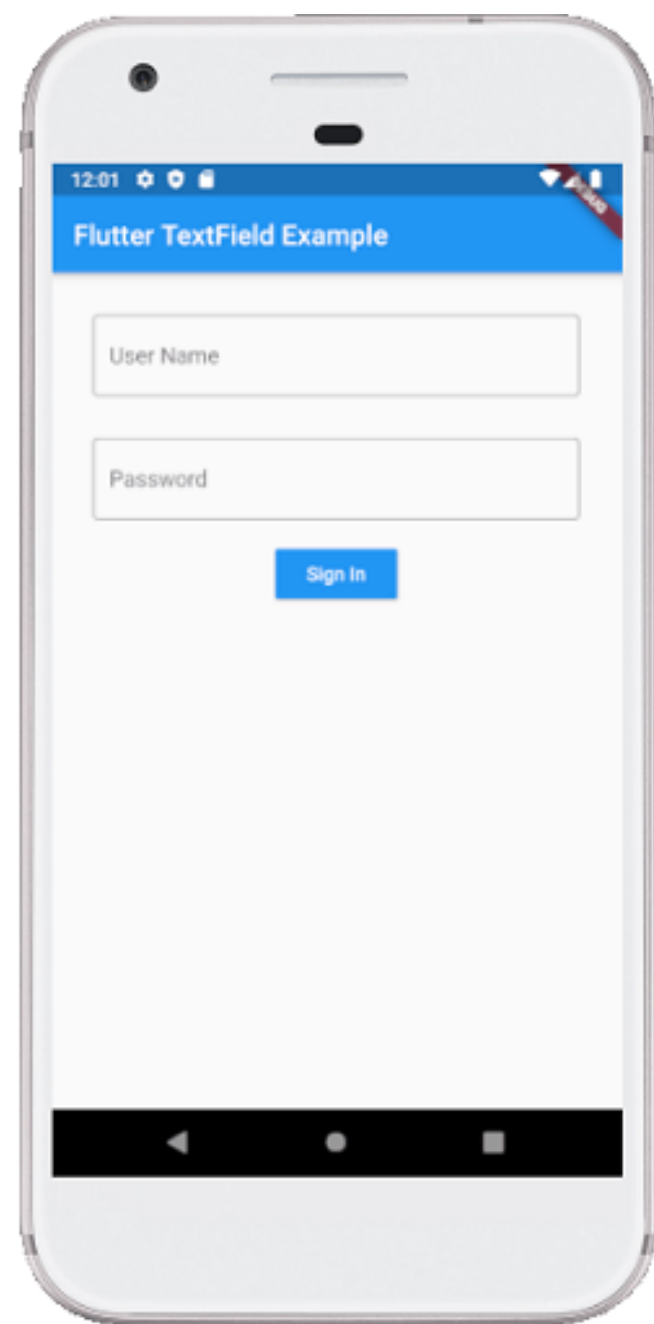
class _State extends State<MyApp> {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text('Flutter TextField Example'),
      ),
```

```
body: Padding(  
  padding: EdgeInsets.all(15),  
  child: Column(  
    children: <Widget>[  
      Padding(  
        padding: EdgeInsets.all(15),  
        child: TextField(  
          decoration: InputDecoration(  
            border: OutlineInputBorder(),  
            labelText: 'User Name',  
            hintText: 'Enter Your Name',  
          ),  
        ),  
      ),  
    ],  
  ),  
)
```

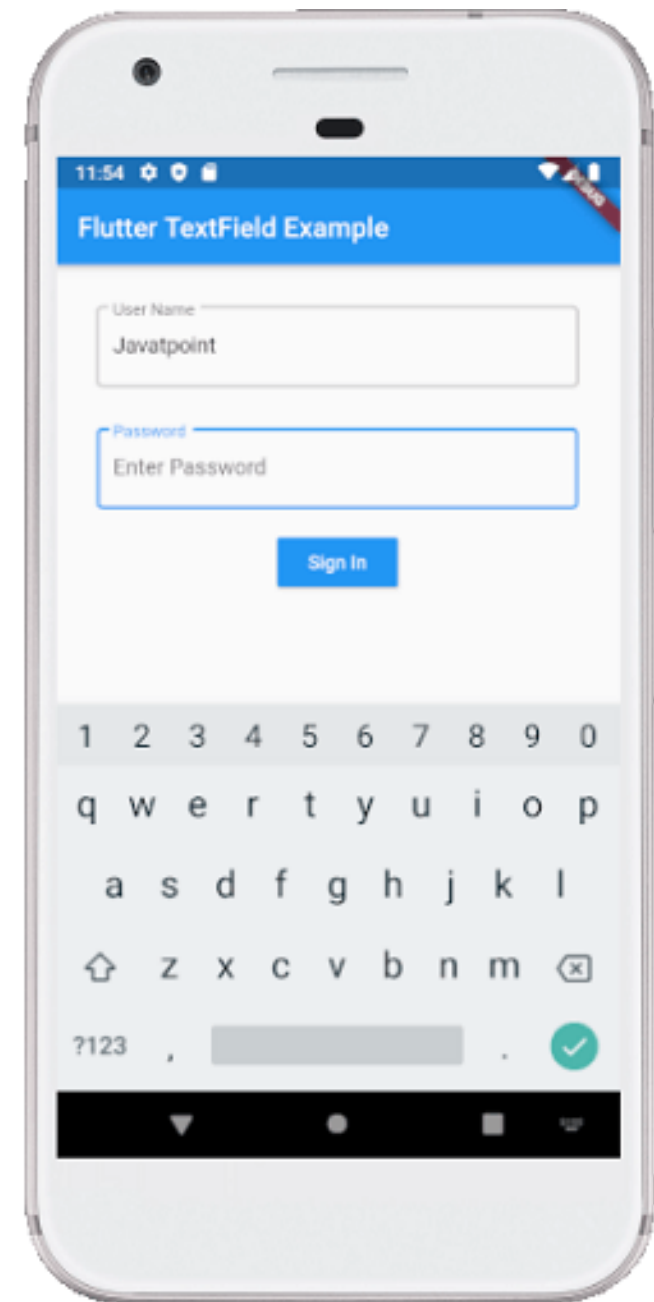
```
Padding(  
  padding: EdgeInsets.all(15),  
  child: TextField(  
    obscureText: true,  
    decoration: InputDecoration(  
      border: OutlineInputBorder(),  
      labelText: 'Password',  
      hintText: 'Enter Password',  
    ),  
  ),  
),  
),  
RaisedButton(  
  textColor: Colors.white,  
  color: Colors.blue,  
  child: Text('Sign In'),  
  onPressed: () {},  
)  
],  
)  
)  
);  
}  
}
```

Output

When we run the application in android emulator, we should get UI similar to the following screenshot:



If we click inside the text box, we can see that a keyboard has appeared from the bottom of the screen, the label goes into the top left corner of the border, and the hint text shown into the field. The below screenshot explains it more clearly:



How to retrieve the value of a TextField?

We know that Flutter does not have an ID like in Android for the TextField widget.

Flutter allows the user to **retrieve the text in mainly two ways**: First is the onChanged method, and another is the controller method.

1. onChanged method:

It is the easiest way to retrieve the text field value.

This method store the current value in a simple variable and then use it in the TextField widget. Below is the sample example:

```
String value = "";  
TextField(  
  onChanged: (text) {  
    value = text;  
  },  
)
```

2. Controller method:

It is a popular method to retrieve text field value using **TextEditingController**.

It will be attached to the TextField widget and then listen to change and control the widget's text value.

Below is the sample code:

```
TextEditingController mycontroller = TextEditingController();  
TextField(  
  controller: mycontroller,  
)
```

Sample code for listening to the changes.

```
controller.addListener() {  
    // Do something here  
});
```

Sample code to get or set the value.

```
print(controller.text); // Print current value  
controller.text = "Demo Text"; // Set new value
```

Let us see the second way in detail to retrieve the text field value in Flutter application with the help of following steps:

1. Create a `TextEditingController`.
2. Attach the `TextEditingController` to a `TextField` using `controller` property.
3. Retrieve the value of the `TextField` by using the **`text()`** method provided by the `TextEditingController`.

In this example, we are going to display the **alert dialog** with the current value of the text field when the user taps on a button.

```
import 'package:flutter/material.dart';
```

```
void main() {
```

```
  runApp(MaterialApp( home: MyApp().));
```

```
}
```

```
class MyApp extends StatefulWidget {
```

```
  @override
```

```
  _State createState() => _State();
```

```
}
```

```
class _State extends State<MyApp> {
```

```
  TextEditingController nameController = TextEditingController();
```

```
  TextEditingController passwordController = TextEditingController();
```

```
  @override
```

```
  Widget build(BuildContext context) {
```

```
    return Scaffold(
```

```
      appBar: AppBar(
```

```
        title: Text('Flutter TextField Example'),
```

```
      ),
```

```
body: Padding(  
  padding: EdgeInsets.all(15),  
  child: Column(  
    children: <Widget>[  
      Padding(  
        padding: EdgeInsets.all(15),  
        child: TextField(  
          controller: nameController,  
          decoration: InputDecoration(  
            border: OutlineInputBorder(),  
            labelText: 'User Name',  
            hintText: 'Enter Your Name',  
          ),  
        ),  
      ),  
    ],  
  ),  
),
```

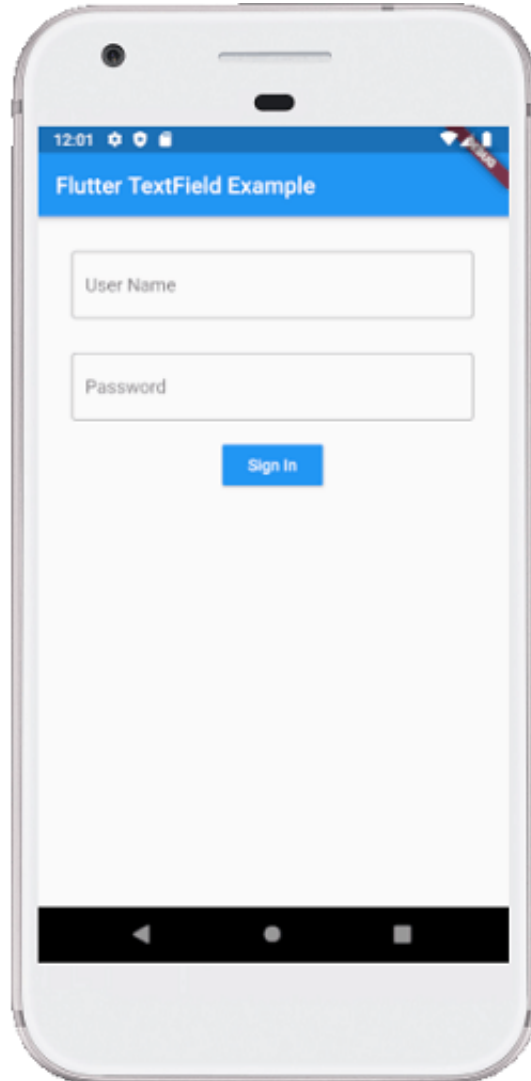
```
Padding(  
  padding: EdgeInsets.all(15),  
  child: TextField(  
    controller: passwordController,  
    obscureText: true,  
    decoration: InputDecoration(  
      border: OutlineInputBorder(),  
      labelText: 'Password',  
      hintText: 'Enter Password',  
    ),  
  ),  
)
```

```
RaisedButton(  
  textColor: Colors.white,  
  color: Colors.blue,  
  child: Text('Sign In'),  
  onPressed: () {  
    return showDialog(  
      context: context,  
      builder: (context) {  
        return AlertDialog(  
          title: Text("Alert Message"),  
          // Retrieve the text which the user has entered by  
          // using the TextEditingController.  
          content: Text(nameController.text),  
        );  
      },  
    );  
  },  
);
```

```
actions: <Widget>[
  new FlatButton(
    child: new Text('OK'),
    onPressed: () {
      Navigator.of(context).pop();
    },
  )
],
);
},
);
},
)
],
)
);
}
}
```

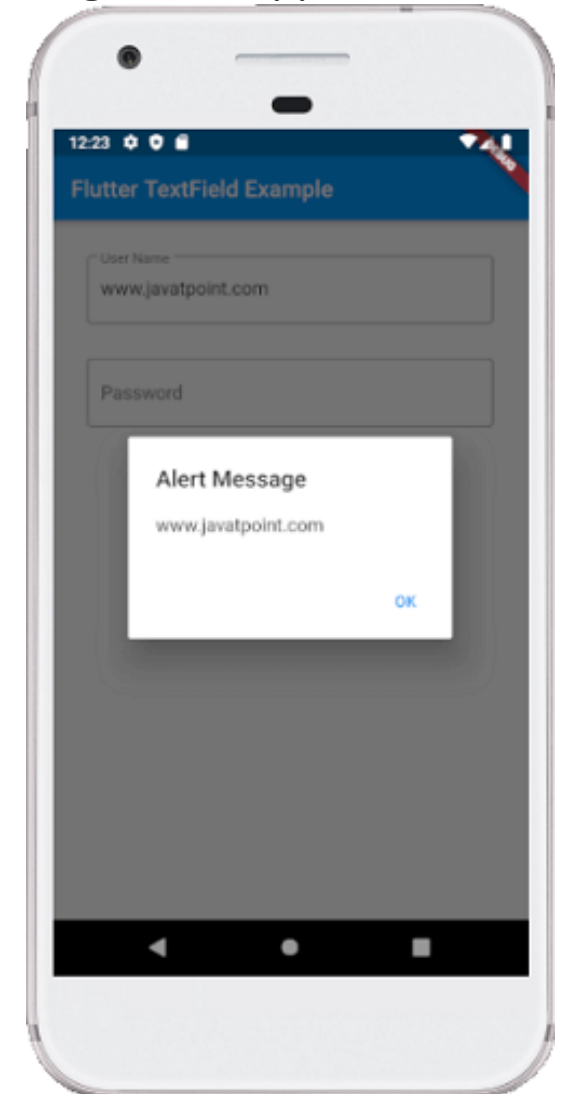
Output:

When we run the application in android emulator, we should get UI similar to the following screenshot:



Click inside the text box and add values, as shown in the field.

When we press the **sign-in button**, an alert dialog box appeared containing the text that the user has entered into the field. If we click on the **OK** button, the alert dialog will disappear. Look into the below screenshot:



How to make TextField expandable?

Sometimes, we want to expand a TextField that means it can have more than one line.

Flutter can do this very easily by adding the attributes **maxLines** and set it to **null**, which is one by default.

We can also specify the exact value to expand the number of lines by default.

```
TextField(  
  maxLines: 4,  
)
```

How to control the size of TextField value?

TextField widget also allows us to restrict the maximum number of characters inside the text field.

We can do this by adding the **maxLength** attributes as below:

```
TextField(  
  maxLength: 10,  
),
```

How to obscure text field value?

Obscure means to make the field not readable or not understandable easily.

The obscure text cannot visible clear.

In Flutter, it is mainly used with a text field that contains a password.

We can make the value in a TextField obscure by setting the **obscureText** to **true**.

```
TextField(  
  obscureText: true,  
),
```



Mobile Applications Developments



COMPUTER SYSTEMS ENGINEERING

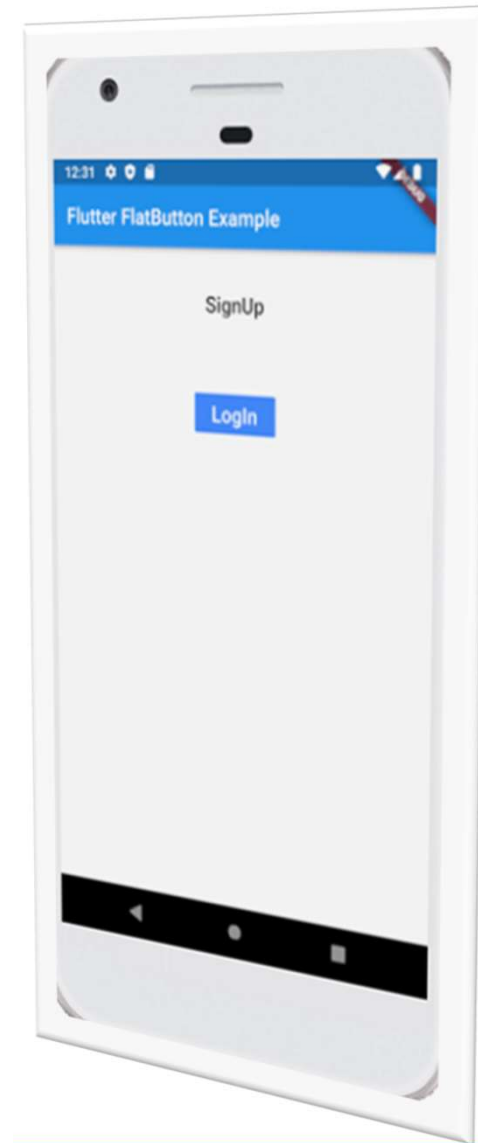
LECTURER: AHMAD SHAQALIEH

SECOND SEMESTER 2025

LECTURE 8

A solid orange horizontal bar spanning the width of the slide at the bottom.

Flutter Buttons



Buttons are the graphical control element that **provides a user to trigger an event** such as taking actions, making choices, searching things, and many more.

They can be placed anywhere in our UI like dialogs, forms, cards, toolbars, etc.

Buttons are the Flutter widgets, which is a part of the material design library.

Flutter provides several types of buttons that have different shapes, styles, and features.



Features of Buttons

The standard features of a button in Flutter are given below:

- ❑ We can easily apply themes on buttons, shapes, color, animation, and behavior.
- ❑ We can also theme icons and text inside the button.
- ❑ Buttons can be composed of different child widgets for different characteristics.

Types of Flutter Buttons

Following are the different types of button available in Flutter:

Old Widget	New Widget
Flat Button	Text Button
Raised Button	Elevated Button
Floating Button	Floating Action Button
Drop Down Button	-
Icon Button	-
Inkwell	-
PopupMenu Button	-
Outline Button	Outlined Button

1. Flat Button

It is a **text label button** that does not have much decoration and displayed **without any elevation**.

The flat button has two required properties that are: **child and onPressed()**.

It is mostly used in toolbars, dialogs, or inline with other content.

By default, the flat button has no color, and its text is black.

But, we can use color to the button and text using **color and textColor** attributes, respectively.

Example:

```
import 'package:flutter/material.dart';

void main() {
  runApp(MyApp());
}

class MyApp extends StatefulWidget {
  @override
  _MyAppState createState() => _MyAppState();
}

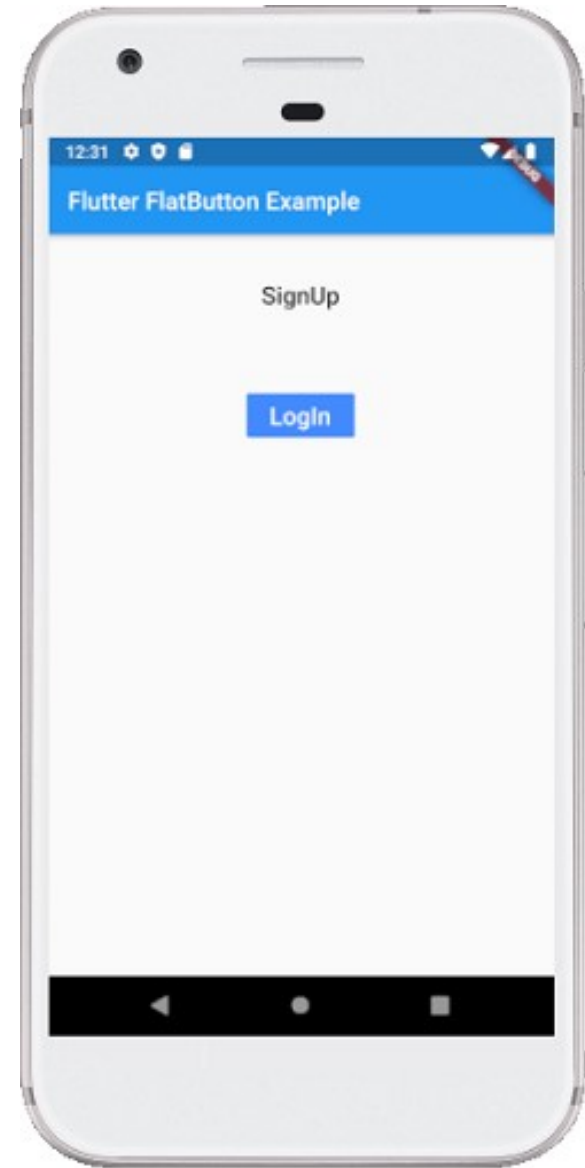
class _MyAppState extends State<MyApp> {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
```

```
home: Scaffold(  
  appBar: AppBar(  
    title: Text('Flutter FlatButton Example'),  
  ),  
  body: Center(child: Column(children: <Widget>[  
    Container(  
      margin: EdgeInsets.all(25),  
      child: FlatButton(  
        child: Text('SignUp', style: TextStyle(fontSize: 20.0)),  
        onPressed: () {},  
      ),  
    ),  
    Container(  
      margin: EdgeInsets.all(25),  
      child: FlatButton(  
        child: Text('LogIn', style: TextStyle(fontSize: 20.0)),  
        color: Colors.blueAccent,  
        textColor: Colors.white,  
        onPressed: () {},  
      ),
```

```
    ),  
  ],  
))  
),  
);  
}  
}
```

Output:

If we run this app, we will see the following screen:



2. Raised Button

It is a button, which is based on the material widget and has a **rectangular body**.

It is similar to a flat button, but it **has an elevation** that will increase when the button is pressed.

It adds dimension to the UI along Z-axis.

It has several properties like text color, shape, padding, button color, the color of a button when disabled, animation time, elevation, etc.

This button has **two callback functions**.

- ❑ **onPressed():** It is triggered when the button is pressed.
- ❑ **onLongPress():** It is triggered when the button is long pressed.

It is to note that this button is in a **disabled state** if `onPressed()` and `onLongPressed()` callbacks are not specified.

Example:

```
import 'package:flutter/material.dart';

void main() {
  runApp(MyApp());
}

class MyApp extends StatefulWidget {
  @override
  _MyAppState createState() => _MyAppState();
}

class _MyAppState extends State<MyApp> {
  String msg = 'Flutter RaisedButton Example';
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
```

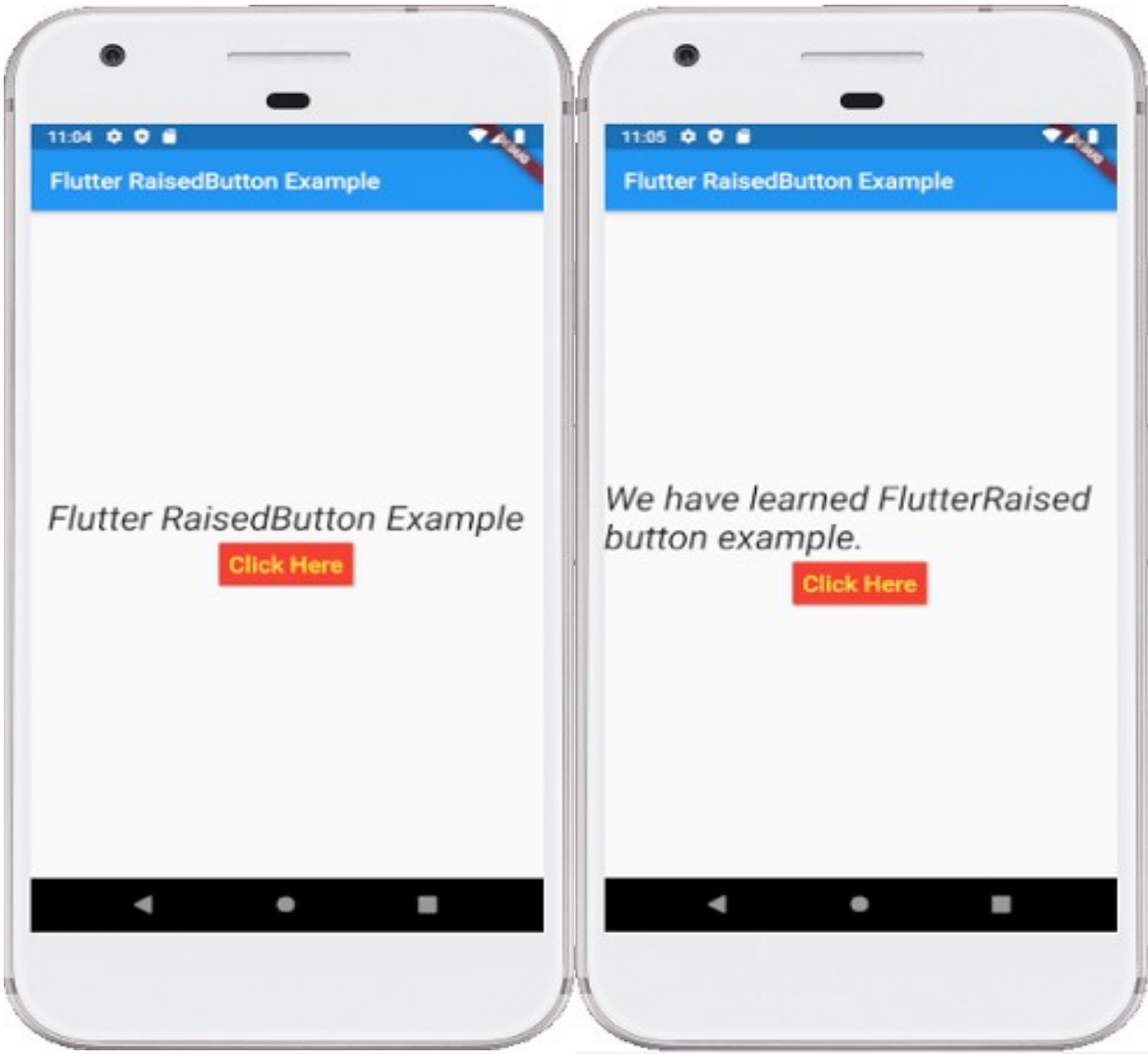
```
appBar: AppBar(  
  title: Text('Flutter RaisedButton Example'),  
),  
body: Container(  
  child: Center(  
    child: Column(  
      mainAxisAlignment: MainAxisAlignment.center,  
      children: [  
        Text(msg, style: TextStyle(fontSize: 30, fontStyle: FontStyle.italic)),  
        RaisedButton(  
          child: Text("Click Here", style: TextStyle(fontSize: 20)),  
          onPressed: _changeText,  
          color: Colors.red,  
          textColor: Colors.yellow,  
          padding: EdgeInsets.all(8.0),  
          splashColor: Colors.grey,  
        )  
      ],  
    ),  
  ),  
),
```



```
    ),  
    ),  
    ),  
    ),  
);  
}  
_changeText() {  
  setState() {  
    if (msg.startsWith('F')) {  
      msg = 'We have learned FlutterRaised button example.';  
    } else {  
      msg = 'Flutter RaisedButton Example';  
    }  
  });  
}  
}
```

Output:

When we run this example, it will give the below screenshot. If we click on the "**Click Here**" button, it will change the text message. Show the second screenshot.



3. Floating Action Button

A FAB button is a **circular icon button** that triggers the primary action in our application. It is the most used button in today's applications. We can use this button for adding, refreshing, or sharing the content. Flutter suggests using at most one FAB button per screen. There are two types of Floating Action Button:

FloatingActionButton: It creates a simple circular floating button with a child widget inside it. It must have a child parameter to display a widget.

FloatingActionButton.extended: It creates a wide floating button along with an icon and a label inside it. Instead of a child, it uses labels and icon parameters.

Example:

```
import 'package:flutter/material.dart';

void main() {
  runApp(MyApp());
}

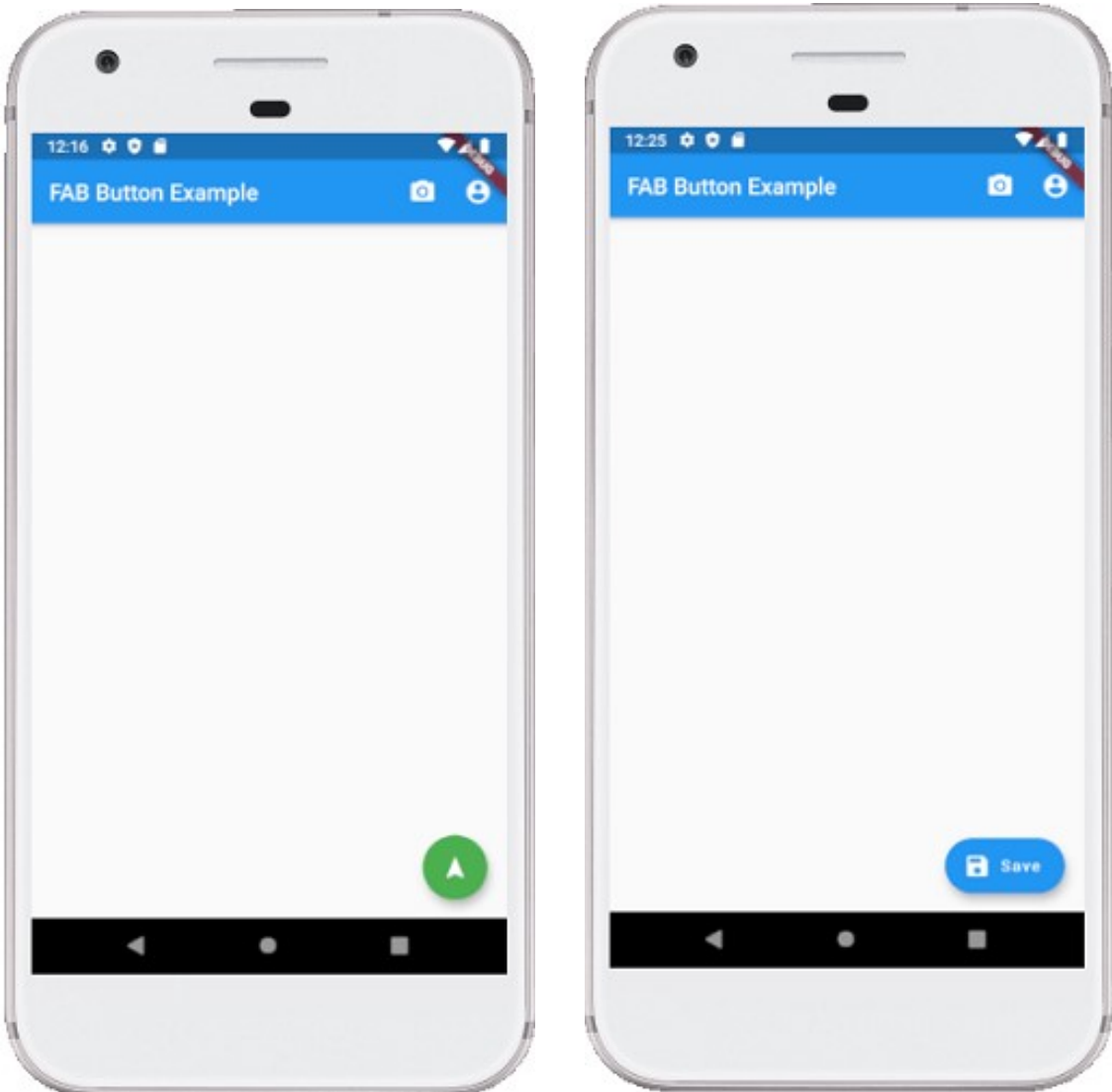
class MyApp extends StatefulWidget {
  @override
  _MyAppState createState() => _MyAppState();
}

class _MyAppState extends State<MyApp> {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(home: Scaffold(
      appBar: AppBar(
        title: Text("FAB Button Example"),
```

```
    backgroundColor: Colors.blue,
    actions: <Widget>[
      IconButton(icon: Icon(Icons.camera_alt), onPressed: () => {}),
      IconButton(icon: Icon(Icons.account_circle), onPressed: () => {}),
    ],
  ),
  floatingActionButton: FloatingActionButton(
    child: Icon(Icons.navigation),
    backgroundColor: Colors.green,
    foregroundColor: Colors.white,
    onPressed: () => {},
  ),
  /*floatingActionButton:FloatingActionButton.extended(
    onPressed: () {},
    icon: Icon(Icons.save),
    label: Text("Save"),
  ), */
),
);
}
}
```

Output:

Run the application in android emulator, and it will give the UI similar to the following screenshot. The second image is an output of the **FAB.extended** button. Its coding can be seen in the above code's comment section.



4. DropDown Button

A drop-down button is used to create a nice overlay on the screen that allows the user to select any item from multiple options. Flutter allows a simple way to implement a drop-down box or drop-down button. This button shows the currently selected item and an arrow that opens a menu to select an item from multiple options.

Flutter provides a **DropDownButton widget** to implement a drop-down list. We can place it anywhere in our app.

Example

```
import 'package:flutter/material.dart';

void main() => runApp(MaterialApp(
  home: MyApp(),
));

class MyApp extends StatefulWidget {
  @override
  _MyAppState createState() => _MyAppState();
}

class _MyAppState extends State<MyApp> {
  List<ListItem> _dropdownItems = [
    ListItem(1, "GeeksforGeeks"),
    ListItem(2, "Javatpoint"),
    ListItem(3, "tutorialandexample"),
    ListItem(4, "guru99")
  ];
}
```



```
List<DropdownMenuItem<ListItem>> _dropdownMenuItems;
ListItem _itemSelected;

void initState() {
  super.initState();
  _dropdownMenuItems = buildDropDownMenuItems(_dropdownItems);
  _itemSelected = _dropdownMenuItems[1].value;
}

List<DropdownMenuItem<ListItem>> buildDropDownMenuItems(List listItems) {
  List<DropdownMenuItem<ListItem>> items = List();
  for (ListItem listItem in listItems) {
    items.add(
      DropdownMenuItem(
        child: Text(listItem.name),
        value: listItem,
      ),
    );
  }
  return items;
}
```

```
@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: Text("DropDown Button Example"),
    ),
    body: Column(
      children: <Widget>[
        Padding(
          padding: const EdgeInsets.all(10.0),
          child: Container(
            padding: const EdgeInsets.all(5.0),
            decoration: BoxDecoration(
              color: Colors.greenAccent,
              border: Border.all(),
            ),
            child: DropdownButtonHideUnderline(
              child: DropdownButton(
                value: _itemSelected,
                items: _dropdownMenuItems,
                onChanged: (value) {
                  setState() {
                    _itemSelected = value;

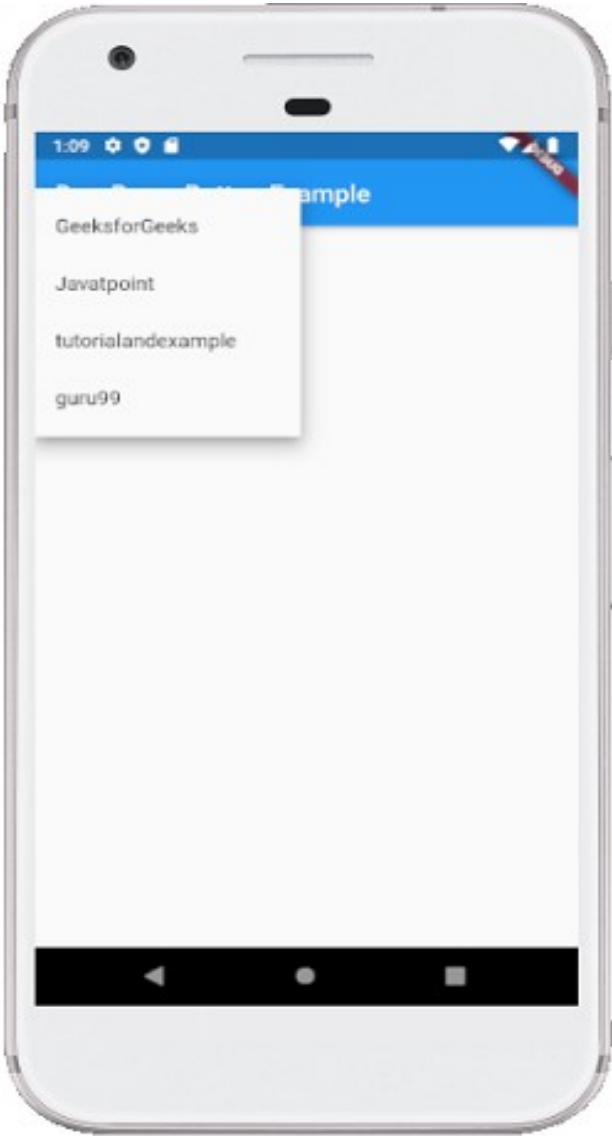
```

```
        });  
    },  
    },  
    },  
    Text("We have selected ${_itemSelected.name}"),  
    ],  
    ),  
    );  
}  
}
```

```
class ListItem {  
    int value;  
    String name;  
  
    ListItem(this.value, this.name);  
}
```

Output

Run the application in android emulator, and it will give the UI similar to the following screenshot. The second image is an output of the list contains in the drop down button.



5. Icon Button

An IconButton is a **picture printed** on the Material widget. It is a useful widget that gives the Flutter UI a material design feel. We can also customize the look and feel of this button. In simple terms, it is an icon that reacts when the user will touch it.

Example:

```
import 'package:flutter/material.dart';

void main() => runApp(MyApp());

class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        appBar: AppBar(
          title: Text("Icon Button Example"),
        ),
        body: Center(
          child: MyStatefulWidget(),
        ),
      ),
    );
  }
}

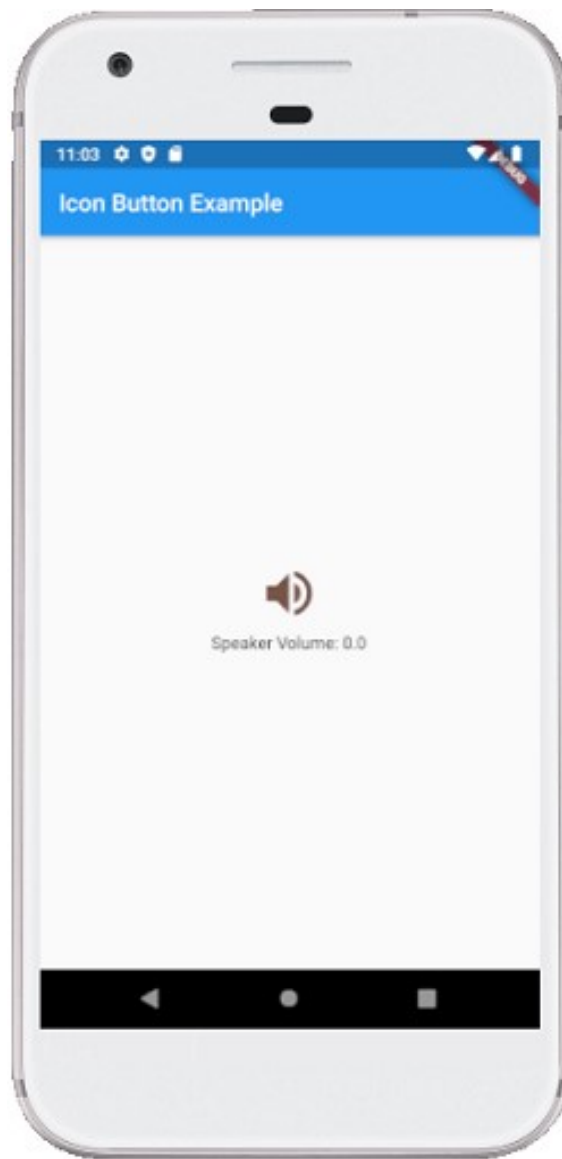
double _speakervolume = 0.0;
```

```
class MyStatefulWidget extends StatefulWidget {  
  MyStatefulWidget({Key key}) : super(key: key);  
  
  @override  
  _MyStatefulWidgetState createState() => _MyStatefulWidgetState();  
}  
  
class _MyStatefulWidgetState extends State<MyStatefulWidget> {  
  Widget build(BuildContext context) {  
    return Column(  
      mainAxisAlignment: MainAxisAlignment.min,  
      children: <Widget>[  
        IconButton(  
          icon: Icon(Icons.volume_up),  
          iconSize: 50,  
          color: Colors.brown,  
          tooltip: 'Increase volume by 5',  
          onPressed: () {  
            setState() {  
              _speakervolume += 5;  
            });  
          },  
        ),  
      ],  
    );  
  }  
}
```

```
        Text('Speaker Volume: $_speakervolume')
    },
);
}
```

Output:

Run the application in android emulator, and it will give the UI similar to the following screenshot. When we press the **volume button**, it will always increase by 5.



6. Inkwell Button

InkWell button is a material design concept, which is used for **touch response**. This widget comes under the Material widget where the ink reactions are actually painted. It creates the app UI interactive by adding gesture feedback. It is mainly used for adding **splash ripple effect**.

Example:

```
import 'package:flutter/material.dart';

void main() => runApp(MyApp());

class MyApp extends StatefulWidget {
  @override
  _MyAppState createState() => _MyAppState();
}

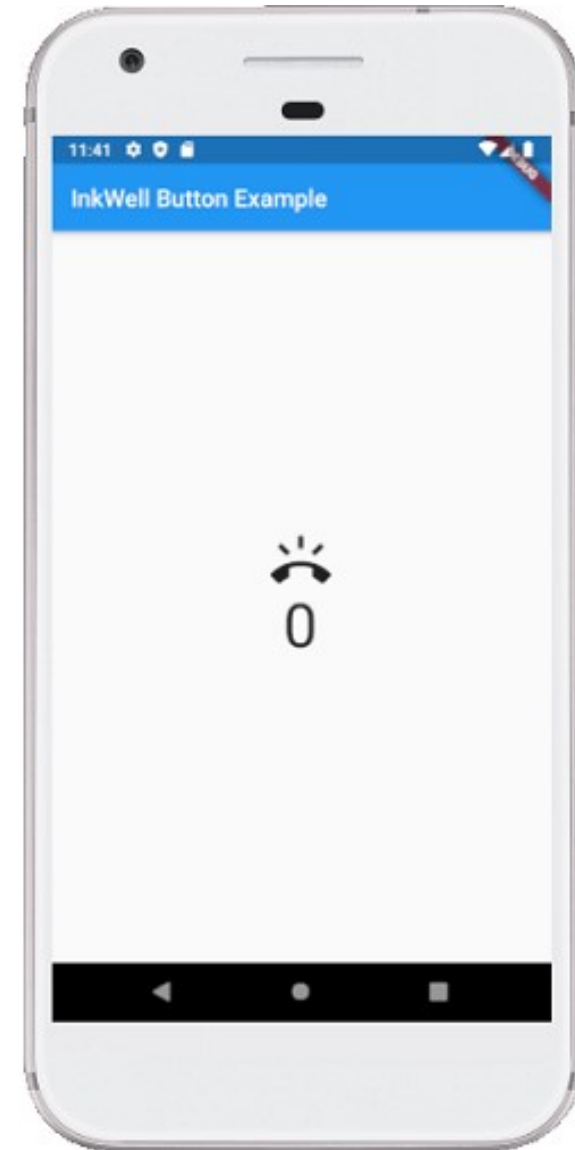
class _MyAppState extends State<MyApp> {
  int _volume = 0;

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        appBar: AppBar(
          title: Text('InkWell Button Example'),
        ),
        body: Center(
          child: new Column(
            mainAxisAlignment: MainAxisAlignment.center,
```

```
children: <Widget>[
  InkWell(
    splashColor: Colors.green,
    highlightColor: Colors.blue,
    child: Icon(Icons.ring_volume, size: 50),
    onTap: () {
      setState(() {
        _volume += 2;
      });
    },
  ),
  Text (
    _volume.toString(),
    style: TextStyle(fontSize: 50)
  ),
],
),
),
);
}
```

Output:

Run the application in android emulator, and it will give the UI similar to the following screenshot. Every time we press the ring volume button, it will increase the volume by 2.



7. PopupMenu Button

It is a button that **displays the menu** when it is pressed and then calls the **onSelected** method the menu is dismissed. It is because the item from the multiple options is selected. This button contains a text and an image. It will mainly use with **Settings** menu to list all options. It helps in making a great user experience.

Example:

```
import 'package:flutter/material.dart';

void main() { runApp(MyApp());}

class MyApp extends StatefulWidget {
  @override
  _MyAppState createState() => _MyAppState();
}

class _MyAppState extends State<MyApp> {
  Choice _selectedOption = choices[0];

  void _select(Choice choice) {
    setState() {
      _selectedOption = choice;
    };
  }

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
```

```
appBar: AppBar(  
  title: const Text('PopupMenu Button Example'),  
  actions: <Widget>[  
    PopupMenuButton<Choice>(  
      onSelect: _select,  
      itemBuilder: (BuildContext context) {  
        return choices.skip(0).map((Choice choice) {  
          return PopupMenuItem<Choice>(  
            value: choice,  
            child: Text(choice.name),  
          );  
        }).toList();  
      },  
    ),  
  ],  
  body: Padding(  
    padding: const EdgeInsets.all(10.0),  
    child: ChoiceCard(choice: _selectedOption),  
  ),  
),  
);
```



```
}  
}  
  
class Choice {  
  const Choice({this.name, this.icon});  
  final String name;  
  final IconData icon;  
}  
  
const List<Choice> choices = const <Choice>[  
  const Choice(name: 'Wi-Fi', icon: Icons.wifi),  
  const Choice(name: 'Bluetooth', icon: Icons.bluetooth),  
  const Choice(name: 'Battery', icon: Icons.battery_alert),  
  const Choice(name: 'Storage', icon: Icons.storage),  
];  
  
class ChoiceCard extends StatelessWidget {  
  const ChoiceCard({Key key, this.choice}) : super(key: key);
```

```
final Choice choice;

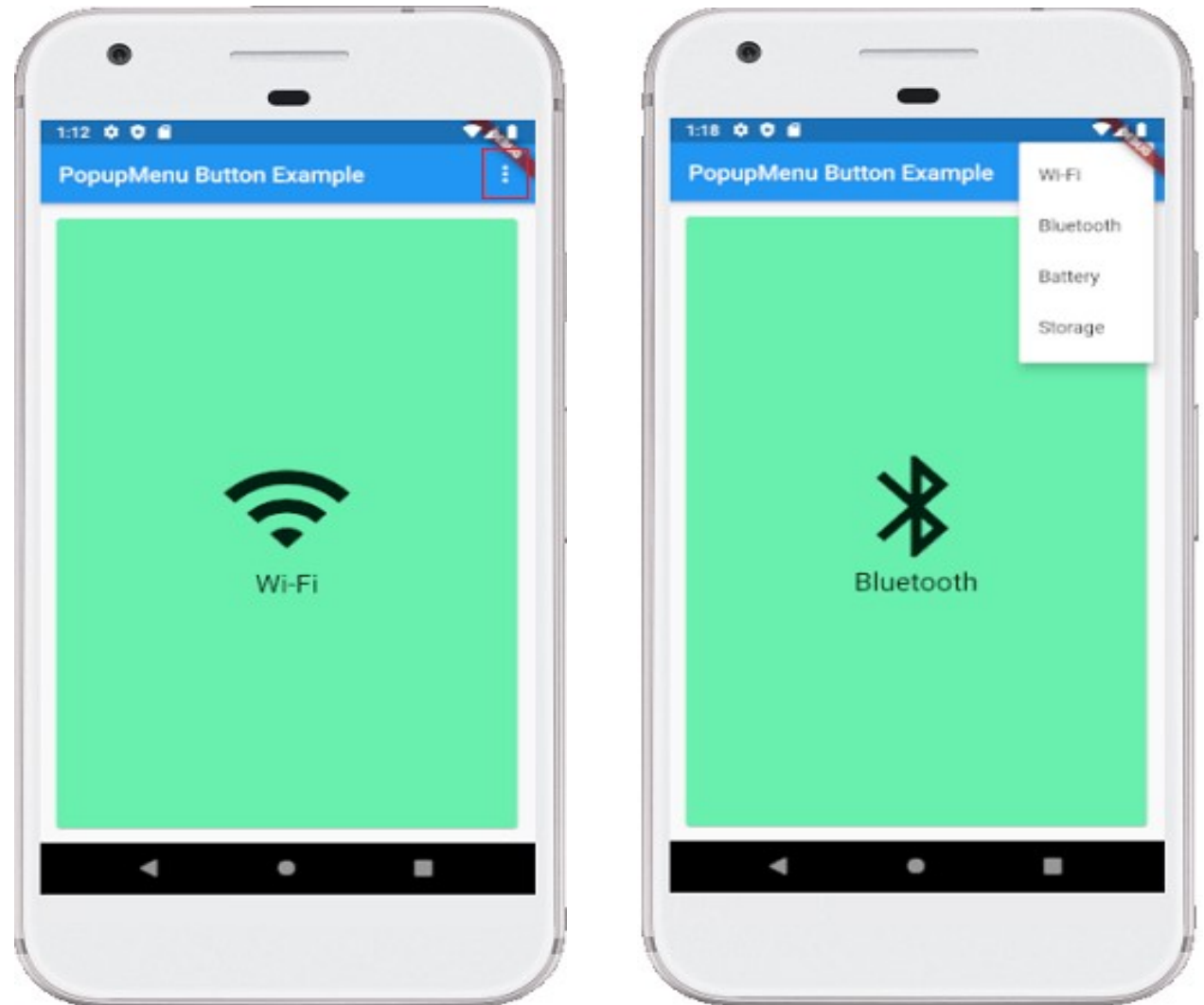
@override
Widget build(BuildContext context) {
  final TextStyle textStyle = Theme.of(context).textTheme.headline;
  return Card(
    color: Colors.greenAccent,
    child: Center(
      child: Column(
        mainAxisAlignment: MainAxisAlignment.min,
        crossAxisAlignment: CrossAxisAlignment.center,
        children: <Widget>[
          Icon(choice.icon, size: 115.0, color: textStyle.color),
          Text(choice.name, style: textStyle),
        ],
      ),
    ),
  );
}
```

Output:

Run the application in android emulator, and it will give the UI similar to the following screenshot.

When we click the **three dots** shown at the top left corner of the screen, it will pop up the multiple options.

Here, we can select any option, and it will keep it in the card, as shown in the second image.



8. Outline Button

It is similar to the flat button, but it contains a thin grey rounded rectangle border.

Its outline border is defined by the shape attribute.

Example:

```
import 'package:flutter/material.dart';

void main() {
  runApp(MyApp());
}

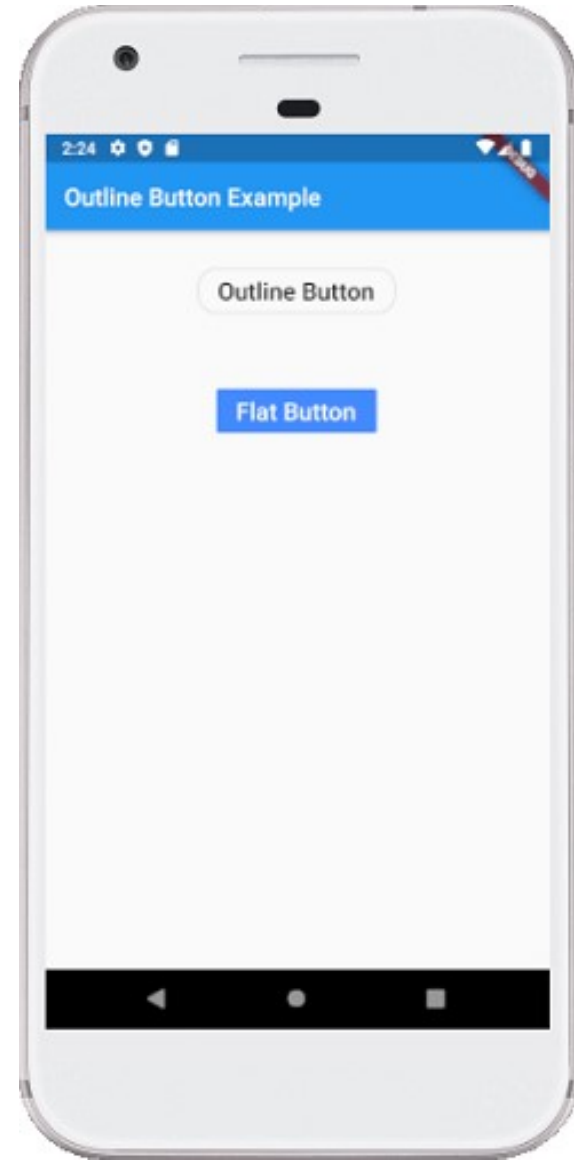
class MyApp extends StatefulWidget {
  @override
  _MyAppState createState() => _MyAppState();
}
```

```
class _MyAppState extends State<MyApp> {  
  @override  
  Widget build(BuildContext context) {  
    return MaterialApp(  
      home: Scaffold(  
        appBar: AppBar(  
          title: Text('Outline Button Example'),  
        ),  
        body: Center(child: Column(children: <Widget>[  
          Container(  
            margin: EdgeInsets.all(25),  
            child: OutlineButton(  
              child: Text("Outline Button", style: TextStyle(fontSize: 20.0)),  
              highlightedBorderColor: Colors.red,  
              shape: RoundedRectangleBorder(  
                borderRadius: BorderRadius.circular(15)),  
              onPressed: () {},  
            ),  
          ],  
        ),  
      ),  
    ),  
  ),  
}
```

```
Container(  
  margin: EdgeInsets.all(25),  
  child: FlatButton(  
    child: Text('Flat Button', style: TextStyle(fontSize: 20.0)),  
    color: Colors.blueAccent,  
    textColor: Colors.white,  
    onPressed: () {},  
  ),  
,  
]  
)  
,  
);  
}  
}
```

Output:

Run the application in android emulator, and it will give the UI similar to the following screenshot.



Button Bar

Flutter provides the flexibility to **arrange the buttons in a bar or a row**. `ButtonBar` widget contains three properties: **alignment**, **children**, and **mainAxisSize**.

- Alignment is used to present the aligning option to the entire button bar widget.
- Children attribute is used to take the number of buttons in a bar.
- `mainAxisSize` attribute is used to provide the horizontal space for the button bar.

Example:

```
import 'package:flutter/material.dart';

void main() {
  runApp(MaterialApp( home: MyApp()));
}

class MyApp extends StatefulWidget {
  @override
  _State createState() => _State();
}

class _State extends State<MyApp> {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text('Flutter ButtonBar Example'),
      ),
      body: Padding(
        padding: EdgeInsets.all(10),
        child: Column(
```

```
children: <Widget>[
  Padding(
    padding: EdgeInsets.all(15),
    child: new ButtonBar(
      mainAxisAlignment: MainAxisAlignment.min,
      children: <Widget>[
        RaisedButton(
          child: new Text('Javatpoint'),
          color: Colors.lightGreen,
          onPressed: () {/** */},
        ),
        FlatButton(
          child: Text('Flutter'),
          color: Colors.lightGreen,
          onPressed: () {/** */},
        ),
        FlatButton(
          child: Text('MySQL'),
          color: Colors.lightGreen,
          onPressed: () {/** */},
```

```
        ),  
      ],  
    ),  
  ),  
  ],  
)  
);  
}  
}
```

Output:

Run the application in android emulator, and it will give the UI similar to the following screenshot. Here, we can see that the three buttons are placed in a horizontal bar or row.





Mobile Applications Developments



COMPUTER SYSTEMS ENGINEERING

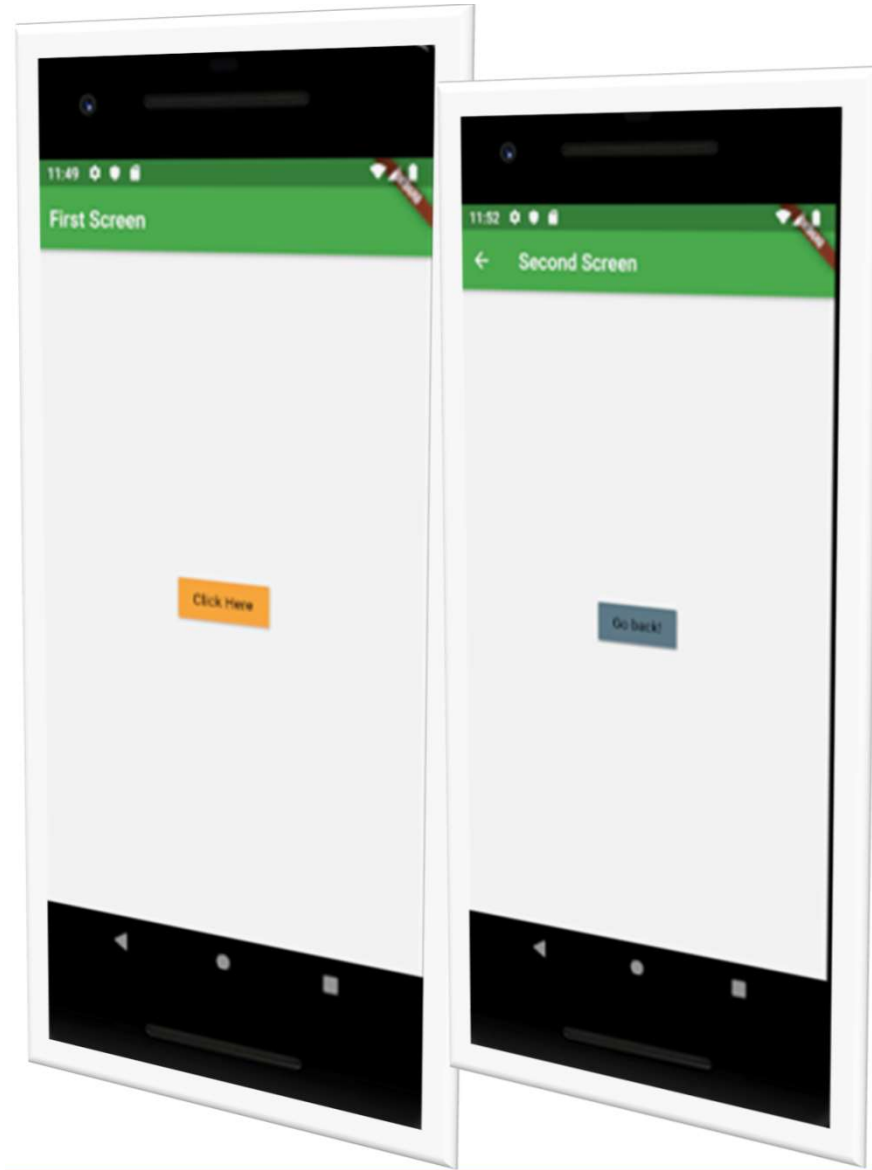
LECTURER: AHMAD SHAQALIEH

SECOND SEMESTER 2025

LECTURE 9

A solid orange horizontal bar spanning the width of the slide at the bottom.

Flutter Navigation and Routing



Navigation and routing are some of the core concepts of all mobile application, which allows the user to move between different pages.

We know that every mobile application contains several screens for displaying different types of information.

For example, an app can have a screen that contains various products.

When the user taps on that product, immediately it will display detailed information about that product.

A solid orange horizontal bar spanning the width of the slide at the bottom.

In Flutter, the screens and pages are known as **routes**, and these routes are just a widget.

In Android, a route is similar to an **Activity**, whereas, in iOS, it is equivalent to a **ViewController**.

In any mobile app, navigating to different pages defines the workflow of the application, and the way to handle the navigation is known as **routing**.

Flutter provides a basic routing class **MaterialPageRoute** and two methods **Navigator.push()** and **Navigator.pop()** that shows how to navigate between two routes.

The following steps are required to start navigation in your application.



Step 1: First, you need to create two routes.

Step 2: Then, navigate to one route from another route by using the `Navigator.push()` method.

Step 3: Finally, navigate to the first route by using the `Navigator.pop()` method.

A solid orange horizontal bar spanning the width of the slide at the bottom.

Create two routes

we are going to create two routes for navigation. In both routes, we have created only a **single button**.

When we tap the button on the first page, it will navigate to the second page.

Again, when we tap the button on the second page, it will return to the first page.

The below code snippet creates two routes in the Flutter application.



```
class FirstRoute extends StatelessWidget {  
  @override  
  Widget build(BuildContext context) {  
    return Scaffold(  
      appBar: AppBar(  
        title: Text('First Route'),  
      ),  
      body: Center(  
        child: RaisedButton(  
          child: Text('Open route'),  
          onPressed: () {  
            // Navigate to second route when tapped.  
          },  
        ),  
      ),  
    );  
  }  
}
```

```
class SecondRoute extends StatelessWidget {  
  @override  
  Widget build(BuildContext context) {  
    return Scaffold(  
      appBar: AppBar(  
        title: Text("Second Route"),  
      ),  
      body: Center(  
        child: RaisedButton(  
          onPressed: () {  
            // Navigate back to first route when tapped.  
          },  
          child: Text('Go back!'),  
        ),  
      ),  
    );  
  }  
}
```

Navigate to the second route using `Navigator.push()` method

The `Navigator.push()` method is used to navigate/switch to a new route/page/screen.

Here, the **`push()`** method adds a page/route on the stack and then manage it by using the **`Navigator`**.

Again we use `MaterialPageRoute` class that allows transition between the routes using a platform-specific animation.

The below code explain the use of the `Navigator.push()` method.

```
// Within the `FirstRoute` widget
```

```
onPressed: () {
```

```
  Navigator.push(
```

```
    context,
```

```
    MaterialPageRoute(builder: (context) => SecondRoute()),
```

```
  );
```

```
}
```

Return to the first route using Navigator.pop() method

we need to use Navigator.pop() method to close the second route and return to the first route.

The **pop()** method allows us to remove the current route from the stack, which is managed by the Navigator.

To implement a return to the original route, we need to update the **onPressed()** callback method in the SecondRoute widget as below code snippet:

```
// Within the SecondRoute widget
onPressed: () {
  Navigator.pop(context);
}
```

Now, let us see the full code to implement the navigation between two routes. First, create a Flutter project and insert the following code in the **main.dart** file.

```
import 'package:flutter/material.dart';

void main() {
  runApp(MaterialApp(
    title: 'Flutter Navigation',
    theme: ThemeData(
      // This is the theme of your application.
      primarySwatch: Colors.green,
    ),
    home: FirstRoute(),
  ));
}
```



```
class FirstRoute extends StatelessWidget {  
  @override  
  Widget build(BuildContext context) {  
    return Scaffold(  
      appBar: AppBar(  
        title: Text("First Screen"),  
      ),  
      body: Center(  
        child: RaisedButton(  
          child: Text('Click Here'),  
          color: Colors.orangeAccent,  
          onPressed: () {  
            Navigator.push(  
              context,  
              MaterialPageRoute(builder: (context) => SecondRoute()),  
            );  
          },  
        ),  
      ),  
    );  
  }  
}
```

```
class SecondRoute extends StatelessWidget {  
  @override  
  Widget build(BuildContext context) {  
    return Scaffold(  
      appBar: AppBar(  
        title: Text("Second Screen"),  
      ),  
      body: Center(  
        child: RaisedButton(  
          color: Colors.blueGrey,  
          onPressed: () {  
            Navigator.pop(context);  
          },  
          child: Text('Go back'),  
        ),  
      ),  
    );  
  }  
}
```

Output:

It is the first screen that contains only a single button.



Click the button **Click Here**, and you will navigate to a second screen as like image.

Next, when you click on the button **Go Back**, you will return to the first page.



Navigation with Named Routes

We have learned how to navigate to a new screen by creating a new route and manage it by using the Navigator.

The Navigator maintains the stack-based history of routes.

If there is a need to navigate to the same screen in many parts of the app, this approach is not beneficial because it results in **code duplication**.

The solution to this problem can be removed by defining the named routes and can use the **named routes** for navigation.

We can work with named routes by using the **Navigator.pushNamed()** function.

This function takes two required arguments (build context and string) and one optional argument.

Also, we know about the `MaterialPageRoute`, which is responsible for page transition. If we do not use this, then it is difficult to change the page.

The following steps are necessary, which demonstrate how to use named routes.

Step 1: First, we need to create two screens. The following code creates the two screens in our app.

```
class HomeScreen extends StatelessWidget {  
  @override  
  Widget build(BuildContext context) {  
    return Scaffold(  
      appBar: AppBar(  
        title: Text('Home Screen'),  
      ),  
    );  
  }  
}
```

```
body: Center(  
  child: RaisedButton(  
    child: Text('Click Here'),  
    color: Colors.orangeAccent,  
    onPressed: () {  
      //  
    },  
  ),  
,  
);  
}  
}  
  
class SecondScreen extends StatelessWidget {  
  @override  
  Widget build(BuildContext context) {  
    return Scaffold(  
      appBar: AppBar(  
        title: Text("Second Screen"),  
      ),  
    ),  
  ),  
}
```

```
body: Center(  
  child: RaisedButton(  
    color: Colors.blueGrey,  
    onPressed: () {  
      //  
    },  
    child: Text('Go back!'),  
  ),  
,  
);  
}
```

Step 2: Define the routes.

In this step, we have to define the routes.

The `MaterialApp` constructor is responsible for defining the initial route and other routes themselves.

Here the initial route tells the start of the page and `routes` property defines the available named routes and widgets. The following code explains it more clearly.



```
MaterialApp(  
  title: 'Named Route Navigation',  
  theme: ThemeData(  
    // This is the theme of your application.  
    primarySwatch: Colors.green,  
  ),  
  // It start the app with the "/" named route. In this case, the app starts  
  // on the HomeScreen widget.  
  initialRoute: '/',  
  routes: {  
    // When navigating to the "/" route, build the HomeScreen widget.  
    '/': (context) => HomeScreen(),  
    // When navigating to the "/second" route, build the SecondScreen widget.  
    '/second': (context) => SecondScreen(),  
  },  
));
```

Step 3: Navigate to the second screen using the `Navigator.pushNamed()` function.

In this step, we need to call `Navigator.pushNamed()` method for navigation.

For this, we need to update an `onPressed()` callback in the build method of **HomeScreen** like below code snippets.

```
onPressed: () {  
  // Navigate to the second screen by using the named route.  
  Navigator.pushNamed(context, '/second');  
}
```

Step 4: Use a `Navigator.pop()` function to return to the first screen.

It is the final step, where we will use `Navigator.pop()` method to return on the first screen.

```
onPressed: () {  
  Navigator.pop(context);  
},
```

Let us see the full code of the above explanation in the Flutter project and run it in the emulator to get the output.



```
import 'package:flutter/material.dart';

void main() {
  runApp( MaterialApp(
    title: 'Named Route Navigation',
    theme: ThemeData(
      // This is the theme of your application.
      primarySwatch: Colors.green,
    ),
    // Start the app with the "/" named route. In this case, the app starts
    // on the FirstScreen widget.
    initialRoute: '/',
    routes: {
      // When navigating to the "/" route, build the FirstScreen widget.
      '/': (context) => HomeScreen(),
      // When navigating to the "/second" route, build the SecondScreen widget.
      '/second': (context) => SecondScreen(),
    },
  ));
}
```

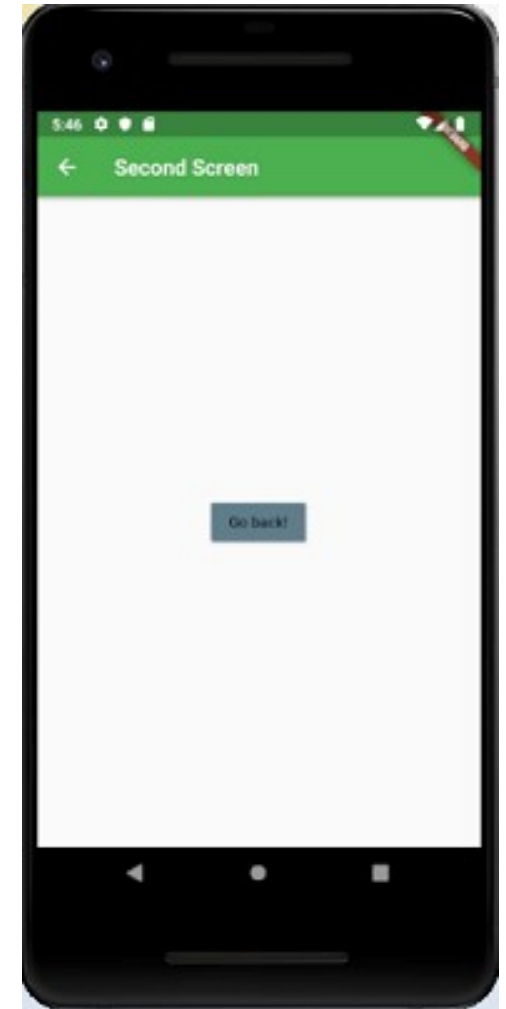
```
class HomeScreen extends StatelessWidget {  
  @override  
  Widget build(BuildContext context) {  
    return Scaffold(  
      appBar: AppBar(  
        title: Text('Home Screen'),  
      ),  
      body: Center(  
        child: RaisedButton(  
          child: Text('Click Here'),  
          color: Colors.orangeAccent,  
          onPressed: () {  
            Navigator.pushNamed(context, '/second');  
          },  
        ),  
      ),  
    );  
  }  
}
```

```
class SecondScreen extends StatelessWidget {  
  @override  
  Widget build(BuildContext context) {  
    return Scaffold(  
      appBar: AppBar(  
        title: Text("Second Screen"),  
      ),  
      body: Center(  
        child: RaisedButton(  
          color: Colors.blueGrey,  
          onPressed: () {  
            Navigator.pop(context);  
          },  
          child: Text('Go back!'),  
        ),  
      ),  
    );  
  }  
}
```

Output



Click the button **Click Here**, and you will navigate to a second screen as below image. Next, when you click on the button **Go Back**, it will return to the Home Page.





Mobile Applications Developments



COMPUTER SYSTEMS ENGINEERING

LECTURER: AHMAD SHAQALIEH

SECOND SEMESTER 2025

LECTURE 10

A solid orange horizontal bar spanning the width of the slide at the bottom.

Flutter Database Concepts



A database is an organized collection of data, which supports the storage and manipulation of data and accessed electronically from a computer system.

We can organize data into rows, columns, tables, and indexes.

It makes data management easy.

We can store many things in the database, like name, age, picture, image, file, pdf, etc.

Flutter provides many packages to work with the database. The most used and popular packages are:

- ❑ **sqlite database:** It allows to access and manipulate SQLite database.
- ❑ **Firebase database:** It will enable you to access and manipulate the cloud database.

SQLite Database

SQLite is a popular database software library that provides a relational database management system for local/client storage.

It is a light-weight and time-tested database engine and contains features like self-contained, server-less, zero-configuration, transactional SQL database engine.

Flutter SDK does not support SQLite directly.

Instead, it provides a plugin **sqflite**, which performs all operations on the database as similar to the SQLite library.

The sqflite provides most of the core functionalities related to the database are as follows:

-
- ❑ It creates or opens the SQLite database.
 - ❑ It can execute SQL statements easily.
 - ❑ It also provides an advanced query method to get information from the SQLite database.

Let us see step by step how we can store and fetch data in the Flutter.

Step 1: First, create a new project in Android Studio and add the dependencies in **pubspec.yaml** file.

```
dependencies:  
  flutter:  
    sdk: flutter  
  sqflite: any  
  path_provider: any
```

-
- ❑ The `sqlite` package provides classes and functions to interact with the SQLite database.
 - ❑ The **`path_provider`** package provides functions to define the location of your database on the local system, such as **`TemporaryDirectory`** and **`ApplicationDocumentsDirectory`**.

Step 2: Create a model class. In this step, we have to define the data that needs to be stored before creating a table to store information. The following code explains it easily.

```
class Book {  
    final int id;  
    final String title;  
    final int price;  
  
    Book({this.id, this.title, this.price});  
}
```

Step 3: Open the database. Here, we need to open the connection to the database. It requires two steps:

- 1) Set the path to the database by using the **getDatabasePath()** method and combined it with the path package.
- 2) Use **openDatabase()** function to open the database.

```
// It allows to open the database and store the reference.  
final Future<Database> database = openDatabase(  
    join(await getDatabasesPath(), 'book_database.db'),  
);
```

Step 4: Create the table.

In this step, we have to create a table that stores information about the books.

Here, we are going to create a table named books, which contains the **id**, **title**, and **price** of the books.

They are represented as three columns in the book table.



```
final Future<Database> database = openDatabase(  
    join(await getDatabasesPath(), 'book_database.db'),  
    // When you create a database, it also needs to create a table to store books.  
    onCreate: (db, version) {  
        // Run the CREATE TABLE statement.  
        return db.execute(  
            "CREATE TABLE books(id INTEGER PRIMARY KEY, title TEXT, price INTEGER)",  
        );  
    },  
    // Set the version to perform database upgrades and downgrades.  
    version: 1,  
);
```

Step 5: Insert a Book into the database. Here, you have to store information on the table about the various books. Inserting a book into the table involves two steps:

- ❑ Convert the Book into a Map
- ❑ Uses insert() method

The following code explains it more clearly.

// Update the Book class.

```
class Book{
```

```
    final int id,
```

```
    final String title;
```

```
    final int price;
```

```
    Book({this.id, this.title, this.price});
```

// It converts a Book into a Map. The keys correspond to the names of the columns in the database.

```
Map<String, dynamic> toMap() {
```

```
    return {
```

```
        'id': id,
```

```
        'title': title,
```

```
        'price': price,
```

```
    };
```

```
}
```

```
}
```

```
Future<void> insertBook(Book book) async {  
    final Database db = await database;  
    await db.insert(  
        'books',  
        book.toMap(),  
        conflictAlgorithm: ConflictAlgorithm.replace,  
    );  
}
```

// Create a Book and add it to the books table.

```
final b1 = Book(  
    id: 0,  
    title: 'Let Us C',  
    price: 350,  
);  
  
await insertBook(b1);
```

Step 6: Retrieve the list of books. Now, we have stored the book into the database, and you can use a query to retrieve a particular book or list of all books. It involves two steps:

- ❑ Run a query that returns `List<Map>`.
- ❑ Convert the `List<Map>` into the `List<Book>`.

You can see the following code to understand it easily.

```
// This method retrieves all the books from the books table.
Future<List<Book>> books() async {
  final Database db = await database;

  // Use query for all Books.
  final List<Map<String, dynamic>> maps = await db.query('books');

  return List.generate(maps.length, (i) {
    return Book(
      id: maps[i]['id'],
      title: maps[i]['title'],
      price: maps[i]['price'],
    );
  });
}

// It prints all the books.
print(await books());
```

Step 7: Update a Book in the database. You can use an `update()` method to update the book that you want. It involves two steps:

- ❑ Convert Book into the Map.
- ❑ Then, use where clause to update the book.

You can see the following code to understand it.


```
Future<void> updateBook(Book book) async {  
  final db = await database;  
  
  // Update the given Book.  
  await db.update(  
    'books',  
    book.toMap(),  
    // It ensure the matching id of a Book.  
    where: "id = ?",  
    whereArgs: [book.id],  
  );  
}
```

```
// Update b1 price.  
await updateBook(Book(  
  id: 0,  
  title: 'Let Us C',  
  price: 420,  
));  
  
// Print the updated results.  
print(await books());
```

Step 8: Delete a Book from the database. You can use the delete() method to delete the database. For this, you need to make a function that takes an id and delete the database of the matching id.

```
Future<void> deleteBook(int id) async {  
  final db = await database;  
  
  // This function removes books from the database.  
  await db.delete(  
    'books',  
    where: "id = ?",  
    whereArgs: [id],  
  );  
}
```

Let us see the complete code to understand how we can create a database file with sqflite plugin.

For this, create a new Flutter project and the sqflite and path package into the pubspec.yaml file.

Next, create a new file into the lib folder and insert the following code in this file. Now, connect the database with your UI and run the code.



```
import 'dart:async';
```

```
import 'package:path/path.dart';
```

```
import 'package:sqflite/sqflite.dart';
```

```
void main() async {
```

```
  final database = openDatabase(
```

```
    join(await getDatabasesPath(), 'book_database.db'),
```

```
    onCreate: (db, version) {
```

```
      return db.execute(
```

```
        "CREATE TABLE books(id INTEGER PRIMARY KEY, title TEXT, price INTEGER)",
```

```
      );
```

```
    },
```

```
    version: 1,
```

```
  );
```

```
Future<void> insertBook(Book book) async {  
    // Get a reference to the database.  
    final Database db = await database;  
  
    await db.insert(  
        'books',  
        book.toMap(),  
        conflictAlgorithm: ConflictAlgorithm.replace,  
    );  
}  
  
Future<List<Book>> books() async {  
    final Database db = await database;  
  
    final List<Map<String, dynamic>> maps = await db.query('books');  
  
    return List.generate(maps.length, (i) {  
        return Book(  
            id: maps[i]['id'],  
            title: maps[i]['title'],  
            price: maps[i]['price'],  
        );  
    });  
}
```

```
});
```

```
}
```

```
Future<void> updateBook(Book book) async {
```

```
    final db = await database;
```

```
    await db.update(
```

```
        'books',
```

```
        book.toMap(),
```

```
        where: "id = ?",
```

```
        whereArgs: [book.id],
```

```
    );
```

```
}
```

```
Future<void> deleteBook(int id) async {
```

```
    final db = await database;
```

```
    await db.delete(
```

```
        'books',
```

```
        where: "id = ?",
```

```
        whereArgs: [id],
```

```
    );
```

```
}
```

```
var b1 = Book(  
  id: 0,  
  title: 'Let Us C',  
  price: 300,  
);  
  
await insertBook(b1);  
  
print(await books());  
  
b1 = Book(  
  id: b1.id,  
  title: b1.title,  
  price: b1.price,  
);  
await updateBook(b1);  
  
print(await books());  
  
await deleteBook(b1.id);  
  
print(await books());  
}
```



```
class Book {  
    final int id;  
    final String title;  
    final int price;  
  
    Book({this.id, this.title, this.price});  
  
    Map<String, dynamic> toMap() {  
        return {  
            'id': id,  
            'title': title,  
            'price': price,  
        };  
    }  
    @override  
    String toString() {  
        return 'Book(id: $id, title: $title, price: $price)';  
    }  
}
```



Mobile Applications Developments



COMPUTER SYSTEMS ENGINEERING

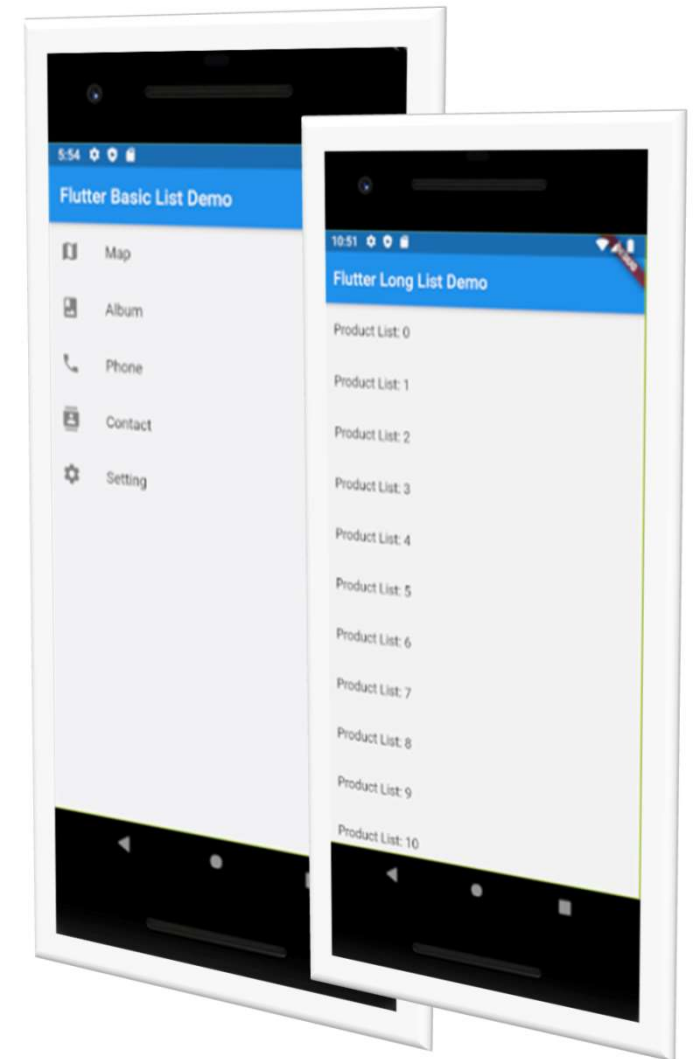
LECTURER: AHMAD SHAQALIEH

SECOND SEMESTER 2025

LECTURE 11

A solid orange horizontal bar spanning the width of the slide at the bottom.

Flutter Lists



Lists are the most popular elements of every web or mobile application.

They are made up of multiple rows of items, which include text, buttons, toggles, icons, thumbnails, and many more.

We can use it for displaying various information such as menus, tabs, or to break the monotony of pure text files.

we are going to learn how we can work with Lists in the Flutter.

Flutter allows you to work with Lists in different ways, which are given below:

- ❑ Basic Lists
- ❑ Long Lists
- ❑ Grid Lists
- ❑ Horizontal Lists

Basic Lists

Flutter includes a **ListView** widget for working with Lists, which is the fundamental concept of displaying data in the mobile apps.

The ListView is a perfect standard for displaying lists that contains only a few items.

ListView also includes **ListTile** widget, which gives more properties for the visual structure to a list of data.

The following example displays a basic list in the Flutter application.

```
import 'package:flutter/material.dart';

void main() => runApp(MyApp());

class MyApp extends StatelessWidget {
```

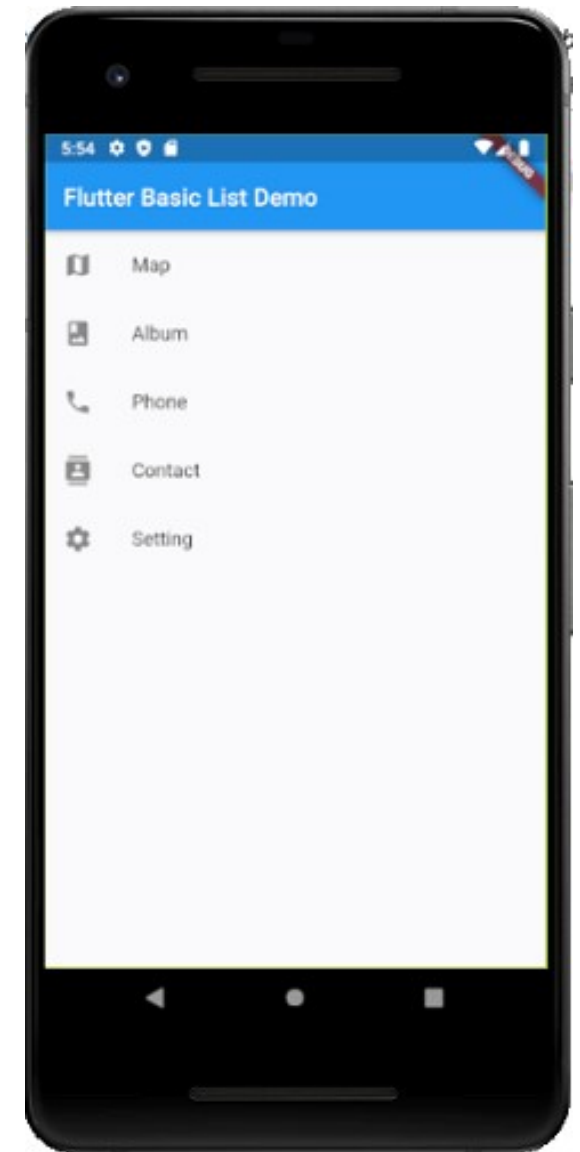
```
@override
Widget build(BuildContext context) {
  final appTitle = 'Flutter Basic List Demo';

  return MaterialApp(
    title: appTitle,
    home: Scaffold(
      appBar: AppBar(
        title: Text(appTitle),
      ),
      body: ListView(
        children: <Widget>[
          ListTile(
            leading: Icon(Icons.map),
            title: Text('Map'),
          ),
          ListTile(
            leading: Icon(Icons.photo_album),
            title: Text('Album'),
```

```
),  
  ListTile(  
    leading: Icon(Icons.phone),  
    title: Text('Phone'),  
  ),  
  ListTile(  
    leading: Icon(Icons.contacts),  
    title: Text('Contact'),  
  ),  
  ListTile(  
    leading: Icon(Icons.settings),  
    title: Text('Setting'),  
  ),  
],  
,  
,  
,  
);  
}  
}
```


Output

Now, run the app in Android Studio. You can see the following screen in your emulator or connected device.



Working with Long Lists

Sometimes you want to display a very long list in a single screen of your app, then, in that case, the above method for displaying the lists is not perfect.

To work with a list that contains a very large number of items, we need to use a **ListView.builder()** constructor.

The main difference between `ListView` and `ListView.builder` is that `ListView` creates all items at once, whereas the `ListView.builder()` constructor creates items when they are scrolled onto the screen.

Let us see the following example. Open the **main.dart** file and replace the following code.

```
import 'package:flutter/material.dart';

void main() {
  runApp(MyApp(
    products: List<String>.generate(500, (i) => "Product List: $i"),
  ));
}

class MyApp extends StatelessWidget {
  final List<String> products;

  MyApp({Key key, @required this.products}) : super(key: key);

  @override
  Widget build(BuildContext context) {
    final appTitle = 'Flutter Long List Demo';
```

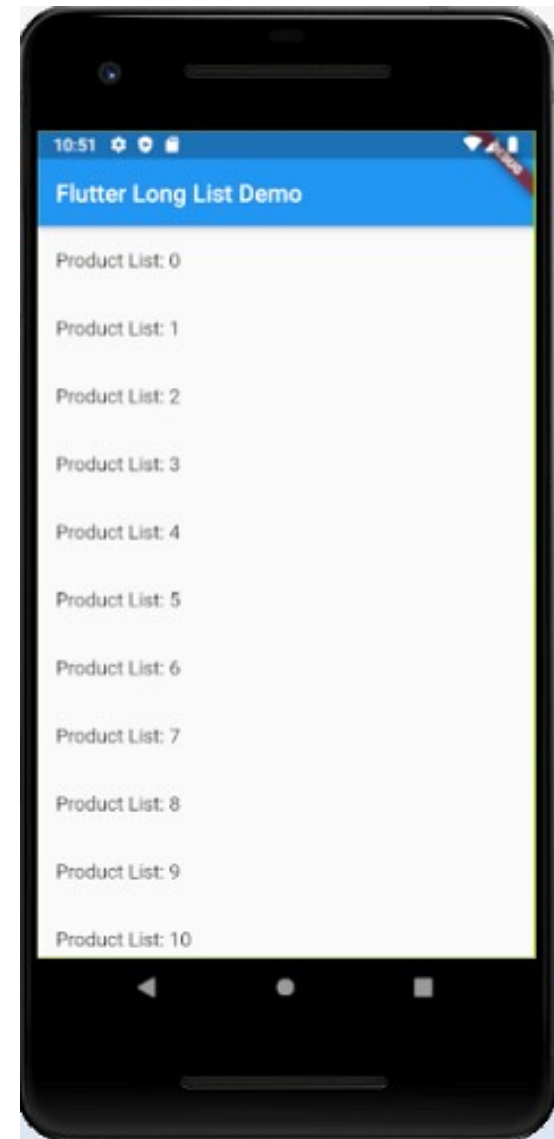
```
return MaterialApp(  
  title: appTitle,  
  home: Scaffold(  
    appBar: AppBar(  
      title: Text(appTitle),  
    ),  
    body: ListView.builder(  
      itemCount: products.length,  
      itemBuilder: (context, index) {  
        return ListTile(  
          title: Text("${products[index]}"),  
        );  
      },  
    ),  
  ),  
);  
}
```

In the above code, the **itemCount** gives how many numbers of items you want to display in a list.

The **itemBuilder** tells about where you want to return the item that you want to display.

Output

Now, run the app in Android Studio. You will get the following screen where you can see all product list by scrolling onto the screen.



Creating a Grid Lists

Sometimes we want to display the items in a grid layout rather than the normal list that comes one after next.

A **GridView** widget allows you to create a grid list in Flutter. The simplest way to create a grid is by using the **GridView.count()** constructor, which specifies the number of rows and columns in a grid.

Let us see the following example of how GridView works in Flutter for creating grid lists. Open the main.dart file and insert the following code.

```
import 'package:flutter/material.dart';

void main() {runApp(MyApp());}

class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    final appTitle = "Flutter Grid List Demo";

    return MaterialApp(
      title: appTitle,
      home: Scaffold(appBar: AppBar(
        title: Text(appTitle),
      ),
        body: GridView.count(
          crossAxisCount: 3,
          children: List.generate(choices.length, (index) {
```

```
        return Center(  
            child: SelectCard(choice: choices[index]),  
        );  
    }  
)  
)  
);  
}  
}
```

```
class Choice {  
    const Choice({this.title, this.icon});  
    final String title;  
    final IconData icon;  
}
```



```
}
```

```
const List<Choice> choices = const <Choice>[  
  const Choice(title: 'Home', icon: Icons.home),  
  const Choice(title: 'Contact', icon: Icons.contacts),  
  const Choice(title: 'Map', icon: Icons.map),  
  const Choice(title: 'Phone', icon: Icons.phone),  
  const Choice(title: 'Camera', icon: Icons.camera_alt),  
  const Choice(title: 'Setting', icon: Icons.settings),  
  const Choice(title: 'Album', icon: Icons.photo_album),  
  const Choice(title: 'WiFi', icon: Icons.wifi),  
  const Choice(title: 'GPS', icon: Icons.gps_fixed),  
];
```

```
class SelectCard extends StatelessWidget {  
  const SelectCard({Key key, this.choice}) : super(key: key);  
  final Choice choice;
```

```
@override
```

```
Widget build(BuildContext context) {  
  final TextStyle textStyle = Theme.of(context).textTheme.display1;  
  return Card(  
    color: Colors.lightGreenAccent,  
    child: Center(child: Column(  
      mainAxisAlignment: MainAxisAlignment.min,  
      crossAxisAlignment: CrossAxisAlignment.center,  
      children: <Widget>[  
        Expanded(child: Icon(choice.icon, size:50.0, color: textStyle.color)),  
        Text(choice.title, style: textStyle),  
      ]  
    ),  
  );  
}
```

Output

Now, run the app in Android Studio. You can see the following screen in your Android Emulator.



Creating a Horizontal List

The ListView widget also supports horizontal lists.

Sometimes we want to create a list that can scroll horizontally rather than vertically. In that case, ListView provides the horizontal **scrollDirection** that overrides the vertical direction.

The following example explains it more clearly.

Open the main.dart file and replace the following code.

```
import 'package:flutter/material.dart';

void main() => runApp(MyApp());

class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    final title = 'Flutter Horizontal Demo List';

    return MaterialApp(
      title: title,
      home: Scaffold(
        appBar: AppBar(
          title: Text(title),
        ),
        body: Container(
          margin: EdgeInsets.symmetric(vertical: 25.0),
          height: 150.0,
          child: ListView(
            scrollDirection: Axis.horizontal,
            children: <Widget>[
```

```
Container(  
  width: 150.0,  
  color: Colors.blue,  
  child: new Stack(  
    children: <Widget>[  
      ListTile(  
        leading: Icon(Icons.home),  
        title: Text('Home'),  
      ),  
    ],  
  ),  
)  
Container(  
  width: 148.0,  
  color: Colors.green,  
  child: new Stack(  
    children: <Widget>[  
      ListTile(  
        leading: Icon(Icons.camera_alt),  
        title: Text('Camera'),
```

```
    ),  
    ],  
  ),  
),  
Container(  
  width: 148.0,  
  color: Colors.yellow,  
  child: new Stack(  
    children: <Widget>[  
      ListTile(  
        leading: Icon(Icons.phone),  
        title: Text('Phone'),  
      ),  
    ],  
  ),  
),  
Container(  
  width: 148.0,  
  color: Colors.red,  
  child: new Stack(  
    children: <Widget>[
```

```
ListTile(  
  leading: Icon(Icons.map),  
  title: Text("Map"),  
),  
],  
),  
),  
Container(  
  width: 148.0,  
  color: Colors.orange,  
  child: new Stack(  
    children: <Widget>[  
      ListTile(  
        leading: Icon(Icons.settings),  
        title: Text("Setting"),  
      ),  
    ],  
  ),  
),  
),  
);  
}
```


Output

Now, run the app in Android Studio. It will give the following screen where you can scroll horizontally to see all the lists.





Mobile Applications Developments



COMPUTER SYSTEMS ENGINEERING

LECTURER: AHMAD SHAQALIEH

SECOND SEMESTER 2025

LECTURE 12

A solid orange horizontal bar spanning the width of the slide at the bottom.

Flutter Forms



Forms are an integral part of all modern mobile and web applications.

It is mainly used to interact with the app as well as gather information from the users.

They can perform many tasks, which depend on the nature of your business requirements and logic, such as authentication of the user, adding user, searching, filtering, ordering, booking, etc.

A form can contain text fields, buttons, checkboxes, radio buttons, etc.

A solid orange horizontal bar spanning the width of the slide at the bottom.

Creating Form

Flutter provides a **Form widget** to create a form.

The form widget acts as a container, which allows us to group and validate the multiple form fields.

When you create a form, it is necessary to provide the **GlobalKey**.

This key uniquely identifies the form and allows you to do any validation in the form fields.

The form widget uses child widget **TextFormField** to provide the users to enter the text field.

This widget renders a material design text field and also allows us to display validation errors when they occur.



Let us create a form. First, create a Flutter project and replace the following code in the main.dart file.

In this code snippet, we have created a custom class named **MyCustomForm**.

Inside this class, we define a global key as **_formKey**.

This key holds a **FormState** and can use to retrieve the form widget.

Inside the **build** method of this class, we have added some custom style and use the TextFormField widget to provide the form fields such as name, phone number, date of birth, or just a normal field.

Inside the TextFormField, we have used **InputDecoration** that provides the look and feel of your form properties such as borders, labels, icons, hint, styles, etc.

Finally, we have added a **button** to submit the form.



```
import 'package:flutter/material.dart';

void main() => runApp(MyApp());

class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    final appTitle = 'Flutter Form Demo';
    return MaterialApp(
      title: appTitle,
      home: Scaffold(
        appBar: AppBar(
          title: Text(appTitle),
        ),
        body: MyCustomForm(),
      ),
    );
  }
}
```



```
// Create a Form widget.  
class MyCustomForm extends StatefulWidget {  
  @override  
  MyCustomFormState createState() {  
    return MyCustomFormState();  
  }  
}  
  
// Create a corresponding State class. This class holds data related to the form.  
class MyCustomFormState extends State<MyCustomForm> {  
  // Create a global key that uniquely identifies the Form widget  
  // and allows validation of the form.  
  final _formKey = GlobalKey<FormState>();  
  
  @override  
  Widget build(BuildContext context) {  
    // Build a Form widget using the _formKey created above.  
    return Form(  
      key: _formKey,  
      child: Column(  
        crossAxisAlignment: CrossAxisAlignment.start,
```

```
children: <Widget>[
  TextFormField(
    decoration: const InputDecoration(
      icon: const Icon(Icons.person),
      hintText: 'Enter your name',
      labelText: 'Name',
    ),
  ),
  TextFormField(
    decoration: const InputDecoration(
      icon: const Icon(Icons.phone),
      hintText: 'Enter a phone number',
      labelText: 'Phone',
    ),
  ),
  TextFormField(
    decoration: const InputDecoration(
      icon: const Icon(Icons.calendar_today),
      hintText: 'Enter your date of birth',
```

```
    labelText: 'Dob',  
  ),  
),  
new Container(  
  padding: const EdgeInsets.only(left: 150.0, top: 40.0),  
  child: new RaisedButton(  
    child: const Text('Submit'),  
    onPressed: null,  
  )),  
],  
),  
);  
}  
}
```

Output

Now, run the app, you can see the following screen in your Android Emulator. This form contains three field name, phone number, date of birth, and submit button.



Form validation

Validation is a method, which allows us to correct or confirms a certain standard. It ensures the authentication of the entered data.

Validating forms is a common practice in all digital interactions. To validate a form in a flutter, we need to implement mainly three steps.

Step 1: Use the Form widget with a global key.

Step 2: Use TextFormField to give the input field with validator property.

Step 3: Create a button to validate form fields and display validation errors.

Let us understand it with the following example.

In the above code, we have to use **validator()** function in the TextFormField to validate the input properties.

If the user gives the wrong input, the validator function returns a string that contains an **error message**; otherwise, the validator function return **null**. In the validator function, make sure that the TextFormField is not empty.

Otherwise, it returns an error message.

The validator() function can be written as below code snippets:



```
validator: (value) {  
  if (value.isEmpty) {  
    return 'Please enter some text';  
  }  
  return null;  
},
```

Now, open the **main.dart** file and add `validator()` function in the `TextFormField` widget. Replace the following code with the `main.dart` file.

```
import 'package:flutter/material.dart';

void main() => runApp(MyApp());

class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    final appTitle = 'Flutter Form Demo';
    return MaterialApp(
      title: appTitle,
      home: Scaffold(
        appBar: AppBar(
          title: Text(appTitle),
        ),
        body: MyCustomForm(),
      ),
    );
  }
}
```



```
// Create a Form widget.  
class MyCustomForm extends StatefulWidget {  
  @override  
  MyCustomFormState createState() {  
    return MyCustomFormState();  
  }  
}  
  
// Create a corresponding State class, which holds data related to the form.  
class MyCustomFormState extends State<MyCustomForm> {  
  // Create a global key that uniquely identifies the Form widget  
  // and allows validation of the form.  
  final _formKey = GlobalKey<FormState>();  
  
  @override  
  Widget build(BuildContext context) {  
    // Build a Form widget using the _formKey created above.  
    return Form(  
      key: _formKey,  
      child: Column(  
        crossAxisAlignment: CrossAxisAlignment.start,
```

```
children: <Widget>[
  TextFormField(
    decoration: const InputDecoration(
      icon: const Icon(Icons.person),
      hintText: 'Enter your full name',
      labelText: 'Name',
    ),
    validator: (value) {
      if (value.isEmpty) {
        return 'Please enter some text';
      }
      return null;
    },
  ),
  TextFormField(
    decoration: const InputDecoration(
      icon: const Icon(Icons.phone),
      hintText: 'Enter a phone number',
      labelText: 'Phone',
    ),
  ),
```

```
validator: (value) {  
  if (value.isEmpty) {  
    return 'Please enter valid phone number';  
  }  
  return null;  
},  
,  
),  
TextFormField(  
  decoration: const InputDecoration(  
    icon: const Icon(Icons.calendar_today),  
    hintText: 'Enter your date of birth',  
    labelText: 'Dob',  
  ),  
  validator: (value) {  
    if (value.isEmpty) {  
      return 'Please enter valid date';  
    }  
    return null;  
  },  
,  
),
```

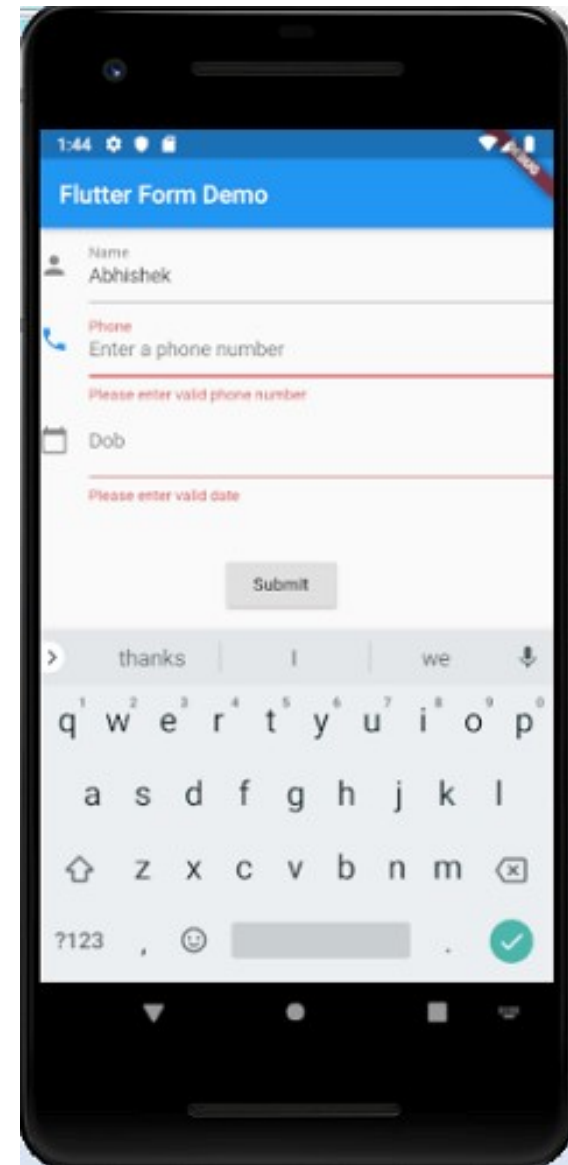
```
new Container(  
  padding: const EdgeInsets.only(left: 150.0, top: 40.0),  
  child: new RaisedButton(  
    child: const Text('Submit'),  
    onPressed: () {  
      // It returns true if the form is valid, otherwise returns false  
      if (_formKey.currentState.validate()) {  
        // If the form is valid, display a SnackBar.  
        Scaffold.of(context)  
          .showSnackBar(SnackBar(content: Text('Data is in processing.')));  
      }  
    },  
  )),  
],  
,  
);  
}
```

Output

Now, run the app. The following screen appears.



In this form, if you left any input field blank, you will get an error message like below screen.





Mobile Applications Developments



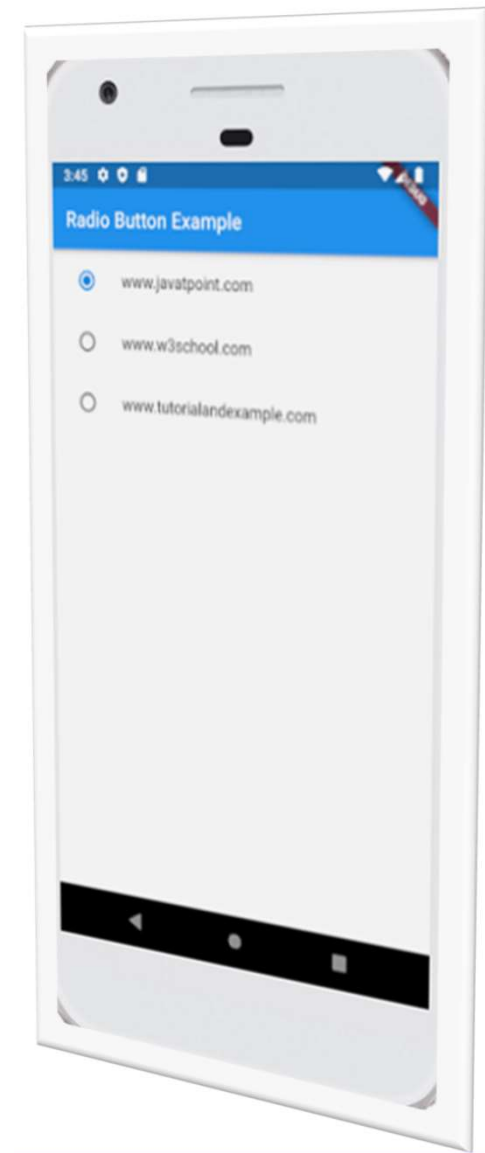
COMPUTER SYSTEMS ENGINEERING

LECTURER: AHMAD SHAQALIEH

SECOND SEMESTER 2025

LECTURE 13

Flutter Radio Button



A radio button is also known as the options button which holds the **Boolean value**.

It allows the user to choose only one option from a predefined set of options.

This feature makes it different from a checkbox where we can select more than one option and the unselected state to be restored.

We can arrange the radio button in a **group of two or more** and displayed on the screen as **circular holes** with white space (for unselected) or a dot (for selected).

We can also provide a **label** for each corresponding radio button describing the choice that the radio button represents.

A radio button can be selected by clicking the mouse on the circular hole or using a keyboard shortcut.



In this section, we are going to explain how to use radio buttons in Flutter.

Flutter allows us to use radio buttons with the help of 'Radio', 'RadioListTile', or 'ListTile' Widgets.

The flutter radio button does not maintain any state itself.

When we select any radio option, it invokes the **onChanged** callback and passing the value as a parameter.

If the value and **groupValue** match, the radio option will be selected.

Let us see how we can create radio buttons in the Flutter app through the following steps:

Step 1: Create a Flutter project in the IDE. Here, I am going to use Android Studio.



step 2: Open the project in Android Studio and navigate to the **lib** folder.

In this folder, open the **main.dart** file and create a `RadioButtonWidget` class (**Here: `MyStatefulWidget`**).

Next, we will create the `Column` widget and put three `RadioListTile` components.

Also, we will create a `Text` widget for displaying the selected item.

The `ListTile` contains the following properties:

groupValue: It is used to specify the currently selected item for the radio button group.

title: It is used to specify the radio button label.

value: It specifies the backhand value, which is represented by a radio button.

onChanged: It will be called whenever the user selects the radio button.



```
ListTile(  
  title: const Text('www.javatpoint.com'),  
  leading: Radio(  
    value: BestTutorSite.javatpoint,  
    groupValue: _site,  
    onChanged: (BestTutorSite value) {  
      setState() {  
        _site = value;  
      };  
    },  
  ),  
),
```

Let us see the complete code of the above steps.

Open the **main.dart** file and replace the following code.

Here, the Radio widgets wrapped in **ListTiles** and the currently selected text is passed into `groupValue` and maintained by the example's State.

Here, the first Radio button will be selected off because `_site` is initialized to **BestTutorSite.javatpoint**. If the second radio button is pressed, the example's State is updated with **setState**, updating `_site` to **BestTutorSite.w3schools**.

It rebuilds the button with the updated `groupValue`, and therefore it will select the second button.



```
import 'package:flutter/material.dart';

void main() => runApp(MyApp());

/// This Widget is the main application widget.
class MyApp extends StatelessWidget {
  static const String _title = 'Radio Button Example';

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: _title,
      home: Scaffold(
        appBar: AppBar(title: const Text(_title)),
        body: Center(
          child: MyStatefulWidget(),
        ),
      ),
    );
  }
}
```

```
enum BestTutorSite { javatpoint, w3schools, tutorialandexample }

class MyStatefulWidget extends StatefulWidget {
  MyStatefulWidget({Key key}) : super(key: key);

  @override
  _MyStatefulWidgetState createState() => _MyStatefulWidgetState();
}

class _MyStatefulWidgetState extends State<MyStatefulWidget> {
  BestTutorSite _site = BestTutorSite.javatpoint;

  Widget build(BuildContext context) {
    return Column(
      children: <Widget>[
        ListTile(
          title: const Text('www.javatpoint.com'),
          leading: Radio(
            value: BestTutorSite.javatpoint,
            groupValue: _site,
            onChanged: (BestTutorSite value) {
              setState() {
```



```
        _site = value;
    });
  },
),
),
ListTile(
  title: const Text('www.w3school.com'),
  leading: Radio(
    value: BestTutorSite.w3schools,
    groupValue: _site,
    onChanged: (BestTutorSite value) {
      setState() {
        _site = value;
      });
    },
  ),
),
ListTile(
  title: const Text('www.tutorialandexample.com'),
  leading: Radio(
    value: BestTutorSite.tutorialandexample,
    groupValue: _site,
    onChanged: (BestTutorSite value) {
```

```
    setState() {  
      _site = value;  
    };  
  },  
),  
],  
);  
}  
}
```

Output

When we run the app, the following output appears. Here, we have three radio buttons, and only one is selected by default. We can also select any other option.

